SISTEMA DE GESTIÓN ACADÉMICA • SERIE EDITORIAL • 1ª EDICIÓN OFICIAL

Programación de Aplicaciones Móviles I

Compendio estructurado de lecciones, fundamentos teóricos y ejercicios prácticos de ingeniería.

PLAN DE ESTUDIOS INTEGRAL

Este libro compila 4 capítulos temáticos y 19 lecciones técnicas con explicaciones conceptuales rigurosas, sintaxis de código comentada y directrices arquitectónicas para el aprendizaje universitario y profesional.

MÓDULOS

4 Capítulos

CONTENIDO

19 Lecciones

AUTOR / DOCENTE

Jhon Fredy Valencia Gómez C

Edición: 1ª Edición Oficial
Año 2026 • 20 de julio de 2026

Tabla de Contenido

Estructura secuencial interactiva: haz clic en cualquier capítulo o lección para saltar a su contenido.

CAPÍTULO 01: Inicio Ir al capítulo

CAPÍTULO 02: Git - Github Ir al capítulo

2.1 Tutorial de Git y GitHub

CAPÍTULO 03: Contenido Ir al capítulo

3.1 Clase 1: Introducción a la POO y Entorno Java
3.2 Clase 2: Clases, Objetos y Constructores
3.3 Clase 4: Relaciones entre Clases
3.4 Clase 3: Encapsulamiento y Modificadores de Acceso
3.5 Clase 5: Herencia
3.6 Clase 7 - Ejercicio Práctico
3.7 Clase 8 - Polimorfismo en Java
3.8 Clase 9 - Clases Abstractas
3.9 Clase 10 - Interfaces
3.10 Clase 13 - Gestores de Dependencias

CAPÍTULO 04: Proyecto Integrador Ir al capítulo

4.1 00_INDICE_Y_ARQUITECTURA
4.2 01_CONFIGURACION_INICIAL_Y_POM
4.3 02_CONEXION_POSTGRESQL_Y_PROPERTIES
4.4 03_ENTIDADES_MODELO_JPA
4.5 04_SERVICIO_CRUD_CON_HIBERNATE
4.6 05_INTERFAZ_GRAFICA_MAINVIEW
4.7 06_EJECUCION_Y_PRUEBAS
4.8 07_GUIA_ADAPTACION_NUEVAS_IDEAS_DE_NEGOCIO

UNIDAD TEMÁTICA • CAPÍTULO 01

Inicio

LECCIONES COMPRENDIDAS EN ESTE MÓDULO:

Bienvenidos a Programación de Aplicaciones Móviles I

Esta es la página principal del proyecto.

UNIDAD TEMÁTICA • CAPÍTULO 02

Git - Github

LECCIONES COMPRENDIDAS EN ESTE MÓDULO:

- **2.1:** Tutorial de Git y GitHub

Git - Github

Escribe aquí tu contenido...

CAPÍTULO 2 • LECCIÓN 2.1

Tutorial de Git y GitHub

Tutorial de Git y GitHub

Introducción

Git es un sistema de control de versiones distribuido que te permite rastrear cambios en tus archivos y colaborar con otros desarrolladores. GitHub es una plataforma que aloja repositorios Git en la nube.

1. Instalación y Configuración Inicial

Instalar Git

BASH

```bash
# Windows: Descargar desde https://git-scm.com/download/win
# macOS:
brew install git
# Linux (Ubuntu/Debian):
sudo apt-get install git
```

Configuración inicial

BASH

```bash
# Configurar tu identidad
git config --global user.name "Tu Nombre"
git config --global user.email "tu.email@ejemplo.com"

# Verificar configuración
git config --list
```

2. Crear un Proyecto Real: Blog Personal

Vamos a crear un blog personal paso a paso para entender Git.

Inicializar el repositorio

BASH

```bash
# Crear carpeta del proyecto
mkdir mi-blog
cd mi-blog

# Inicializar repositorio Git
git init

# Verificar estado
git status
```

3. Primer Commit: Estructura Básica

Crear archivos iniciales

```bash
# Crear archivo README
echo "# Mi Blog Personal" > README.md

# Crear estructura de carpetas
mkdir css js posts
```

Crear el primer archivo HTML

```html
<!-- index.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi Blog Personal</title>
</head>
<body>
    <header>
        <h1>Bienvenido a Mi Blog</h1>
    </header>
    <main>
        <p>Próximamente: Contenido interesante</p>
    </main>
</body>
</html>
```

Agregar y confirmar cambios

```bash
# Ver qué archivos han cambiado
git status

# Agregar archivos al staging area
git add README.md index.html

# O agregar todos los archivos
git add .

# Crear el primer commit
git commit -m "feat: estructura inicial del blog"

# Ver historial
git log
```

4. Agregar Estilos y Funcionalidad

Crear archivo CSS

```css
/* css/styles.css */
body {
```

```
    font-family: Arial, sans-serif;
    max-width: 800px;
    margin: 0 auto;
    padding: 20px;
    background-color: #f4f4f4;
}

header {
    background-color: #333;
    color: white;
    padding: 20px;
    border-radius: 5px;
    text-align: center;
}

.post {
    background-color: white;
    padding: 20px;
    margin: 20px 0;
    border-radius: 5px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}
```

Actualizar HTML con estilos

HTML

```html
<!-- index.html actualizado -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi Blog Personal</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
    <header>
        <h1>Bienvenido a Mi Blog</h1>
        <p>Un espacio para compartir ideas</p>
    </header>
    <main>
        <article class="post">
            <h2>Mi Primer Post</h2>
            <p>¡Hola! Este es el comienzo de mi aventura en el blogging.</p>
        </article>
    </main>
</body>
</html>
```

Commit de los cambios

BASH

```bash
git add .
git commit -m "feat: agregar estilos y primer post"
```

5. Entendiendo el Historial de Commits

Ver el historial completo

BASH

```
# Historial detallado
git log

# Historial resumido en una línea
git log --oneline

# Historial con gráfico
git log --graph --oneline --all

# Ver cambios específicos de un commit
git show <hash-del-commit>
```

Ejemplo de historial

JAVASCRIPT

```
* a1b2c3d (HEAD -> main) feat: agregar estilos y primer post
* 9f8e7d6 feat: estructura inicial del blog
```

6. Viajar en el Tiempo: Retroceder en Commits

Método 1: Checkout (modo exploración)

BASH

```
# Ver un commit anterior sin perder cambios
git checkout <hash-del-commit>

# Ejemplo: volver al primer commit
git checkout 9f8e7d6

# Volver al último commit (main)
git checkout main
```

Método 2: Reset (deshacer cambios)

BASH

```
# Soft reset: mantiene cambios en staging
git reset --soft HEAD~1

# Mixed reset (default): mantiene cambios en working directory
git reset HEAD~1

# Hard reset: elimina todos los cambios
git reset --hard HEAD~1

# Reset a un commit específico
git reset --hard <hash-del-commit>
```

Método 3: Revert (crear commit inverso)

BASH

```
# Revertir el último commit (crea un nuevo commit)
git revert HEAD
```

```
# Revertir un commit específico
git revert <hash-del-commit>
```

7. Crear Ramas para Nuevas Funcionalidades

Crear y cambiar a una nueva rama

BASH

```bash
# Crear nueva rama para agregar funcionalidad de comentarios
git branch agregar-comentarios

# Cambiar a la nueva rama
git checkout agregar-comentarios

# O crear y cambiar en un solo comando
git checkout -b agregar-comentarios
```

Agregar funcionalidad de comentarios

JAVASCRIPT

```javascript
// js/comments.js
function agregarComentario() {
    const comentario = prompt("Escribe tu comentario:");
    if (comentario) {
        const nuevoComentario = document.createElement('div');
        nuevoComentario.className = 'comment';
        nuevoComentario.innerHTML = `
            <p><strong>Visitante:</strong> ${comentario}</p>
            <small>${new Date().toLocaleString()}</small>
        `;
        document.getElementById('comments-section').appendChild(nuevoComentario);
    }
}
```

Commit en la nueva rama

BASH

```bash
git add js/comments.js
git commit -m "feat: agregar sistema de comentarios"
```

8. Fusionar Ramas (Merge)

Volver a la rama principal y fusionar

```bash
# Cambiar a main
git checkout main

# Fusionar la rama de comentarios
git merge agregar-comentarios

# Eliminar la rama si ya no la necesitas
git branch -d agregar-comentarios
```

9. Conectar con GitHub

Crear repositorio en GitHub

1. Ve a https://github.com
2. Crea una cuenta o inicia sesión
3. Haz clic en "+" ' "New repository"
4. Nombre: "mi-blog"
5. No inicialices con README (ya lo tenemos)
6. Haz clic en "Create repository"

Conectar repositorio local con GitHub

```bash
# Agregar repositorio remoto
git remote add origin https://github.com/tu-usuario/mi-blog.git

# Verificar remotos
git remote -v

# Subir cambios al repositorio remoto
git push -u origin main
```

10. Flujo de Trabajo Diario

Actualizar cambios desde GitHub

```bash
# Descargar y fusionar cambios remotos
git pull origin main
```

Hacer cambios y subirlos

```bash
# 1. Ver estado
git status

# 2. Agregar cambios
git add .
```

```
# 3. Commit
git commit -m "feat: descripción del cambio"

# 4. Subir a GitHub
git push origin main
```

11. Manejo de Conflictos

Ejemplo de conflicto

Cuando dos personas editan el mismo archivo:

BASH

```
# Intentar hacer pull
git pull origin main

# Si hay conflicto, Git mostrará:
# CONFLICT (content): Merge conflict in index.html

# Abrir el archivo y verás marcadores:
<<<<<<< HEAD
Tu versión del código
=======
Versión de GitHub
>>>>>>> origin/main

# Resolver manualmente y luego:
git add index.html
git commit -m "fix: resolver conflicto en index.html"
git push origin main
```

12. Comandos Útiles y Buenas Prácticas

Comandos esenciales

BASH

```
# Ver estado detallado
git status

# Ver diferencias
git diff

# Ver diferencias en staging
git diff --staged

# Crear alias útiles
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.ci commit
git config --global alias.st status

# Ver archivos ignorados
cat .gitignore

# Limpiar archivos no rastreados
git clean -n  # Ver qué se eliminaría
git clean -f  # Eliminar realmente
```

Crear .gitignore

```bash
# .gitignore
node_modules/
*.log
.env
.DS_Store
dist/
build/
```

13. Ejercicio Práctico Completo

Escenario: Agregar una página "Acerca de"

1. **Crear rama para la nueva funcionalidad**

```bash
git checkout -b agregar-acerca
```

1. **Crear la página**

```html
<!-- acerca.html -->
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Acerca de - Mi Blog</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
    <header>
        <h1>Acerca de Mí</h1>
    </header>
    <main>
        <div class="post">
            <h2>Mi Historia</h2>
            <p>Soy un apasionado desarrollador que ama compartir conocimiento...</p>
        </div>
    </main>
</body>
</html>
```

1. **Commit y push**

```bash
git add acerca.html
git commit -m "feat: agregar página acerca de"
git push origin agregar-acerca
```

1. **Crear Pull Request en GitHub**

2. Ve a tu repositorio en GitHub
3. Haz clic en "Compare & pull request"
4. Revisa los cambios
5. Haz clic en "Create pull request"
6. Luego "Merge pull request"

1. **Actualizar local**

```bash
git checkout main
git pull origin main
```

14. Solución de Problemas Comunes

Deshacer último commit pero mantener cambios

```bash
git reset --soft HEAD~1
```

Cambiar mensaje del último commit

```bash
git commit --amend -m "nuevo mensaje"
```

UNIDAD TEMÁTICA • CAPÍTULO 03

Contenido

LECCIONES COMPRENDIDAS EN ESTE MÓDULO:

- **3.1:** Clase 1: Introducción a la POO y Entorno Java
- **3.2:** Clase 2: Clases, Objetos y Constructores
- **3.3:** Clase 4: Relaciones entre Clases
- **3.4:** Clase 3: Encapsulamiento y Modificadores de Acceso
- **3.5:** Clase 5: Herencia
- **3.6:** Clase 7 - Ejercicio Práctico
- **3.7:** Clase 8 - Polimorfismo en Java
- **3.8:** Clase 9 - Clases Abstractas
- **3.9:** Clase 10 - Interfaces
- **3.10:** Clase 13 - Gestores de Dependencias

Contenido

Escribe aquí tu contenido...

CAPÍTULO 3 • LECCIÓN 3.1

Clase 1: Introducción a la POO y Entorno Java

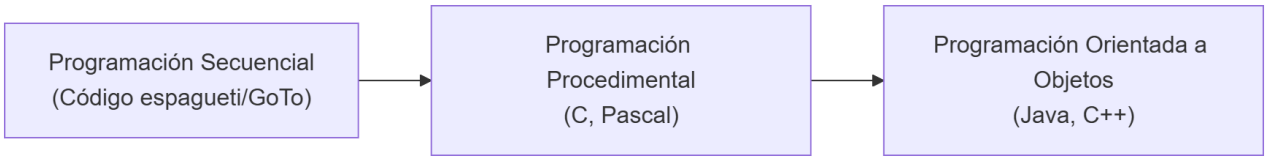
Clase 1: Introducción a la POO y Entorno Java

Clase 1: Introducción a la POO y Entorno Java

1. Evolución de los Paradigmas de Programación

Para comprender verdaderamente el valor de la **Programación Orientada a Objetos (POO)**, es indispensable repasar brevemente la historia de cómo la ingeniería de software ha solucionado el problema de la complejidad a lo largo de las décadas.

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



A. Programación Secuencial / Estructurada

En los inicios de la informática, los programas ejecutaban líneas de código una tras otra. Cuando los proyectos crecían a miles de líneas, el código se volvía inmanejable ("código espagueti") debido al uso descontrolado de saltos (`goto`) y variables globales que cualquier parte del programa podía modificar sin control.

B. Programación Procedimental (Modular)

Posteriormente surgió la división en funciones o procedimientos. Aunque mejoró la reutilización de código, continuaba existiendo una separación conceptual estricta:

- **Los Datos** estaban por un lado (estructuras de datos, variables).
- **Las Funciones** estaban por otro lado operando sobre esos datos.

Problema: Si la estructura de los datos cambiaba, todas las funciones del programa que tocaban esos datos debían ser reescritas manualmente.

C. Programación Orientada a Objetos (POO)

La POO soluciona este problema unificando los datos y las funciones que los manipulan dentro de una sola unidad conceptual llamada **OBJETO**.

Beneficios Clave de la POO:

1. **Modularity (Modularidad):** El código se divide en partes autónomas e independientes.
2. **Reusability (Reutilización):** Puedes usar clases creadas previamente en múltiples proyectos sin copiar y pegar código.

3. **Maintainability (Mantenibilidad):** Si hay un error en la lógica de un auto, solo editas la clase `Automovil`, sin afectar el resto del sistema.
4. **Modelado Intuitivo:** Diseñas software utilizando conceptos del mundo real (Usuarios, Facturas, Pedidos, Productos).

---

2. Profundización Teórica: Clase vs. Objeto y Gestión de Memoria

La Clase (El Plano / Molde)

Una **Clase** es una especificación de diseño. Define qué variables (atributos) y qué funciones (métodos) tendrán todos los objetos creados bajo su nombre.

JAVA

```java
// Ejemplo conceptual de una Clase
public class Celular {
    // Atributos (Definen la estructura)
    String marca;
    String modelo;
    int porcentajeBateria;

    // Métodos (Definen las capacidades)
    void cargarBateria() { ... }
    void hacerLlamada() { ... }
}
```

El Objeto (Instancia en Memoria)

Un **Objeto** es un bloque de memoria RAM reservado dinámicamente que contiene valores reales para las propiedades definidas en la clase.

¿Cómo gestiona Java la Memoria? Stack vs. Heap

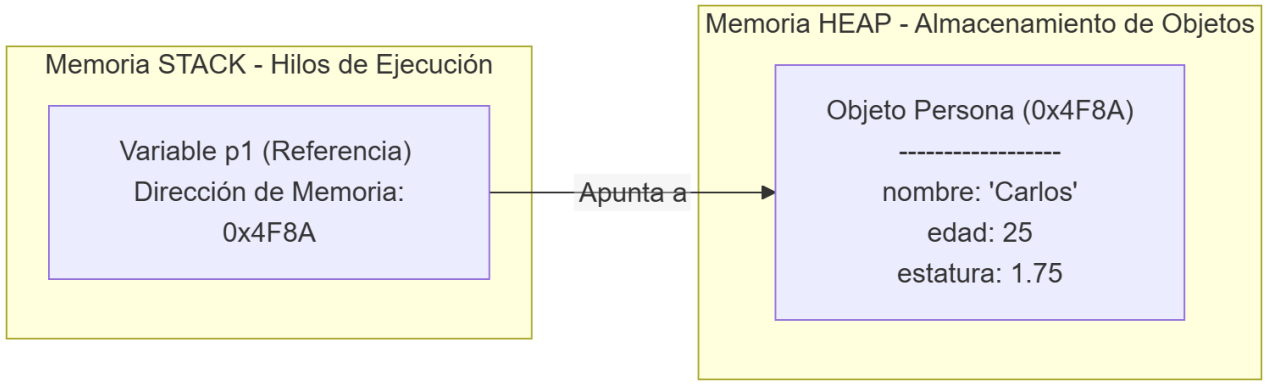
Cuando ejecutas la siguiente instrucción en Java:

JAVA

```java
Persona p1 = new Persona();
```

Ocurren dos cosas en la memoria de la JVM (Java Virtual Machine):

DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



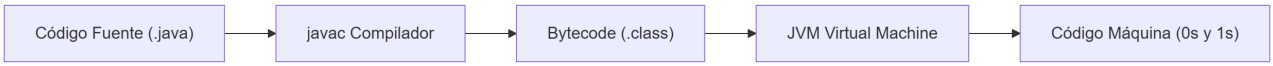
1. **Memoria STACK (Pila):** Es ultrarrápida y almacena la **variable de referencia** (`p1`). `p1` no contiene la persona en sí, sino una *dirección de memoria* (puntero seguro administrado).
2. **Memoria HEAP (Montículo):** Es el espacio donde se crea el **objeto real** usando la palabra reservada `new`. Ahí se guardan sus atributos `nombre`, `edad`, etc.

---

3. El Proceso de Compilación y Ejecución en Java (JDK vs JRE vs JVM)

Para entender cómo corre un programa Java:

DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



- **JDK (Java Development Kit):** Es el kit completo para desarrolladores. Contiene el compilador `javac`, documentación y herramientas de depuración.
- **JRE (Java Runtime Environment):** Es el entorno necesario solo para *ejecutar* programas Java ya compilados.
- **JVM (Java Virtual Machine):** Es la máquina virtual que interpreta el **Bytecode (`.class`)** y lo traduce a instrucciones de CPU nativas según el sistema operativo (Windows, Linux, macOS). ¡Esto permite que Java sea *"Escribe una vez, ejecuta en cualquier lugar"*!

---

4. Anatomía Detallada del Método `main`

El método ejecutable en Java tiene la siguiente firma exacta:

JAVA

```java
public static void main(String[] args)
```

- **`public` (Modificador de Acceso):** Permite que el motor de tiempo de ejecución de Java (JVM) pueda llamar al método desde fuera de la clase.

- **`static` (Estático):** Indica que la JVM no necesita crear un objeto de la clase contenedora para llamar a este método.
- **`void` (Tipo de Retorno):** Significa que el método realiza sus operaciones y no devuelve ningún valor al finalizar.
- **`main` (Nombre del Método):** Es el nombre reservado que busca la JVM para iniciar la ejecución del programa.
- **`String[] args` (Parámetros):** Arreglo de cadenas de texto que recibe los argumentos pasados por la consola al lanzar el programa.

---

5. Ejemplos de Código Completos y Profundizados

A continuación se presentan **4 ejemplos completos y ejecutables** que abordan distintos dominios del mundo real.

Ejemplo 1: Sistema de Calificaciones de un Estudiante

Este ejemplo demuestra cómo una clase maneja arreglos simples, cálculos de promedio y cambios de estado.

JAVA

```java
// Archivo: EjemploEstudiante.java

class Estudiante {
    // Atributos de Instancia
    String matricula;
    String nombreCompleto;
    String carrera;
    double nota1;
    double nota2;
    double nota3;

    // Método para calcular el promedio ponderado
    double calcularPromedio() {
        return (nota1 * 0.30) + (nota2 * 0.30) + (nota3 * 0.40);
    }

    // Método para evaluar la condición académica
    boolean haAprobado() {
        return calcularPromedio() >= 3.0; // Puntuación mínima de aprobación: 3.0 / 5.0
    }

    // Método de presentación
    void imprimirBoletin() {
        double promedio = calcularPromedio();
        System.out.println("================================================");
        System.out.println("BOLETÍN ACADÉMICO - " + carrera.toUpperCase());
        System.out.println("Estudiante : " + nombreCompleto + " (Matrícula: " + matricula + ")");
        System.out.println("Nota 1 (30%): " + nota1);
        System.out.println("Nota 2 (30%): " + nota2);
        System.out.println("Nota 3 (40%): " + nota3);
        System.out.println("------------------------------------------------");
        System.out.println("PROMEDIO FINAL: " + String.format("%.2f", promedio));
        System.out.println("ESTADO        : " + (haAprobado() ? " APROBADO" : "LREPROBADO"));
        System.out.println("================================================\n");
    }
}

public class EjemploEstudiante {
    public static void main(String[] args) {
        // Estudiante 1
        Estudiante est1 = new Estudiante();
```

```
        est1.matricula = "2026-EST-01";
        est1.nombreCompleto = "Mariana Ríos";
        est1.carrera = "Ingeniería de Software";
        est1.nota1 = 4.5;
        est1.nota2 = 3.8;
        est1.nota3 = 4.2;

        // Estudiante 2
        Estudiante est2 = new Estudiante();
        est2.matricula = "2026-EST-02";
        est2.nombreCompleto = "Pedro Gutiérrez";
        est2.carrera = "Ingeniería de Software";
        est2.nota1 = 2.5;
        est2.nota2 = 2.8;
        est2.nota3 = 3.0;

        // Ejecución de métodos
        est1.imprimirBoletin();
        est2.imprimirBoletin();
    }
}
```

Ejemplo 2: Control de Vehículo y Consumo de Combustible

Este ejemplo ilustra cómo los métodos modifican activamente los atributos internos de un objeto imitando la física del mundo real.

JAVA

```
// Archivo: EjemploAutomovil.java

class Automovil {
    String marca;
    String modelo;
    double capacidadTanqueLitros; // Capacidad máxima del tanque
    double nivelGasolinaActual;   // Litros en el tanque actualmente
    double kilometrajeActual;     // Kilómetros recorridos
    double consumoKmPorLitro;     // Rendimiento (km que recorre por cada litro)

    // Método para encender y conducir el automóvil
    void conducir(double kilometrosADistancia) {
        System.out.println("\n=Intentando conducir " + kilometrosADistancia + " km con el " + marca + " " + modelo + "...");

        // Calcular cuántos litros de gasolina se necesitan para ese viaje
        double gasolinaNecesaria = kilometrosADistancia / consumoKmPorLitro;

        if (gasolinaNecesaria <= nivelGasolinaActual) {
            // Se puede realizar el viaje completo
            nivelGasolinaActual -= gasolinaNecesaria;
            kilometrajeActual += kilometrosADistancia;
            System.out.println(" Viaje completado. Recorriste " + kilometrosADistancia + " km.");
            System.out.println("  Gasolina consumida: " + String.format("%.2f", gasolinaNecesaria) + " L.");
        } else {
            // No alcanza la gasolina para todo el trayecto
            double kmPosibles = nivelGasolinaActual * consumoKmPorLitro;
            kilometrajeActual += kmPosibles;
            nivelGasolinaActual = 0; // Se agota la gasolina
            System.out.println("LEl combustible se agotó a los " + String.format("%.2f", kmPosibles) + " km!");
            System.out.println("  Te quedaste varado antes de llegar a tu destino de " + kilometrosADistancia + " km.");
```

```
        }
    }

    // Método para repostar gasolina en la estación de servicio
    void reabastecerGasolina(double litros) {
        if (litros <= 0) {
            System.out.println("Monto de litros no válido.");
            return;
        }

        if (nivelGasolinaActual + litros > capacidadTanqueLitros) {
            double desborde = (nivelGasolinaActual + litros) - capacidadTanqueLitros;
            nivelGasolinaActual = capacidadTanqueLitros;
            System.out.println("ýTanque Lleno al 100%. Se rebosaron " + String.format("%.2f", desborde-
) + " litros.");
        } else {
            nivelGasolinaActual += litros;
            System.out.println("ýReabastecimiento exitoso. Litros actuales en tanque: " + String.format-
("%.2f", nivelGasolinaActual) + " L.");
        }
    }

    void mostrarTableroInfo() {
        System.out.println("-------------------------------------------------");
        System.out.println(" TABLERO DE CONTROL - " + marca + " " + modelo);
        System.out.println(" Kilometraje total: " + kilometrajeActual + " km");
        System.out.println(" Tanque de gasolina: " + String.format("%.2f", nivelGasolinaActual) + " / "-
 + capacidadTanqueLitros + " L");
        System.out.println("-------------------------------------------------");
    }
}

public class EjemploAutomovil {
    public static void main(String[] args) {
        Automovil miAuto = new Automovil();
        miAuto.marca = "Toyota";
        miAuto.modelo = "Corolla";
        miAuto.capacidadTanqueLitros = 50.0;
        miAuto.nivelGasolinaActual = 15.0; // Inicia con 15 Litros
        miAuto.kilometrajeActual = 12000.0;
        miAuto.consumoKmPorLitro = 12.0;    // Hace 12 km por litro

        miAuto.mostrarTableroInfo();

        // 1. Intentamos conducir 200 km (requiere 200/12 = 16.66L -> No alcanzará con 15L)
        miAuto.conducir(200.0);
        miAuto.mostrarTableroInfo();

        // 2. Reabastecemos gasolina
        miAuto.reabastecerGasolina(40.0);
        miAuto.mostrarTableroInfo();

        // 3. Volvemos a intentar el viaje de 200 km
        miAuto.conducir(200.0);
        miAuto.mostrarTableroInfo();
    }
}
```

Ejemplo 3: Personaje de Videojuego RPG (Combate y Puntos de Vida)

Demuestra cómo interactúan dos objetos del mismo tipo en una acción del mundo real (el ataque de un personaje a otro).

JAVA

```java
// Archivo: EjemploRPG.java

class PersonajeRPG {
    String nombre;
    String claseHeroe; // Guerrero, Mago, Arquero
    int puntosVida;
    int puntosVidaMaximos;
    int poderAtaque;
    int defensa;

    // Método para atacar a OTRO personaje pasándolo como parámetro
    void atacar(PersonajeRPG objetivo) {
        System.out.println("\n" + nombre + " (" + claseHeroe + ") ataca a " + objetivo.nombre + "...");

        // Cálculo del daño real considerando la defensa del objetivo
        int danoNeto = poderAtaque - objetivo.defensa;
        if (danoNeto < 1) {
            danoNeto = 1; // Al menos causa 1 punto de daño mínimo
        }

        // Aplicamos el daño al objetivo
        objetivo.recibirDano(danoNeto);
    }

    void recibirDano(int cantidadDano) {
        puntosVida -= cantidadDano;
        if (puntosVida < 0) {
            puntosVida = 0;
        }
        System.out.println(" =¥ " + nombre + " recibió " + cantidadDano + " puntos de daño. Vida restante: "-
 + puntosVida + "/" + puntosVidaMaximos);

        if (puntosVida == 0) {
            System.out.println(" =€" + nombre + " ha sido derrotado en combate!");
        }
    }

    void curar(int cantidadCuracion) {
        if (puntosVida == 0) {
            System.out.println("LNo se puede curar a " + nombre + " porque ya ha caído en batalla.");
            return;
        }
        puntosVida += cantidadCuracion;
        if (puntosVida > puntosVidaMaximos) {
            puntosVida = puntosVidaMaximos;
        }
        System.out.println("(" + nombre + " se ha curado. Vida actual: " + puntosVida + "/" + puntosVi-
daMaximos);
    }

    void mostrarFicha() {
        System.out.println("HÉROE: " + nombre + " [" + claseHeroe + "] | HP: " + puntosVida + "/" + pun-
tosVidaMaximos
                           + " | ATK: " + poderAtaque + " | DEF: " + defensa);
    }
}

public class EjemploRPG {
    public static void main(String[] args) {
        // Crear Heroe 1
        PersonajeRPG guerrero = new PersonajeRPG();
        guerrero.nombre = "Thorin";
        guerrero.claseHeroe = "Guerrero";
        guerrero.puntosVidaMaximos = 100;
```

```
        guerrero.puntosVida = 100;
        guerrero.poderAtaque = 25;
        guerrero.defensa = 10;

        // Crear Heroe 2
        PersonajeRPG orco = new PersonajeRPG();
        orco.nombre = "Gorgoth";
        orco.claseHeroe = "Orco Berserker";
        orco.puntosVidaMaximos = 120;
        orco.puntosVida = 120;
        orco.poderAtaque = 20;
        orco.defensa = 5;

        System.out.println("=== ESTADO INICIAL DE LOS COMBATIENTES ===");
        guerrero.mostrarFicha();
        orco.mostrarFicha();

        System.out.println("\n=== INICIO DEL COMBATE ===");
        // RUTA DE COMBATE: Thorin ataca a Gorgoth
        guerrero.atacar(orco);

        // Gorgoth contraataca
        orco.atacar(guerrero);

        // Thorin usa poción de curación
        guerrero.curar(15);

        // Thorin asesta ataque crítico a Gorgoth
        guerrero.poderAtaque = 120; // Super ataque
        guerrero.atacar(orco);
    }
}
```

Ejemplo 4: Smartphone y Gestión de Aplicaciones y Batería

Un ejemplo enfocado en simular dispositivos tecnológicos del día a día.

JAVA

```
// Archivo: EjemploCelular.java

class Celular {
    String marca;
    String modelo;
    int almacenamientoGB;
    int almacenamientoUsadoGB;
    int porcentajeBateria;
    boolean pantallaEncendida;

    void presionarBotonEncendido() {
        pantallaEncendida = !pantallaEncendida; // Alternar estado (On/Off)
        System.out.println("=Pantalla del " + marca + " " + modelo + " está ahora: " + (pantallaEncendida ? "ENCENDIDA" : "APAGADA"));
    }

    void instalarAplicacion(String nombreApp, int pesoGB) {
        System.out.println("\nIntentando instalar " + nombreApp + " (" + pesoGB + " GB)...");

        if (almacenamientoUsadoGB + pesoGB <= almacenamientoGB) {
            almacenamientoUsadoGB += pesoGB;
            System.out.println(" App " + nombreApp + " instalada con éxito.");
```

```
            System.out.println("  Espacio libre restante: " + (almacenamientoGB - almacenamientoUsadoGB-
) + " GB.");
        } else {
            System.out.println("LError: Espacio de almacenamiento insuficiente. Libera espacio para in-
stalar " + nombreApp);
        }
    }

    void usarAplicacion(String nombreApp, int minutosUso) {
        if (!pantallaEncendida) {
            System.out.println("LNo puedes usar " + nombreApp + " porque la pantalla está apagada.");
            return;
        }

        // Cada 10 minutos de uso consumen 2% de batería
        int gastoBateria = (minutosUso / 10) * 2;
        if (gastoBateria == 0) gastoBateria = 1; // Al menos 1%

        if (porcentajeBateria - gastoBateria <= 0) {
            porcentajeBateria = 0;
            pantallaEncendida = false;
            System.out.println("><La batería se ha agotado por completo! El celular se apagó mientras us-
abas " + nombreApp);
        } else {
            porcentajeBateria -= gastoBateria;
            System.out.println("=Usaste " + nombreApp + " por " + minutosUso + " minu-
tos. Batería restante: " + porcentajeBateria + "%");
        }
    }

    void cargarBateria(int porcentaje) {
        porcentajeBateria += porcentaje;
        if (porcentajeBateria > 100) porcentajeBateria = 100;
        System.out.println("=Celular cargado. Batería actual: " + porcentajeBateria + "%");
    }
}

public class EjemploCelular {
    public static void main(String[] args) {
        Celular miPhone = new Celular();
        miPhone.marca = "Samsung";
        miPhone.modelo = "Galaxy S24";
        miPhone.almacenamientoGB = 128;
        miPhone.almacenamientoUsadoGB = 40;
        miPhone.porcentajeBateria = 15; // Batería baja
        miPhone.pantallaEncendida = false;

        // Intentar usar app con pantalla apagada
        miPhone.usarAplicacion("Instagram", 20);

        // Encender teléfono
        miPhone.presionarBotonEncendido();

        // Instalar un juego pesado
        miPhone.instalarAplicacion("Genshin Impact", 30);
        miPhone.instalarAplicacion("Call of Duty Mobile", 70); // Superará los 128GB totales

        // Usar la app hasta que se agote la batería
        miPhone.usarAplicacion("YouTube", 90);

        // Poner a cargar el dispositivo
        miPhone.cargarBateria(50);
    }
}
```

6. Ejercicios Prácticos Adicionales (Con Solución)

Ejercicio 1: El Reloj Digital

Consigna: Crea una clase `RelojDigital` que contenga:

- Atributos: `horas` (0 a 23), `minutos` (0 a 59), `segundos` (0 a 59).
- Método `avanzarSegundo()`: Incrementa en 1 segundo la hora actual. Toma en cuenta el paso de minutos cuando los segundos lleguen a 60, y el paso de horas cuando los minutos lleguen a 60, reseteando a 00:00:00 al pasar las 23:59:59.
- Método `mostrarHora()`: Imprime la hora en formato `HH:MM:SS`.

Solución Explicada:

JAVA

```java
class RelojDigital {
    int horas;
    int minutos;
    int segundos;

    void avanzarSegundo() {
        segundos++;
        if (segundos == 60) {
            segundos = 0;
            minutos++;
            if (minutos == 60) {
                minutos = 0;
                horas++;
                if (horas == 24) {
                    horas = 0;
                }
            }
        }
    }

    void mostrarHora() {
        // String.format("%02d") asegura que los números de un solo dígito se impri-
    man con cero a la izquierda (ej. 05 en vez de 5)
        System.out.println("ðHora Actual: "
                           + String.format("%02d", horas) + ":"
                           + String.format("%02d", minutos) + ":"
                           + String.format("%02d", segundos));
    }
}

public class PruebaReloj {
    public static void main(String[] args) {
        RelojDigital reloj = new RelojDigital();
        reloj.horas = 23;
        reloj.minutos = 59;
        reloj.segundos = 58;

        System.out.println("--- Probando cambio de medianoche ---");
        reloj.mostrarHora();

        reloj.avanzarSegundo(); // 23:59:59
```

```
        reloj.mostrarHora();

        reloj.avanzarSegundo(); // 00:00:00
        reloj.mostrarHora();
    }
}
```

Ejercicio 2: La Cafetera Inteligente

Consigna: Crea una clase `Cafetera` con los siguientes datos:

- Atributos: `capacidadMaxima` (en c.c.), `cantidadActual` (en c.c.).
- Métodos:
- `llenarCafetera()`: Hace que la cantidad actual sea igual a la capacidad máxima.
- `servirTaza(int capacidadTaza)`: Simula el acto de servir una taza de café. Si la cantidad actual no alcanza para llenar la taza, se sirve lo que quede en la cafetera y la cantidad actual queda en 0.
- `vaciarCafetera()`: Pone la cantidad actual de café en 0.
- `agregarCafe(int cantidad)`: Añade la cantidad de café indicada a la cafetera sin sobrepasar la capacidad máxima.

Solución Explicada:

JAVA

```java
class Cafetera {
    int capacidadMaxima;
    int cantidadActual;

    void llenarCafetera() {
        cantidadActual = capacidadMaxima;
        System.out.println(" Cafetera llena al 100% (" + capacidadMaxima + " c.c.).");
    }

    void servirTaza(int capacidadTaza) {
        System.out.println("\nServir taza de " + capacidadTaza + " c.c. ...");
        if (cantidadActual == 0) {
            System.out.println("LNo hay café disponible en la cafetera.");
        } else if (cantidadActual >= capacidadTaza) {
            cantidadActual -= capacidadTaza;
            System.out.println(" Taza servida completamente. Quedan " + cantidadActual + " c.c. de café.");
        } else {
            System.out.println(" No alcanzó para llenar la taza. Se sirvieron únicamente " + cantidadAc-
tual + " c.c.");
            cantidadActual = 0;
        }
    }

    void vaciarCafetera() {
        cantidadActual = 0;
        System.out.println(">Cafetera vaciada.");
    }

    void agregarCafe(int cantidad) {
        if (cantidadActual + cantidad > capacidadMaxima) {
            cantidadActual = capacidadMaxima;
            System.out.println(" Se agregó café hasta el límite máximo de " + capacidadMaxima + " c.c.");
        } else {
            cantidadActual += cantidad;
```

```
            System.out.println(" Se agregaron " + cantidad + " c.c. Cantidad actual: " + cantidadActu-
al + " c.c.");
        }
    }
}

public class PruebaCafetera {
    public static void main(String[] args) {
        Cafetera miCafetera = new Cafetera();
        miCafetera.capacidadMaxima = 1000; // 1 Litro (1000 c.c.)
        miCafetera.cantidadActual = 0;      // Vacía inicialmente

        miCafetera.llenarCafetera();
        miCafetera.servirTaza(250); // Sirve taza grande
        miCafetera.servirTaza(250);
        miCafetera.servirTaza(600); // Intenta servir más de lo que queda (quedan 500 c.c.)

        miCafetera.agregarCafe(300);
        miCafetera.servirTaza(200);
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.2

Clase 2: Clases, Objetos y Constructores

Clase 2: Clases, Objetos y Constructores

1. Métodos y la Firma de un Método

En Java, un **Método** es un bloque de código reutilizable que realiza una tarea específica cuando es invocado.

¿Qué compone la Firma de un Método?

La **firma de un método** es la combinación única de:

1. **Nombre del Método.**
2. **Número, tipo y orden de sus parámetros.**

JAVA

```java
// Firma: calcularPrecioFinal(double, double, int)
public double calcularPrecioFinal(double precioBase, double descuento, int impuesto) {
    return (precioBase - descuento) * (1 + impuesto / 100.0);
}
```

> **NOTA CLAVE**
> **IMPORTANTE:** El tipo de retorno (ej. `double`, `void`) y los modificadores (ej. `public`, `static`) **NO** forman parte de la firma del método. Por lo tanto, no puedes tener dos métodos con la misma firma en la misma clase cambiando solo el tipo de retorno.

2. Paso de Parámetros en Java: ¿Por Valor o por Referencia?

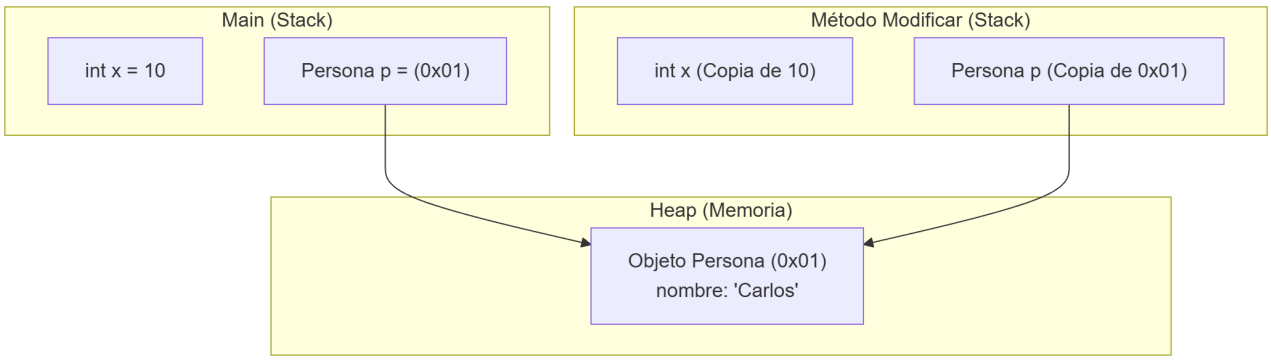
Existe una confusión muy común en el aprendizaje de Java respecto a cómo se pasan los parámetros a los métodos.

> **NOTA CLAVE**
> **REGLA DE ORO DE JAVA:** En Java, **TODO se pasa estrictamente POR VALOR** (*Pass-by-Value*).

¿Qué significa esto?

- **Tipos Primitivos (`int`, `double`, `boolean`, etc.):** Se pasa una **copia exacta del valor**. Cualquier cambio realizado dentro del método no afecta a la variable original fuera del método.
- **Objetos (Tipos de Referencia):** Se pasa una **copia de la dirección de memoria (referencia)**. Por lo tanto:
- Si modificas el *estado interno* del objeto a través de la referencia copiada (ej. `objeto.setNombre("Nuevo")`), **SÍ se reflejará fuera del método**.
- Si intentas reasignar la referencia dentro del método (`objeto = new OtroObjeto()`), **NO afectará a la variable original fuera del método**.

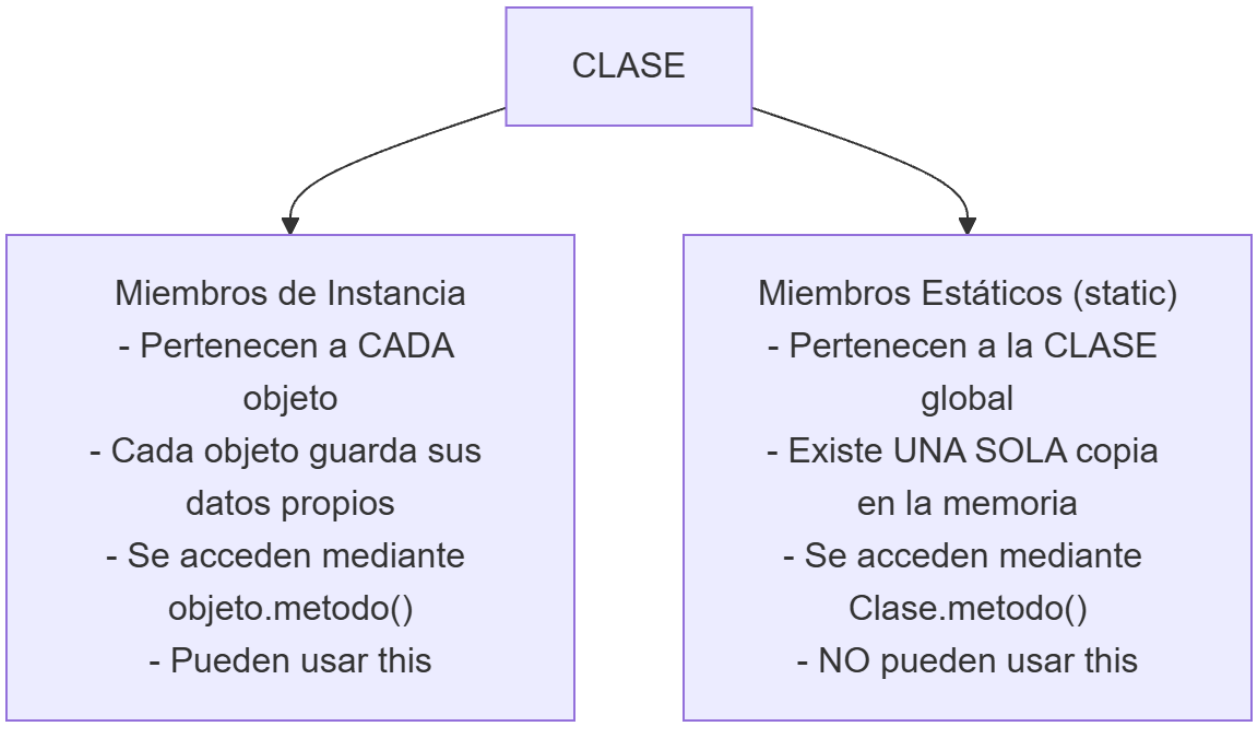
DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



3. Miembros de Instancia vs. Estáticos (`static`)

Los miembros (atributos y métodos) de una clase se clasifican en dos categorías:

DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



Bloques de Inicialización Estáticos (`static { ... }`)

Se ejecutan **una sola vez** cuando la clase es cargada en memoria por la JVM, antes de crear cualquier objeto.

JAVA

```java
class ConfiguracionSistema {
    static String urlServidor;
```

```
    static int puerto;

    // Bloque de inicialización estático
    static {
        urlServidor = "https://api.empresa.com";
        puerto = 8080;
        System.out.println("™Configuración del sistema cargada estáticamente.");
    }
}
```

4. Constructores a Fondo, Tipos y Sobrecarga (*Constructor Overloading*)

Un **Constructor** es una subrutina especial dentro de una clase cuyo único propósito es **instanciar el objeto** en la memoria Heap e **inicializar el estado de sus campos**.

4.1. Fundamentos y Ciclo de Vida del Constructor

Cuando ejecutas la sentencia `Persona p = new Persona("Carlos", 25);`, la Máquina Virtual de Java (JVM) realiza internamente los siguientes pasos:

1. **Reserva de Memoria:** Se asigna un bloque de memoria en el **Heap** con el tamaño suficiente para almacenar todos los atributos de la clase.
2. **Inicialización por Defecto:** Java asigna automáticamente valores por defecto a todos los campos:
3. Números (`byte`, `short`, `int`, `long`): `0`
4. Flotantes (`float`, `double`): `0.0`
5. Booleans (`boolean`): `false`
6. Caracteres (`char`): `'\u0000'` (carácter nulo)
7. Referencias a Objetos (`String`, listas, etc.): `null`
8. **Ejecución del Constructor:** Se invoca el constructor correspondiente pasando los argumentos necesarios para inicializar los campos con valores reales.
9. **Retorno de Referencia:** El operador `new` devuelve la dirección de memoria de la instancia creada y se asigna a la variable de referencia (`p`).

> **NOTA CLAVE**
> **IMPORTANT Reglas Estrictas de un Constructor en Java:** 1. debe llamarse **exactamente igual** que la clase (respetando mayúsculas y minúsculas). 2. **NO tiene tipo de retorno** (ni siquiera `void`). Si le agregas un tipo de retorno (ej. `public void Persona()`), Java lo interpreta como un método normal y NO como un constructor. 3. Se invoca únicamente durante la creación del objeto con la palabra clave `new` (o mediante encadenamiento con `this()` / `super()`).

4.2. Tipos de Constructores en Java (Con Ejemplos Explicados)

Java ofrece diversas formas de definir y utilizar constructores según el diseño de la aplicación. A continuación se detalla cada tipo con su respectivo ejemplo completo en Java:

1ãConstructor por Defecto (*Default Constructor / Implícito*)

Si **NO escribes ningún constructor** en tu clase, el compilador de Java (`javac`) genera automáticamente un constructor público sin parámetros en tiempo de compilación.

- **Propósito:** Permitir crear instancias del objeto asignando únicamente los valores por defecto del lenguaje.
- **Nota:** En el momento en que defines **al menos un constructor** (parametrizado o no), Java **elimina automáticamente** el constructor por defecto implícito.

JAVA

```java
// Archivo: EjemploConstructorDefecto.java

class ProductoSinConstructor {
    String codigo;  // Inicializado por defecto a null
    double precio;  // Inicializado por defecto a 0.0
    boolean activo; // Inicializado por defecto a false
}

public class EjemploConstructorDefecto {
    public static void main(String[] args) {
        // Al no haber declarado ningún constructor, Java inserta automáticamente:
        // public ProductoSinConstructor() {}
        ProductoSinConstructor prod = new ProductoSinConstructor();

        System.out.println("=== CONSTRUCTOR POR DEFECTO ===");
        System.out.println("Código: " + prod.codigo);   // null
        System.out.println("Precio: " + prod.precio);   // 0.0
        System.out.println("Activo: " + prod.activo);   // false
    }
}
```

2ãConstructor Sin Parámetros Explícito (*Explicit No-Arg Constructor*)

Es un constructor declarado explícitamente sin argumentos que permite establecer **valores predeterminados personalizados** en lugar de los valores por defecto nulos o ceros de la JVM.

JAVA

```java
// Archivo: EjemploConstructorSinParametros.java

class ConfiguracionApp {
    String idioma;
    int tiempoEsperaSegundos;
    boolean modoOscuro;

    // Constructor explícito sin parámetros
    public ConfiguracionApp() {
        this.idioma = "Español";
        this.tiempoEsperaSegundos = 30;
        this.modoOscuro = true;
    }
}

public class EjemploConstructorSinParametros {
    public static void main(String[] args) {
        ConfiguracionApp config = new ConfiguracionApp();

        System.out.println("=== CONSTRUCTOR SIN PARÁMETROS EXPLÍCITO ===");
```

```
        System.out.println("Idioma por defecto     : " + config.idioma);
        System.out.println("Timeout por defecto    : " + config.tiempoEsperaSegundos + " segundos");
        System.out.println("Modo Oscuro activado   : " + config.modoOscuro);
    }
}
```

3ã Constructor Parametrizado (*Parameterized Constructor*)

Es el constructor más común. Recibe uno o más parámetros de entrada para inicializar el estado del objeto con valores específicos pasados al momento de su instanciación.

JAVA

```java
// Archivo: EjemploConstructorParametrizado.java

class Estudiante {
    String matricula;
    String nombre;
    double promedio;

    // Constructor Parametrizado
    public Estudiante(String matricula, String nombre, double promedio) {
        this.matricula = matricula;
        this.nombre = nombre;
        this.promedio = promedio;
    }

    public void mostrarFicha() {
        System.out.println("<Estudiante [" + matricula + "] " + nombre + " | Promedio: " + promedio);
    }
}

public class EjemploConstructorParametrizado {
    public static void main(String[] args) {
        // Se pasan datos específicos en el orden exacto definido por los parámetros
        Estudiante e1 = new Estudiante("EST-2026-01", "María López", 9.5);
        Estudiante e2 = new Estudiante("EST-2026-02", "Juan Pérez", 8.8);

        System.out.println("=== CONSTRUCTOR PARAMETRIZADO ===");
        e1.mostrarFicha();
        e2.mostrarFicha();
    }
}
```

4ã Constructor Encadenado (*Chained Constructor con `this(...)`*)

Permite que un constructor invoque a otro constructor dentro de la **misma clase**. Sirve para centralizar la lógica de inicialización en un único constructor principal y evitar la duplicación de código (*DRY - Don't Repeat Yourself*).

> **PRECAUCIÓN**
> **CAUTION** La llamada `this(...)` debe ser estrictamente la **primera línea de código** dentro del cuerpo del constructor.

JAVA

```java
// Archivo: EjemploConstructorEncadenado.java
```

```
2 |
3 | class Empleado {
4 |     String id;
5 |     String nombre;
6 |     String departamento;
7 |     double salario;
8 |
9 |     // Constructor Principal (4 parámetros): Contiene la lógica real de inicialización
10|     public Empleado(String id, String nombre, String departamento, double salario) {
11|         this.id = id;
12|         this.nombre = nombre;
13|         this.departamento = departamento;
14|         this.salario = salario;
15|     }
16|
17|     // Constructor Secundario 1: Asigna departamento por defecto ("General")
18|     public Empleado(String id, String nombre, double salario) {
19|         this(id, nombre, "General", salario); // Llama al constructor de 4 parámetros
20|     }
21|
22|     // Constructor Secundario 2: Asigna departamento "General" y salario base (1200.0)
23|     public Empleado(String id, String nombre) {
24|         this(id, nombre, 1200.0); // Llama al Constructor Secundario 1
25|     }
26|
27|     public void mostrarDetalles() {
28|         System.out.println("ID: " + id + " | Nombre: " + nombre + " | Depto: " + departamento +-
    " | Salario: $" + salario);
29|     }
30| }
31|
32| public class EjemploConstructorEncadenado {
33|     public static void main(String[] args) {
34|         Empleado emp1 = new Empleado("EMP-01", "Ana Torres", "Sistemas", 3500.0);
35|         Empleado emp2 = new Empleado("EMP-02", "Luis Ramírez", 2000.0);
36|         Empleado emp3 = new Empleado("EMP-03", "Sofía Castro");
37|
38|         System.out.println("=== CONSTRUCTORES ENCADENADOS (this) ===");
39|         emp1.mostrarDetalles();
40|         emp2.mostrarDetalles();
41|         emp3.mostrarDetalles();
42|     }
43| }
```

5ãConstructor en Herencia (*Super Constructor con `super(...)`*)

Cuando existe una relación de herencia entre clases (`extends`), la subclase debe invocar al constructor de la superclase mediante `super(...)` para asegurar la correcta inicialización de los atributos heredados.

JAVA

```
1 | // Archivo: EjemploConstructorSuper.java
2 |
3 | class Vehiculo {
4 |     String marca;
5 |     String modelo;
6 |
7 |     // Constructor de la Superclase
8 |     public Vehiculo(String marca, String modelo) {
9 |         this.marca = marca;
10|         this.modelo = modelo;
```

```
        System.out.println("™Constructor de Vehiculo (Padre) ejecutado.");
    }
}

class AutoElectrico extends Vehiculo {
    int capacidadBateriakWh;

    // Constructor de la Subclase
    public AutoElectrico(String marca, String modelo, int capacidadBateriakWh) {
        super(marca, modelo); // Invoca al constructor de Vehiculo (DEBE ser la primera línea)
        this.capacidadBateriakWh = capacidadBateriakWh;
        System.out.println("=Constructor de AutoElectrico (Hijo) ejecutado.");
    }

    public void mostrarInfo() {
        System.out.println("Auto: " + marca + " " + modelo + " | Batería: " + capacidadBateriakWh +-
 " kWh");
    }
}

public class EjemploConstructorSuper {
    public static void main(String[] args) {
        System.out.println("=== CONSTRUCTOR EN HERENCIA (super) ===");
        AutoElectrico tesla = new AutoElectrico("Tesla", "Model 3", 75);
        tesla.mostrarInfo();
    }
}
```

4.3. Sobrecarga de Constructores (*Constructor Overloading*)

La **Sobrecarga de Constructores** consiste en definir **múltiples constructores dentro de la misma clase**, todos compartiendo el mismo nombre (el nombre de la clase), pero diferidos en la **firma de parámetros** (número, tipo o secuencia de los argumentos).

¿Por qué es fundamental la Sobrecarga?

Ofrece polimorfismo en la creación de objetos, permitiendo instanciar una clase de múltiples maneras flexibles según la cantidad de información disponible en ese momento.

Reglas de la Firma en la Sobrecarga:

Para que la sobrecarga sea válida, el compilador debe poder diferenciar un constructor de otro analizando su **lista de parámetros**:

- **Diferente número de parámetros:** `(String)`, `(String, int)`, `(String, int, double)`.
- **Diferentes tipos de datos:** `(int, String)` vs `(double, String)`.
- **Diferente orden de tipos:** `(String, int)` vs `(int, String)`.

> **ADVERTENCIA**
> **WARNING** Cambiar únicamente los nombres de las variables en los parámetros **NO** constituye sobrecarga válida si los tipos de datos y la cantidad son los mismos. El compilador generará un error de "duplicación de método/constructor".

Cuadro Comparativo de Firmas Sobrecargadas:

CONSTRUCTOR	FIRMA DE PARÁMETROS	VÁLIDO	RAZÓN

`Producto()`	Sin parámetros `()`	Sí	Constructor básico / por defecto explícito
`Producto(String id)`	`(String)`	Sí	1 parámetro tipo String
`Producto(int id)`	`(int)`	Sí	1 parámetro tipo int (difiere en tipo)
`Producto(String id, double p)`	`(String, double)`	Sí	2 parámetros (String, double)
`Producto(String nombre, double precio)`	`(String, double)`	**ERROR**	Misma firma que el anterior (-`String, double`)

Ejemplo Completo de Sobrecarga en Java

El siguiente programa demuestra cómo conviven múltiples constructores sobrecargados en una misma clase (`Tarjeta-Credito`) y cómo el compilador selecciona dinámicamente el adecuado según los argumentos suministrados:

JAVA

```java
// Archivo: EjemploSobrecargaConstructores.java

class TarjetaCredito {
    private String numeroTarjeta;
    private String titular;
    private double limiteCredito;
    private double tasaInteresAnual;

    // Constructor 1: Inicialización Completa (4 Parámetros)
    public TarjetaCredito(String numeroTarjeta, String titular, double limiteCredito, double tasaIntere-
sAnual) {
        this.numeroTarjeta = numeroTarjeta;
        this.titular = titular;
        this.limiteCredito = limiteCredito;
        this.tasaInteresAnual = tasaInteresAnual;
    }

    // Constructor 2: Sobrecargado (3 Parámetros - Tasa de interés estándar por defecto: 24.0%)
    public TarjetaCredito(String numeroTarjeta, String titular, double limiteCredito) {
        this(numeroTarjeta, titular, limiteCredito, 24.0);
    }

    // Constructor 3: Sobrecargado (2 Parámetros - Límite estándar $1,000 y Tasa estándar 24.0%)
    public TarjetaCredito(String numeroTarjeta, String titular) {
        this(numeroTarjeta, titular, 1000.0, 24.0);
    }

    // Constructor 4: Sobrecargado (1 Parámetro - Tarjeta Anónima / Temporal)
    public TarjetaCredito(String numeroTarjeta) {
        this(numeroTarjeta, "CLIENTE GENERAL", 500.0, 30.0);
    }

    public void mostrarDetallesTarjeta() {
        System.out.println("=Tarjeta N°: " + numeroTarjeta
                + " | Titular: " + titular
                + " | Límite: $" + limiteCredito
                + " | Tasa Anual: " + tasaInteresAnual + "%");
    }
}

public class EjemploSobrecargaConstructores {
```

```java
    public static void main(String[] args) {
        System.out.println("=== DEMOSTRACIÓN DE SOBRECARGA DE CONSTRUCTORES ===");

        // Invocación del Constructor 1 (4 params)
        TarjetaCredito t1 = new TarjetaCredito("4532-XXXX-1111", "Alejandro Morales", 15000.0, 18.5);

        // Invocación del Constructor 2 (3 params)
        TarjetaCredito t2 = new TarjetaCredito("4532-XXXX-2222", "Brenda Gómez", 5000.0);

        // Invocación del Constructor 3 (2 params)
        TarjetaCredito t3 = new TarjetaCredito("4532-XXXX-3333", "Camilo Ruiz");

        // Invocación del Constructor 4 (1 param)
        TarjetaCredito t4 = new TarjetaCredito("4532-XXXX-4444");

        t1.mostrarDetallesTarjeta();
        t2.mostrarDetallesTarjeta();
        t3.mostrarDetallesTarjeta();
        t4.mostrarDetallesTarjeta();
    }
}
```

5. Ejemplos de Código Completos y Profundizados

Ejemplo 1: Sistema de Gestión de Biblioteca con Contador Estático

Muestra el uso de atributos estáticos para contar objetos globales y la sobrecarga de constructores con `this(...)`.

JAVA

```java
// Archivo: EjemploBiblioteca.java

class Libro {
    // Atributos de Instancia
    String isbn;
    String titulo;
    String autor;
    int numeroPaginas;
    boolean prestado;

    // Atributo Estático (Compartido por todos los libros creados)
    static int totalLibrosRegistrados = 0;
    static int totalLibrosPrestados = 0;

    // Constructor 1: Inicialización completa
    Libro(String isbn, String titulo, String autor, int numeroPaginas) {
        this.isbn = isbn;
        this.titulo = titulo;
        this.autor = autor;
        this.numeroPaginas = numeroPaginas;
        this.prestado = false;

        // Incrementar el contador global estático
        totalLibrosRegistrados++;
    }

    // Constructor 2: Sobrecargado (Solo ISBN y Título, autor Desconocido)
    Libro(String isbn, String titulo) {
        this(isbn, titulo, "Autor Desconocido", 0); // Llama al Constructor 1
```

```
    }

    // Métodos de Instancia
    void prestar() {
        if (!prestado) {
            prestado = true;
            totalLibrosPrestados++; // Modifica estado global estático
            System.out.println(" =Ël libro '" + titulo + "' ha sido prestado.");
        } else {
            System.out.println(" El libro '" + titulo + "' ya se encuentra prestado actualmente.");
        }
    }

    void devolver() {
        if (prestado) {
            prestado = false;
            totalLibrosPrestados--;
            System.out.println(" =El libro '" + titulo + "' ha sido devuelto.");
        } else {
            System.out.println(" El libro '" + titulo + "' no estaba prestado.");
        }
    }

    void mostrarDetalles() {
        System.out.println("LIBRO [" + isbn + "] '" + titulo + "' por " + autor
                           + " (" + numeroPaginas + " pág.) | Estado: " + (prestado ? "PRESTADO" : "DISPONIBLE"));
    }

    // Método Estático Reporte Global
    static void mostrarReporteBiblioteca() {
        System.out.println("\n================================================");
        System.out.println(" REPORTE GENERAL DE LA BIBLIOTECA");
        System.out.println(" Total de Libros Registrados en Sistema: " + totalLibrosRegistrados);
        System.out.println(" Total de Libros Prestados Actualmente : " + totalLibrosPrestados);
        System.out.println(" Total de Libros Disponibles en Estante : " + (totalLibrosRegistrados - to-
talLibrosPrestados));
        System.out.println("================================================\n");
    }
}

public class EjemploBiblioteca {
    public static void main(String[] args) {
        // Crear Libros usando distintos constructores
        Libro l1 = new Libro("978-0134685991", "Effective Java", "Joshua Bloch", 412);
        Libro l2 = new Libro("978-0596009205", "Head First Java", "Kathy Sierra", 720);
        Libro l3 = new Libro("978-0000000000", "Manuscrito Antiguo"); // Usa Constructor 2

        l1.mostrarDetalles();
        l2.mostrarDetalles();
        l3.mostrarDetalles();

        // Mostrar estado inicial estático
        Libro.mostrarReporteBiblioteca();

        // Realizar préstamos
        l1.prestar();
        l3.prestar();
        l1.prestar(); // Intento duplicado

        // Mostrar reporte estático actualizado
        Libro.mostrarReporteBiblioteca();

        // Devolución
        l1.devolver();
        Libro.mostrarReporteBiblioteca();
    }
```

```
95 | }
```

Ejemplo 2: Conversor de Unidades Estático (Clase de Utilidad)

Muestra cómo diseñar una **clase pura de utilidad estática** (similar a `java.lang.Math`), donde no se requiere instanciar objetos.

JAVA

```java
// Archivo: EjemploConversor.java

class ConversorUnidades {
    // Constantes Estáticas Globales
    public static final double KM_A_MILLAS = 0.621371;
    public static final double CELSIUS_A_FAHRENHEIT_MULT = 1.8;
    public static final double KG_A_LIBRAS = 2.20462;

    // Constructor Privado para Evitar Instanciación (Buena Práctica en Clases de Utilidad)
    private ConversorUnidades() {
        throw new IllegalStateException("Clase de utilidad. No debe instanciarse.");
    }

    // Métodos Estáticos de Conversión
    public static double kilometrosAMillas(double km) {
        return km * KM_A_MILLAS;
    }

    public static double millasAKilometros(double millas) {
        return millas / KM_A_MILLAS;
    }

    public static double celsiusAFahrenheit(double celsius) {
        return (celsius * CELSIUS_A_FAHRENHEIT_MULT) + 32;
    }

    public static double fahrenheitACelsius(double fahrenheit) {
        return (fahrenheit - 32) / CELSIUS_A_FAHRENHEIT_MULT;
    }

    public static double kilogramosALibras(double kg) {
        return kg * KG_A_LIBRAS;
    }
}

public class EjemploConversor {
    public static void main(String[] args) {
        System.out.println("=== CONVERSOR DE UNIDADES (MÉTODOS ESTÁTICOS) ===");

        double distanciaKm = 100.0;
        double distanciaMillas = ConversorUnidades.kilometrosAMillas(distanciaKm);
        System.out.println(distanciaKm + " km equivalen a: " + String.format("%.2f", distanciaMillas) +-
 " millas.");

        double tempC = 36.5;
        double tempF = ConversorUnidades.celsiusAFahrenheit(tempC);
        System.out.println(tempC + " °C equivalen a: " + String.format("%.2f", tempF) + " °F.");

        double pesoKg = 75.0;
        double pesoLbs = ConversorUnidades.kilogramosALibras(pesoKg);
        System.out.println(pesoKg + " kg equivalen a: " + String.format("%.2f", pesoLbs) + " lbs.");
    }
}
```

Ejemplo 3: Registro de Usuarios con Generador Correlativo de IDs

Ilustra la autogeneración de códigos identificadores correlativos mediante el incremento de un contador de clase.

JAVA

```java
// Archivo: EjemploUsuarioSistema.java

class UsuarioSistema {
    // Atributo Estático Generador de ID Correlativo
    private static int contadorId = 1000;

    // Atributos de Instancia
    String idUsuario;
    String nombre;
    String email;
    String rol;

    // Constructor Principal
    UsuarioSistema(String nombre, String email, String rol) {
        // Asignación de ID autogenerado correlativo (USR-1001, USR-1002...)
        this.idUsuario = "USR-" + (++contadorId);
        this.nombre = nombre;
        this.email = email;
        this.rol = rol;
    }

    // Constructor Secundario (Asigna Rol por defecto: "CLIENTE")
    UsuarioSistema(String nombre, String email) {
        this(nombre, email, "CLIENTE");
    }

    void imprimirFichaUsuario() {
        System.out.println(" =ID: " + idUsuario + " | Nombre: " + nombre + " | Email: " + email +-
 " | Rol: [" + rol + "]");
    }
}

public class EjemploUsuarioSistema {
    public static void main(String[] args) {
        System.out.println("=== REGISTRO AUTOMÁTICO DE USUARIOS ===");

        UsuarioSistema u1 = new UsuarioSistema("Carlos Mendoza", "carlos@empresa.com", "ADMINISTRADOR");
        UsuarioSistema u2 = new UsuarioSistema("Beatriz Restrepo", "beatriz@gmail.com"); // Rol Cliente
        UsuarioSistema u3 = new UsuarioSistema("Daniel Silva", "daniel@gmail.com");     // Rol Cliente

        u1.imprimirFichaUsuario();
        u2.imprimirFichaUsuario();
        u3.imprimirFichaUsuario();
    }
}
```

Ejemplo 4: Calculadora de Vectores Matemáticos 2D

Muestra cómo una clase representa una entidad matemática abstracta operando entre diferentes instancias.

JAVA

```java
// Archivo: EjemploVector2D.java

class Vector2D {
    double x;
    double y;

    // Constructor 1: Vector nulo (0, 0)
    Vector2D() {
        this(0.0, 0.0);
    }

    // Constructor 2: Vector con componentes
    Vector2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Método para sumar este vector con otro vector recibido como parámetro
    Vector2D sumar(Vector2D otro) {
        return new Vector2D(this.x + otro.x, this.y + otro.y);
    }

    // Método para calcular la magnitud/módulo del vector: (x² + y²)
    double calcularMagnitud() {
        return Math.sqrt((x * x) + (y * y));
    }

    // Método para multiplicar por un escalar
    void multiplicarEscalar(double escalar) {
        this.x *= escalar;
        this.y *= escalar;
    }

    void mostrarVector() {
        System.out.println("Vector (" + x + ", " + y + ") | Magnitud: " + String.format("%.2f", calcularMagnitud()));
    }
}

public class EjemploVector2D {
    public static void main(String[] args) {
        System.out.println("=== OPERACIONES CON VECTORES 2D ===");

        Vector2D v1 = new Vector2D(3.0, 4.0);
        Vector2D v2 = new Vector2D(1.0, 2.0);

        System.out.print("Vector 1: "); v1.mostrarVector();
        System.out.print("Vector 2: "); v2.mostrarVector();

        // Sumar v1 + v2
        Vector2D vSuma = v1.sumar(v2);
        System.out.print("Resultado v1 + v2: "); vSuma.mostrarVector();

        // Escalar v1 por 2.0
        v1.multiplicarEscalar(2.0);
        System.out.print("Vector 1 escalado por 2: "); v1.mostrarVector();
    }
}
```

6. Ejercicios Prácticos Adicionales (Con Solución)

Ejercicio 1: La Factura Comercial

Consigna: Diseña la clase `Factura` con los siguientes requerimientos:

- Atributo estático `IVA = 0.19` (19%).
- Atributos de instancia: `numeroFactura`, `nombreCliente`, `montoSubtotal`, `porcentajeDescuento`.
- Constructores sobrecargados:
- Sin descuento (asigna 0.0%).
- Con descuento personalizado.
- Métodos: `calcularMontoDescuento()`, `calcularIVA()`, `calcularTotalPagar()`, `imprimirFactura()`.

Solución Explicada:

JAVA

```java
class Factura {
    static final double IVA = 0.19; // 19% de IVA estático

    String numeroFactura;
    String nombreCliente;
    double montoSubtotal;
    double porcentajeDescuento; // Valor entre 0 y 100

    // Constructor 1: Con descuento
    Factura(String numeroFactura, String nombreCliente, double montoSubtotal, double porcentajeDescuento) {
        this.numeroFactura = numeroFactura;
        this.nombreCliente = nombreCliente;
        this.montoSubtotal = montoSubtotal;
        this.porcentajeDescuento = porcentajeDescuento;
    }

    // Constructor 2: Sin descuento
    Factura(String numeroFactura, String nombreCliente, double montoSubtotal) {
        this(numeroFactura, nombreCliente, montoSubtotal, 0.0);
    }

    double calcularMontoDescuento() {
        return montoSubtotal * (porcentajeDescuento / 100.0);
    }

    double calcularSubtotalConDescuento() {
        return montoSubtotal - calcularMontoDescuento();
    }

    double calcularMontoIVA() {
        return calcularSubtotalConDescuento() * IVA;
    }

    double calcularTotalPagar() {
        return calcularSubtotalConDescuento() + calcularMontoIVA();
    }

    void imprimirFactura() {
        System.out.println("==================================================");
        System.out.println(" FACTURA DE VENTA N°: " + numeroFactura);
        System.out.println(" Cliente: " + nombreCliente);
        System.out.println(" Subtotal Base   : $" + String.format("%.2f", montoSubtotal));
```

```
        System.out.println(" Descuento (" + porcentajeDescuento + "%): -$" + String.format("%.2f", calcularMontoDescuento()));
        System.out.println(" IVA (19%)        : +$" + String.format("%.2f", calcularMontoIVA()));
        System.out.println(" TOTAL A PAGAR    : $" + String.format("%.2f", calcularTotalPagar()));
        System.out.println("================================================\n");
    }
}

public class PruebaFactura {
    public static void main(String[] args) {
        Factura f1 = new Factura("FAC-001", "Empresa Alpha", 1000.0, 10.0); // 10% Descuento
        Factura f2 = new Factura("FAC-002", "Pedro Pérez", 500.0);          // Sin Descuento

        f1.imprimirFactura();
        f2.imprimirFactura();
    }
}
```

Ejercicio 2: Operaciones con Fracciones Matemáticas

Consigna: Crea una clase `Fraccion` que represente números fraccionarios a/b:

- Atributos: `numerador` (int), `denominador` (int).
- Constructores:
- Sin parámetros: Crea $0/1$.
- Con 1 parámetro: Crea $n/1$.
- Con 2 parámetros: Crea a/b (si $b == 0$, asigna 1 para evitar división por cero).
- Método `multiplicar(Fraccion otra)`: Devuelve un nuevo objeto `Fraccion` con el producto.

Solución Explicada:

JAVA

```java
class Fraccion {
    int numerador;
    int denominador;

    Fraccion() {
        this(0, 1);
    }

    Fraccion(int numerador) {
        this(numerador, 1);
    }

    Fraccion(int numerador, int denominador) {
        this.numerador = numerador;
        if (denominador == 0) {
            System.out.println(" El denominador no puede ser 0. Asignando 1 por defecto.");
            this.denominador = 1;
        } else {
            this.denominador = denominador;
        }
    }

    Fraccion multiplicar(Fraccion otra) {
        int nuevoNum = this.numerador * otra.numerador;
        int nuevoDen = this.denominador * otra.denominador;
```

```
        return new Fraccion(nuevoNum, nuevoDen);
    }

    void mostrar() {
        System.out.println("Fracción: " + numerador + "/" + denominador
                           + " = " + String.format("%.2f", (double)numerador / denominador));
    }
}

public class PruebaFraccion {
    public static void main(String[] args) {
        Fraccion f1 = new Fraccion(3, 4);  // 3/4
        Fraccion f2 = new Fraccion(2, 5);  // 2/5

        System.out.print("F1: "); f1.mostrar();
        System.out.print("F2: "); f2.mostrar();

        Fraccion resultado = f1.multiplicar(f2); // (3*2)/(4*5) = 6/20
        System.out.print("Resultado Multiplicación: "); resultado.mostrar();
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.3

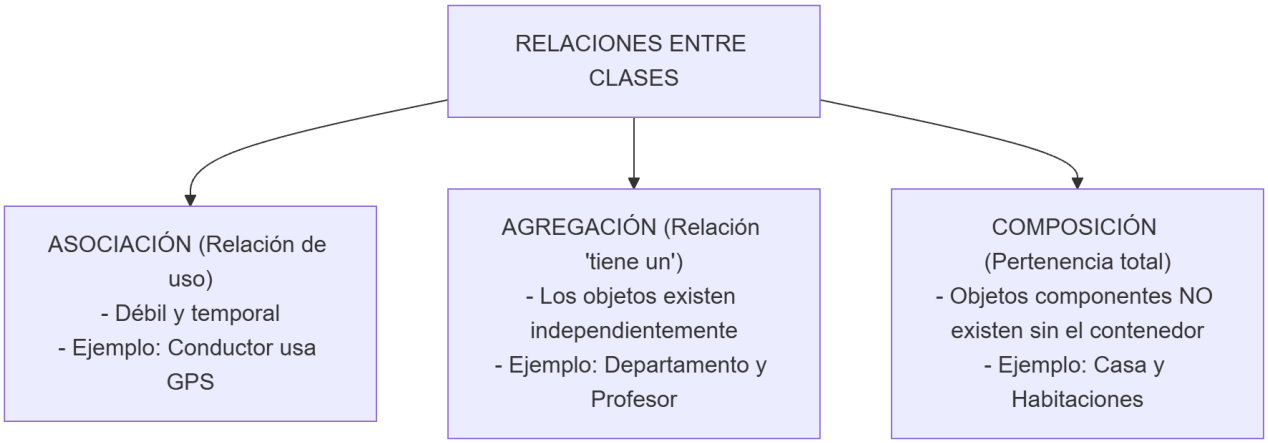
Clase 4: Relaciones entre Clases

Clase 4: Relaciones entre Clases

1. Diagramación y Clasificación de Relaciones

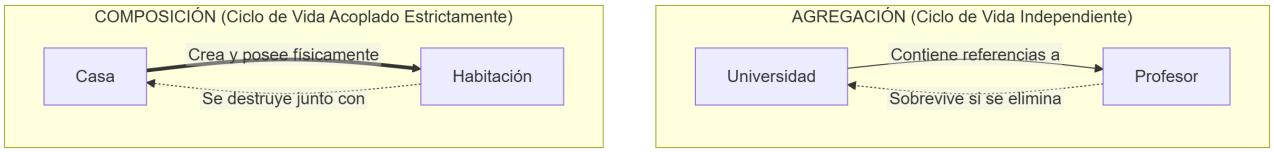
En el análisis de software, los objetos no funcionan aislados. Se comunican y conectan mediante tres tipos de relaciones estructurales principales:

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



2. Comparativa de Ciclo de Vida (*Lifecycle Dependence*)

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



3. Manejo Avanzado de Listas de Objetos (`ArrayList`)

Para manejar relaciones 1-a-Muchos en Java, `ArrayList` provee colecciones dinámicas:

JAVA

```java
import java.util.ArrayList;
```

```
2 |
3 | ArrayList<Empleado> lista = new ArrayList<>();
4 | lista.add(new Empleado("Carlos")); // Agregar
5 | lista.remove(0);                   // Eliminar por índice
6 | lista.size();                      // Tamaño actual
```

4. Ejemplos de Código Completos y Profundizados

Ejemplo 1: Sistema Hospitalario (Hospital, Departamentos, Médicos y Pacientes)

JAVA

```
1 | // Archivo: EjemploHospital.java
2 | import java.util.ArrayList;
3 | import java.util.List;
4 |
5 | class Paciente {
6 |     private String id;
7 |     private String nombre;
8 |
9 |     public Paciente(String id, String nombre) {
10|         this.id = id;
11|         this.nombre = nombre;
12|     }
13|     public String getNombre() { return nombre; }
14| }
15|
16| class Medico {
17|     private String id;
18|     private String nombre;
19|     private String especialidad;
20|     // Asociación: Lista de pacientes atendidos
21|     private List<Paciente> pacientesAtendidos;
22|
23|     public Medico(String id, String nombre, String especialidad) {
24|         this.id = id;
25|         this.nombre = nombre;
26|         this.especialidad = especialidad;
27|         this.pacientesAtendidos = new ArrayList<>();
28|     }
29|
30|     public void atenderPaciente(Paciente p) {
31|         pacientesAtendidos.add(p);
32|         System.out.println("El Dr. " + nombre + " está atendiendo al paciente " + p.getNombre());
33|     }
34|
35|     public String getNombre() { return nombre; }
36| }
37|
38| class Departamento {
39|     private String nombreDepartamento;
40|     // Agregación: Médicos asignados al departamento
41|     private List<Medico> medicos;
42|
43|     public Departamento(String nombreDepartamento) {
44|         this.nombreDepartamento = nombreDepartamento;
45|         this.medicos = new ArrayList<>();
46|     }
47|
```

```
    public void agregarMedico(Medico m) {
        medicos.add(m);
    }

    public void listarMedicos() {
        System.out.println(" Depto. " + nombreDepartamento + " - Médicos:");
        for (Medico m : medicos) {
            System.out.println("  - Dr. " + m.getNombre());
        }
    }
}

class Hospital {
    private String nombre;
    // Composición: El hospital crea y posee sus departamentos
    private List<Departamento> departamentos;

    public Hospital(String nombre) {
        this.nombre = nombre;
        this.departamentos = new ArrayList<>();
    }

    public void crearDepartamento(String nombreDept) {
        departamentos.add(new Departamento(nombreDept));
    }

    public List<Departamento> getDepartamentos() { return departamentos; }
}

public class EjemploHospital {
    public static void main(String[] args) {
        Hospital hosp = new Hospital("Hospital Central");
        hosp.crearDepartamento("Cardiología");
        hosp.crearDepartamento("Pediatría");

        Medico m1 = new Medico("M1", "Ana Gómez", "Cardióloga");
        Paciente p1 = new Paciente("P1", "Juan Pérez");

        hosp.getDepartamentos().get(0).agregarMedico(m1);
        hosp.getDepartamentos().get(0).listarMedicos();

        m1.atenderPaciente(p1);
    }
}
```

Ejemplo 2: Gestión de Empresas, Proyectos y Tareas

JAVA

```
// Archivo: EjemploEmpresaProyectos.java
import java.util.ArrayList;
import java.util.List;

class Tarea {
    private String descripcion;
    private int horasEstimadas;
    private boolean completada;

    public Tarea(String descripcion, int horasEstimadas) {
        this.descripcion = descripcion;
        this.horasEstimadas = horasEstimadas;
        this.completada = false;
```

```
    }

    public String getDescripcion() { return descripcion; }
    public int getHorasEstimadas() { return horasEstimadas; }
}

class Proyecto {
    private String nombreProyecto;
    // Composición: Las tareas son parte integral del proyecto
    private List<Tarea> tareas;

    public Proyecto(String nombreProyecto) {
        this.nombreProyecto = nombreProyecto;
        this.tareas = new ArrayList<>();
    }

    public void agregarTarea(String desc, int horas) {
        tareas.add(new Tarea(desc, horas));
    }

    public int calcularHorasTotales() {
        int total = 0;
        for (Tarea t : tareas) {
            total += t.getHorasEstimadas();
        }
        return total;
    }

    public void mostrarResumen() {
        System.out.println("=Âroyecto: " + nombreProyecto + " | Horas Totales Estimadas: " + calcularHo-
rasTotales() + " hrs.");
        for (Tarea t : tareas) {
            System.out.println("  - Tarea: " + t.getDescripcion() + " (" + t.getHorasEstimadas() + "h)");
        }
    }
}

public class EjemploEmpresaProyectos {
    public static void main(String[] args) {
        Proyecto p = new Proyecto("Rediseño Web ERP");
        p.agregarTarea("Diseño de Maquetas UI", 20);
        p.agregarTarea("Desarrollo de API Rest Java", 50);
        p.agregarTarea("Pruebas de Integración", 15);

        p.mostrarResumen();
    }
}
```

Ejemplo 3: Edificio y Habitaciones (Composición Estricta)

JAVA

```
// Archivo: EjemploEdificio.java
import java.util.ArrayList;
import java.util.List;

class Habitacion {
    private int numero;
    private double metrosCuadrados;

    public Habitacion(int numero, double metrosCuadrados) {
        this.numero = numero;
```

```
        this.metrosCuadrados = metrosCuadrados;
    }

    public void mostrarInfo() {
        System.out.println("  =Habitación N° " + numero + " (" + metrosCuadrados + " m²)");
    }
}

class Edificio {
    private String direccion;
    // Composición Estricta: Las habitaciones se crean DENTRO del Edificio
    private List<Habitacion> habitaciones;

    public Edificio(String direccion, int cantidadHabitaciones) {
        this.direccion = direccion;
        this.habitaciones = new ArrayList<>();

        // Creación interna de componentes
        for (int i = 1; i <= cantidadHabitaciones; i++) {
            habitaciones.add(new Habitacion(100 + i, 25.5));
        }
    }

    public void inspeccionarEdificio() {
        System.out.println("<Edificio en " + direccion + " con " + habitaciones.size() + " habitaciones:");
        for (Habitacion h : habitaciones) {
            h.mostrarInfo();
        }
    }
}

public class EjemploEdificio {
    public static void main(String[] args) {
        Edificio ed = new Edificio("Av. Principal #456", 3);
        ed.inspeccionarEdificio();
    }
}
```

Ejemplo 4: Pedido Comercial y Líneas de Detalle

JAVA

```
// Archivo: EjemploPedidoDetalle.java
import java.util.ArrayList;
import java.util.List;

class ItemProducto {
    private String nombre;
    private double precio;

    public ItemProducto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
}

class LineaDetalle {
    private ItemProducto producto; // Asociación
    private int cantidad;

```

```
    public LineaDetalle(ItemProducto producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
    }

    public double getSubtotal() {
        return producto.getPrecio() * cantidad;
    }

    public void imprimirLinea() {
        System.out.println("  x" + cantidad + " " + producto.getNombre() + " @ $" + producto.getPrecio-
() + " c/u = $" + getSubtotal());
    }
}

class Pedido {
    private String idPedido;
    private List<LineaDetalle> lineas; // Composición

    public Pedido(String idPedido) {
        this.idPedido = idPedido;
        this.lineas = new ArrayList<>();
    }

    public void agregarLinea(ItemProducto p, int cant) {
        lineas.add(new LineaDetalle(p, cant));
    }

    public double calcularTotalPedido() {
        double total = 0;
        for (LineaDetalle l : lineas) {
            total += l.getSubtotal();
        }
        return total;
    }

    public void imprimirFacturaPedido() {
        System.out.println(">PEDIDO N° " + idPedido);
        for (LineaDetalle l : lineas) {
            l.imprimirLinea();
        }
        System.out.println(" TOTAL PEDIDO: $" + calcularTotalPedido());
    }
}

public class EjemploPedidoDetalle {
    public static void main(String[] args) {
        ItemProducto p1 = new ItemProducto("Monitor LED", 200.0);
        ItemProducto p2 = new ItemProducto("Teclado USB", 25.0);

        Pedido ped = new Pedido("PED-990");
        ped.agregarLinea(p1, 2); // 2 Monitores
        ped.agregarLinea(p2, 1); // 1 Teclado

        ped.imprimirFacturaPedido();
    }
}
```

5. Ejercicios Prácticos Adicionales (Con Solución)

Ejercicio 1: Playlist de Música

JAVA

```java
import java.util.ArrayList;
import java.util.List;

class Cancion {
    private String titulo;
    private int duracionSegundos;

    public Cancion(String titulo, int duracionSegundos) {
        this.titulo = titulo;
        this.duracionSegundos = duracionSegundos;
    }
    public String getTitulo() { return titulo; }
    public int getDuracionSegundos() { return duracionSegundos; }
}

class Playlist {
    private String nombre;
    private List<Cancion> canciones = new ArrayList<>();

    public Playlist(String nombre) { this.nombre = nombre; }

    public void agregarCancion(Cancion c) { canciones.add(c); }

    public int getDuracionTotalSegundos() {
        int total = 0;
        for (Cancion c : canciones) total += c.getDuracionSegundos();
        return total;
    }

    public void reproducir() {
        System.out.println("<¶reproduciendo Playlist: " + nombre + " (" + getDuracionTotalSegundos() +-
 " segs total)");
        for (Cancion c : canciones) {
            System.out.println(" ¶Sonando: " + c.getTitulo() + " (" + c.getDuracionSegundos() + "s)");
        }
    }
}

public class PruebaPlaylist {
    public static void main(String[] args) {
        Cancion c1 = new Cancion("Bohemian Rhapsody", 354);
        Cancion c2 = new Cancion("Hotel California", 391);

        Playlist p = new Playlist("Mis Favoritas Rock");
        p.agregarCancion(c1);
        p.agregarCancion(c2);

        p.reproducir();
    }
}
```

Ejercicio 2: Carrito de Supermercado

JAVA

```java
import java.util.ArrayList;
```

```java
import java.util.List;

class ProductoBasico {
    private String nombre;
    private double precio;

    public ProductoBasico(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
}

class CarritoSupermercado {
    private List<ProductoBasico> productos = new ArrayList<>();

    public void agregar(ProductoBasico p) { productos.add(p); }

    public double obtenerTotal() {
        double t = 0;
        for (ProductoBasico p : productos) t += p.getPrecio();
        return t;
    }
}

public class PruebaCarrito {
    public static void main(String[] args) {
        CarritoSupermercado c = new CarritoSupermercado();
        c.agregar(new ProductoBasico("Leche", 1.50));
        c.agregar(new ProductoBasico("Pan", 0.80));

        System.out.println("Total Carrito: $" + c.obtenerTotal());
    }
}
```

En Programación Orientada a Objetos (POO), estas tres relaciones representan distintas formas en las que dos clases o componentes se conectan entre sí. La diferencia principal radica en qué tan fuerte es esa dependencia y qué pasa con el ciclo de vida de los objetos.

1. Asociación ("Usan" o "Se relacionan")

Es la relación más general y débil. Ocurre cuando un objeto **usa** a otro o interactúa con él, pero **ambos son totalmente independientes** y tienen vidas separadas.

- **Ejemplo real:** Un `Estudiante` y un `Curso`. El estudiante se inscribe a un curso, pero si el curso se cancela, el estudiante sigue existiendo.
- **En Java:** Generalmente se pasa como parámetro en un método o se guarda como una referencia básica.

JAVA

```java
class Estudiante {
    String nombre;

    void inscribirCurso(Curso curso) {
        // Usa la instancia del curso de manera temporal o independiente
    }
```

```
7 | }
```

2. Agregación ("Tiene un" - Relación Débil)

Es un tipo especial de asociación más específica donde un objeto "posee" o "contiene" a otro, pero **el objeto contenido puede existir de forma independiente** si el objeto contenedor desaparece.

- **Ejemplo real:** Un `Carro` y una `Rueda`. Un carro tiene ruedas, pero si destruyes el carro, las ruedas aún existen como objetos individuales que podrías montar en otro vehículo.
- **En Java:** El objeto hijo se pasa por el constructor o setter; no se crea dentro de la clase contenedora.

JAVA

```java
class Rueda {
    String marca;
    public Rueda(String marca) { this.marca = marca; }
}

class Carro {
    private Rueda rueda;

    // Recibe la rueda ya creada desde afuera
    public Carro(Rueda rueda) {
        this.rueda = rueda;
    }
}
```

3. Composición ("Es parte de" - Relación Fuerte)

Es la forma más estricta de agregación. Aquí hay una relación de dependencia vital: **el objeto contenido no puede existir sin el objeto contenedor**. Si destruyes el objeto principal, los objetos internos se destruyen automáticamente con él.

- **Ejemplo real:** Una `Casa` y una `Habitacion`. La habitación no puede existir de forma independiente sin la casa que la contiene.
- **En Java:** El objeto hijo se instancia (con `new`) dentro del propio constructor o clase contenedora, controlando su ciclo de vida.

JAVA

```java
class Habitacion {
    String tipo;
    public Habitacion(String tipo) { this.tipo = tipo; }
}

class Casa {
    private Habitacion habitacion;

    public Casa() {
        // La casa crea directamente su propia habitación
        this.habitacion = new Habitacion("Dormitorio principal");
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.4

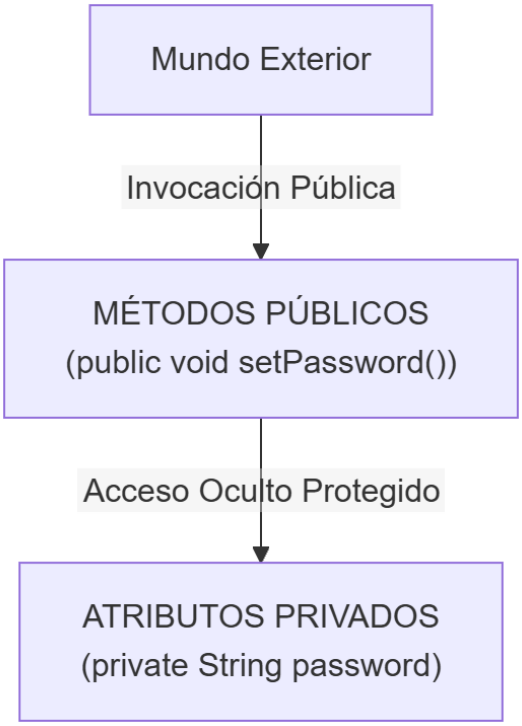
Clase 3: Encapsulamiento y Modificadores de Acceso

Clase 3: Encapsulamiento y Modificadores de Acceso

1. El Principio de Ocultamiento de Información y Principio de Menor Privilegio

El **Encapsulamiento** es el pilar de la POO que consiste en ocultar los detalles internos de implementación de una clase y exponer únicamente una interfaz pública limpia y segura.

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



El Principio de Menor Privilegio (*Principle of Least Privilege - POLP*)

Este principio establece que **cada entidad de software (clase, atributo o método) debe tener acceso únicamente a la información y recursos que sean estrictamente necesarios para su funcionamiento**.

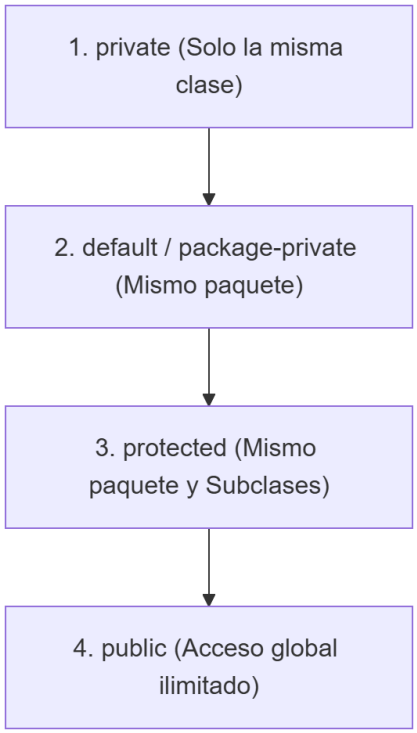
¿Por qué aplicar el Principio de Menor Privilegio en Java?

1. **Evita la Corrupción Accidental de Datos:** Si un atributo es `public`, cualquier parte del código puede cambiar su valor por error a un estado inválido (ej. asignar `edad = -200`).
2. **Reduce el Acoplamiento (*Decoupling*):** Al ocultar la implementación interna, puedes cambiar cómo se almacenan o calculan los datos sin romper el código de los clientes que usan tu clase.
3. **Facilita la Depuración (*Debugging*):** Si el estado de un objeto se corrompe, no tienes que revisar miles de líneas de código en todo el proyecto; solo debes revisar los métodos de la clase donde el atributo está encapsulado.

2. Modificadores de Acceso en Java: `private`, `default`, `protected` y `public`

Java proporciona cuatro niveles de visibilidad para controlar el acceso a los miembros de una clase:

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



Detalle de cada Modificador de Acceso:

A. `private` (Acceso Privado)

- **Visibilidad:** Únicamente accesible dentro de la **misma clase** donde fue declarado.
- **Uso Recomendado:** Se debe aplicar a **todos los atributos de instancia** y a métodos auxiliares internos que no deban ser llamados desde el exterior.

JAVA

```java
public class Cuenta {
    private double saldo; // Nadie fuera de la clase Cuenta puede tocar directamente 'saldo'
}
```

B. *package-private* / Default (Acceso por Defecto de Paquete)

- **Visibilidad:** Accesible por cualquier clase que pertenezca al **mismo paquete** (*package*). No requiere ninguna palabra clave (se aplica cuando no pones ningún modificador).
- **Uso Recomendado:** Útil para clases o métodos colaboradores dentro de un mismo módulo/paquete que no deben exponerse como API pública del sistema.

JAVA

```
class ValidadorInterno { // Sin palabra clave -> package-private
    void validarFormato() { ... }
}
```

C. `protected` (Acceso Protegido)

- **Visibilidad:** Accesible desde el **mismo paquete** y desde **subclases** (clases hijas que extienden de ella), incluso si las subclases están en paquetes diferentes.
- **Uso Recomendado:** Para atributos o métodos pensados para ser heredados y personalizados por clases derivadas.

JAVA

```
public class Vehiculo {
    protected String marca; // Las clases hijas como 'Auto' o 'Moto' pueden acceder directamente
}
```

D. `public` (Acceso Público Global)

- **Visibilidad:** Accesible desde **cualquier lugar** del programa sin restricciones.
- **Uso Recomendado:** Para la API pública de la clase (constructores y métodos Getters/Setters/Negocio).

Tabla Comparativa de Visibilidad

MODIFICADOR	MISMA CLASE	MISMO PAQUETE	SUBCLASES (`EXTENDS`)	CUALQUIER LUGAR
`private`	**SÍ**	NO	NO	NO
(default / package)	**SÍ**	**SÍ**	NO	NO
`protected`	**SÍ**	**SÍ**	**SÍ**	NO
`public`	**SÍ**	**SÍ**	**SÍ**	**SÍ**

---

3. Métodos de Acceso y Modificación: Getters y Setters

Dado que los atributos deben ser `private`, la forma estándar de interactuar con ellos es a través de métodos públicos conocidos como **Getters** (lectores) y **Setters** (modificadores).

A. Convenciones de Nombres (JavaBeans Standard)

1. **Getter:** Comienza con `get` seguido del nombre del atributo con la primera letra en mayúscula (CamelCase). Retorna el tipo de dato del atributo.
2. *Excepción para Booleanos:* Se suele usar el prefijo `is` o `has` en lugar de `get` (ej. `isActivo()`, `hasPermiso()`).
3. **Setter:** Comienza con `set` seguido del nombre del atributo en CamelCase. Recibe un parámetro del tipo del atributo y retorna `void`.

JAVA

```
public class Persona {
```

```
    private String nombre;
    private boolean activo;

    // Getter para String
    public String getNombre() {
        return this.nombre;
    }

    // Setter para String
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Getter para boolean (usa 'is')
    public boolean isActivo() {
        return this.activo;
    }
}
```

B. Tipos de Propiedades según Getters y Setters

- **Propiedad de Lectura y Escritura:** Tiene tanto Getter como Setter públicos.
- *Propiedad de Solo Lectura (*Read-Only*):* Solo tiene Getter público (el valor se asigna en el constructor y no se puede modificar después).
- *Propiedad de Solo Escritura (*Write-Only*):* Solo tiene Setter público (ej. un campo de contraseña donde nunca permites consultar el valor almacenado).
- **Propiedad Calculada / Derivada:** Método Getter que calcula un resultado en tiempo real sin tener un atributo físico almacenado.

JAVA

```
// Ejemplo de Propiedad Calculada Dinámica
public double getPrecioConIVA() {
    return this.precioBase * 1.19; // Se calcula al vuelo, no requiere atributo 'precioConIVA'
}
```

4. Invariantes de Clase y Validación de Reglas de Negocio en Setters

¿Qué es un Invariante de Clase?

Un **Invariante de Clase** es una condición lógica o regla de negocio que **debe mantenerse VERDADERA durante todo el ciclo de vida de un objeto**, desde el momento en que se construye hasta que se destruye.

> **NOTA CLAVE**
> **Ejemplos de Invariantes de Clase:** - *"El saldo de una cuenta corriente de ahorro nunca puede ser menor que 0." - "El stock de un producto en bodega no puede ser negativo." - "La temperatura de un congelador industrial debe estar entre -30°C y 5°C."*

Validación de Reglas de Negocio en los Setters

Los Setters son los guardianes encargados de proteger los invariantes de clase. Antes de modificar el atributo, el Setter debe **verificar que el nuevo valor cumpla con las reglas de negocio**. Si la validación falla, se debe rechazar el cambio o lanzar una excepción.

JAVA

```java
public class CuentaBancaria {
    private double saldo;

    // El Setter protege el invariante "saldo >= 0"
    public void setSaldo(double nuevoSaldo) {
        if (nuevoSaldo >= 0) {
            this.saldo = nuevoSaldo;
        } else {
            // Rechazamos el cambio para proteger el invariante
            throw new IllegalArgumentException("ERROR DE INVARIANTE: El saldo no puede ser negativo: $"+ nuevoSaldo);
        }
    }
}
```

5. Clases Inmutables (*Immutable Objects*) y Copia Defensiva

Un **Objeto Inmutable** es aquel cuyo estado no se puede modificar de ninguna manera una vez construido (ejemplo clásico en Java: `java.lang.String`).

Reglas para diseñar una Clase Inmutable:

1. Declarar la clase como **`final`** (para evitar que subclases sobreescriban métodos).
2. Declarar todos los atributos como **`private final`**.
3. **No proporcionar ningún método Setter**.
4. *Copia Defensiva (*Defensive Copy*):* Si la clase contiene referencias a objetos mutables (ej. `java.util.Date` o `ArrayList`), debes hacer una copia del objeto en el constructor y en el Getter para que el cliente no pueda modificar el objeto interno desde fuera.

6. Ejemplos de Código Completos y Profundizados

Ejemplo 1: Control de Inventario con Validaciones e Invariantes de Clase

JAVA

```java
// Archivo: EjemploProductoTienda.java

class ProductoTienda {
    private String sku;
    private String nombre;
    private double precioCosto;
    private double precioVenta;
    private int stock;

```

```
    public ProductoTienda(String sku, String nombre, double precioCosto, double precioVenta, int stockInicial) {
        this.sku = sku;
        setNombre(nombre);
        setPrecioCosto(precioCosto);
        setPrecioVenta(precioVenta);
        setStock(stockInicial);
    }

    public String getSku() { return sku; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) {
        if (nombre != null && !nombre.trim().isEmpty()) {
            this.nombre = nombre.trim();
        } else {
            System.out.println("LERROR: El nombre del producto no puede estar vacío.");
        }
    }

    public double getPrecioCosto() { return precioCosto; }
    public void setPrecioCosto(double precioCosto) {
        if (precioCosto > 0) {
            this.precioCosto = precioCosto;
        } else {
            System.out.println("LERROR: El precio de costo debe ser positivo.");
        }
    }

    public double getPrecioVenta() { return precioVenta; }
    public void setPrecioVenta(double precioVenta) {
        // Regla de Negocio / Invariante: precioVenta debe ser superior a precioCosto
        if (precioVenta > this.precioCosto) {
            this.precioVenta = precioVenta;
        } else {
            System.out.println("LERROR: El precio de venta ($" + precioVenta + ") debe ser mayor al precio de costo ($" + this.precioCosto + ").");
        }
    }

    public int getStock() { return stock; }
    public void setStock(int stock) {
        // Invariante: stock >= 0
        if (stock >= 0) {
            this.stock = stock;
        } else {
            System.out.println("LERROR: El stock no puede ser negativo.");
        }
    }

    // Propiedad Calculada (Sin atributo físico almacenado)
    public double getMargenGanancia() {
        return precioVenta - precioCosto;
    }

    public void imprimirFicha() {
        System.out.println("PRODUCTO [" + sku + "] " + nombre + " | Costo: $" + precioCosto
                           + " | Venta: $" + precioVenta + " | Ganancia: $" + getMargenGanancia() + " | Stock: " + stock);
    }
}

public class EjemploProductoTienda {
    public static void main(String[] args) {
        ProductoTienda p = new ProductoTienda("SKU-99", "Teclado Mecánico", 30.0, 50.0, 20);
        p.imprimirFicha();
```

```
73 |
74 |         System.out.println("\n--- Intentando violar invariantes y reglas de negocio ---");
75 |         p.setPrecioVenta(20.0); // Error: menor al costo
76 |         p.setStock(-5);         // Error: stock negativo
77 |         p.imprimirFicha();
78 |     }
79 | }
```

Ejemplo 2: Sistema de Credenciales y Hashing Simulado

JAVA

```java
// Archivo: EjemploUsuarioSeguridad.java

class UsuarioSeguridad {
    private String username;
    private String passwordHash;
    private boolean cuentaBloqueada;

    public UsuarioSeguridad(String username, String passwordPlana) {
        this.username = username;
        setPassword(passwordPlana);
        this.cuentaBloqueada = false;
    }

    public String getUsername() { return username; }
    public boolean isCuentaBloqueada() { return cuentaBloqueada; }

    public void setPassword(String nuevaPasswordPlana) {
        // Validación de regla de seguridad
        if (nuevaPasswordPlana != null && nuevaPasswordPlana.length() >= 8) {
            this.passwordHash = simularHash(nuevaPasswordPlana);
            System.out.println("=Contraseña actualizada de forma segura (Hashed).");
        } else {
            System.out.println("LERROR: La contraseña debe tener al menos 8 caracteres.");
        }
    }

    public boolean validarPassword(String passwordPlana) {
        if (cuentaBloqueada) {
            System.out.println("LCuenta bloqueada por seguridad.");
            return false;
        }
        return this.passwordHash.equals(simularHash(passwordPlana));
    }

    public void bloquearCuenta() {
        this.cuentaBloqueada = true;
        System.out.println(" La cuenta de " + username + " ha sido BLOQUEADA.");
    }

    private String simularHash(String texto) {
        return "HASH_" + texto.hashCode() + "_SECURE";
    }
}

public class EjemploUsuarioSeguridad {
    public static void main(String[] args) {
        UsuarioSeguridad user = new UsuarioSeguridad("admin_boss", "ClaveSegura123");

        System.out.println("Intentando login con clave errónea: " + user.validarPassword("1234"));
```

```
        System.out.println("Intentando login con clave correcta: " + user.validarPassword("ClaveSegura123"));

        user.setPassword("123"); // Rechazado por corta
        user.bloquearCuenta();
        System.out.println("Intentando login post-bloqueo: " + user.validarPassword("ClaveSegura123"));
    }
}
```

Ejemplo 3: Patrón de Clase Inmutable (`Punto2DInmutable`)

JAVA

```java
// Archivo: EjemploPuntoInmutable.java

public final class Punto2DInmutable {
    private final double x;
    private final double y;

    public Punto2DInmutable(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return x; }
    public double getY() { return y; }

    // En lugar de modificar el objeto actual, los métodos devuelven NUEVAS instancias
    public Punto2DInmutable mover(double deltaX, double deltaY) {
        return new Punto2DInmutable(this.x + deltaX, this.y + deltaY);
    }

    @Override
    public String toString() {
        return "Punto2D(" + x + ", " + y + ")";
    }
}

class PruebaPunto {
    public static void main(String[] args) {
        Punto2DInmutable p1 = new Punto2DInmutable(10.0, 20.0);
        System.out.println("P1 Original: " + p1);

        // Mover genera un NUEVO objeto; p1 sigue intacto
        Punto2DInmutable p2 = p1.mover(5.0, -2.0);
        System.out.println("P1 tras mover: " + p1);
        System.out.println("P2 Nuevo     : " + p2);
    }
}
```

Ejemplo 4: Perfil de Empleado con Propiedades de Solo Lectura

JAVA

```java
// Archivo: EjemploPerfilEmpleado.java
import java.time.LocalDate;
import java.time.Period;

```

```java
class PerfilEmpleado {
    private final String cedula; // Propiedad de Solo Lectura
    private String nombre;
    private final LocalDate fechaIngreso; // Propiedad de Solo Lectura

    public PerfilEmpleado(String cedula, String nombre, LocalDate fechaIngreso) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.fechaIngreso = fechaIngreso;
    }

    public String getCedula() { return cedula; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public LocalDate getFechaIngreso() { return fechaIngreso; }

    // Propiedad Calculada Dinámica (Sin atributo de estado almacenado)
    public int getAntiguedadAnos() {
        return Period.between(fechaIngreso, LocalDate.now()).getYears();
    }

    public void mostrarPerfil() {
        System.out.println("Empleado [" + cedula + "] " + nombre
                           + " | Fecha Ingreso: " + fechaIngreso + " | Antigüedad: " + getAntiguedadAnos-
() + " años.");
    }
}

public class EjemploPerfilEmpleado {
    public static void main(String[] args) {
        PerfilEmpleado emp = new PerfilEmpleado("CC-1092837", "Laura Restrepo", LocalDate.of(2018, 5, 15));
        emp.mostrarPerfil();
    }
}
```

7. Ejercicios Prácticos Adicionales (Con Solución)

Ejercicio 1: El Sensor de Temperatura Industrial

Consigna: Crea la clase `SensorTemperatura`:

- Atributos privados: `temperaturaActual`, `limiteMaximoPermitido`.
- Encapsula con Setters que disparen alertas en consola si se sobrepasa el límite máximo.

JAVA

```java
class SensorTemperatura {
    private double temperaturaActual;
    private double limiteMaximoPermitido;

    public SensorTemperatura(double limiteMaximoPermitido) {
        this.limiteMaximoPermitido = limiteMaximoPermitido;
        this.temperaturaActual = 20.0;
    }

    public double getTemperaturaActual() { return temperaturaActual; }

    public void setTemperaturaActual(double nuevaTemp) {
```

```
        this.temperaturaActual = nuevaTemp;
        if (this.temperaturaActual > limiteMaximoPermitido) {
            System.err.println("=ALERTA CRÍTICA: Temperatura (" + temperaturaActual + "°C) su-
peró el límite de " + limiteMaximoPermitido + "°C!");
        }
    }
}

public class PruebaSensor {
    public static void main(String[] args) {
        SensorTemperatura s = new SensorTemperatura(80.0);
        s.setTemperaturaActual(50.0);
        s.setTemperaturaActual(95.0); // Dispara Alerta
    }
}
```

Ejercicio 2: Tarjeta de Débito con Bloqueo de PIN

Consigna: Implementa una clase `TarjetaDebito` que bloquee la tarjeta tras 3 intentos fallidos de ingresar el PIN.

JAVA

```java
class TarjetaDebito {
    private String numeroTarjeta;
    private String pinSeguridad;
    private int intentosFallidos;
    private boolean bloqueada;

    public TarjetaDebito(String numeroTarjeta, String pinInicial) {
        this.numeroTarjeta = numeroTarjeta;
        this.pinSeguridad = pinInicial;
        this.intentosFallidos = 0;
        this.bloqueada = false;
    }

    public boolean validarPin(String pinIngresado) {
        if (bloqueada) {
            System.out.println("LTARJETA BLOQUEADA. Diríjase a su banco.");
            return false;
        }

        if (this.pinSeguridad.equals(pinIngresado)) {
            intentosFallidos = 0; // Reinicia contador tras éxito
            System.out.println(" PIN Correcto.");
            return true;
        } else {
            intentosFallidos++;
            System.out.println("LPIN Incorrecto. Intentos restantes: " + (3 - intentosFallidos));
            if (intentosFallidos >= 3) {
                bloqueada = true;
                System.err.println("=MÁXIMO DE INTENTOS SUPERADO. Tarjeta bloqueada automáticamente.");
            }
            return false;
        }
    }
}

public class PruebaTarjeta {
    public static void main(String[] args) {
        TarjetaDebito t = new TarjetaDebito("4532-1111", "1234");
        t.validarPin("0000"); // Fallo 1
        t.validarPin("9999"); // Fallo 2
```

```
        t.validarPin("1234"); // Éxito (Resetea contador)
        t.validarPin("1111"); // Fallo 1
        t.validarPin("2222"); // Fallo 2
        t.validarPin("3333"); // Fallo 3 (Bloqueo)
        t.validarPin("1234"); // Rechazado por bloqueo
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.5

Clase 5: Herencia

Clase 5: Herencia y Reutilización de Código

La **herencia en Java es un mecanismo de la programación orientada a objetos que permite a una clase nueva (clase hija o subclase) reutilizar los atributos y métodos de una clase existente (clase madre o superclase) mediante la palabra clave `extends`. Esto evita la duplicación de código, facilita su mantenimiento y establece una relación jerárquica de tipo *"es un"* (por ejemplo, un `Perro` *es un* `Animal`). Además, permite la sobrescritura de métodos (`@Override`), lo que habilita el polimorfismo para personalizar el comportamiento en las subclases.**

La Prueba Estricta del "ES UN" (*IS-A Test*)

Antes de usar herencia, verifica que se cumpla la relación semántica **"ES UN"**:

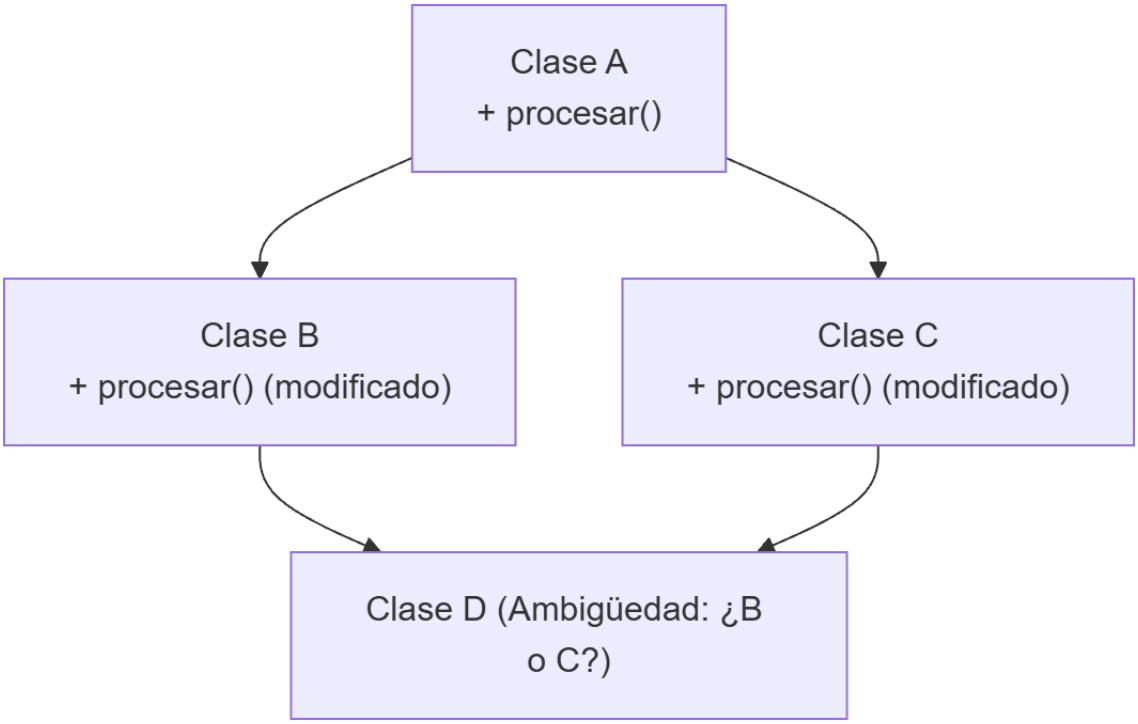
- **¿Un `Profesor` ES UNA `Persona`? SÍ.** $\rightarrow$ Usar Herencia (`class Profesor extends Persona`).
- **¿Un `CuentaAhorros` ES UNA `CuentaBancaria`? SÍ.** $\rightarrow$ Usar Herencia (`class CuentaAhorros extends CuentaBancaria`).
- **¿Un `Cliente` ES UN `Factura`? NO.** (Una factura *pertenece a* o *relaciona a* un cliente) $\rightarrow$ Usar **Asociación**.
- **¿Un `Edificio` ES UN `Ascensor`? NO.** (Un edificio *tiene* ascensores) $\rightarrow$ Usar **Composición**.

Herencia Simple en Java y el Problema del Diamante

Java admite **únicamente Herencia Simple de Clases** (una subclase solo puede extender directamente de una única superclase).

Esto previene el **Problema del Diamante** (*Diamond Problem*), donde una clase `D` que heredara de `B` y `C` simultáneamente no sabría qué versión de un método ambiguo ejecutar:

DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



2. Sintaxis y Mecanismos de Java: `extends` y `super`

La palabra clave `extends`

Sintaxis en Java:

JAVA

```java
public class Subclase extends Superclase {
    // Nuevos atributos y métodos o redefinidos
}
```

La palabra clave `super`

1. **Invocación de Constructores (`super(...)`):**

Garantiza que la superclase inicialice primero sus atributos. **Debe ser la primera línea dentro del constructor de la subclase.**

JAVA

```java
    public class Persona {
        protected String cedula;
        protected String nombre;

        public Persona(String cedula, String nombre) {
```

```
            this.cedula = cedula;
            this.nombre = nombre;
        }
    }

    public class Estudiante extends Persona {
        private String codigoEstudiantil;

        public Estudiante(String cedula, String nombre, String codigoEstudiantil) {
            super(cedula, nombre); //  OBLIGATORIO: Invocación inmediata a la superclase
            this.codigoEstudiantil = codigoEstudiantil;
        }
    }
```

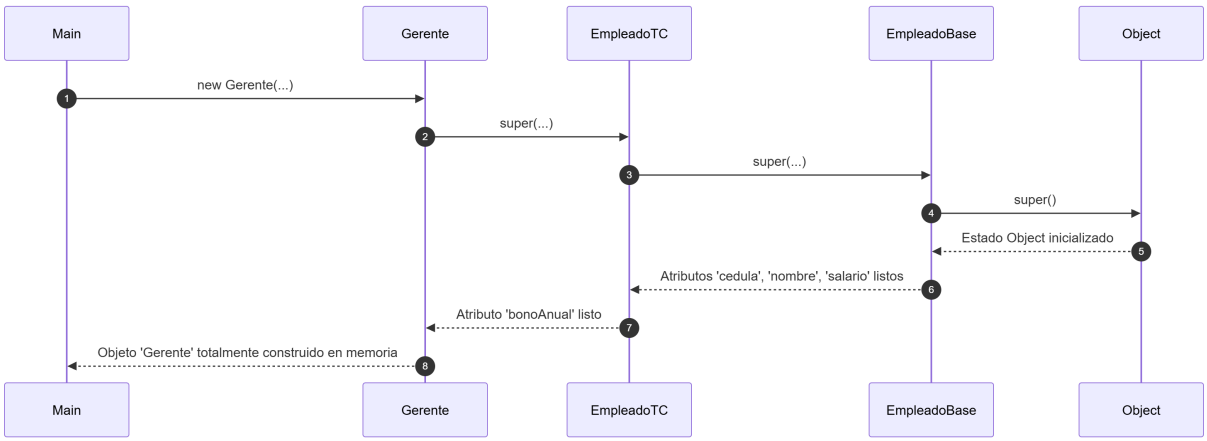
1. **Acceso a Métodos Sobrescritos (`super.metodo()`):**

Permite ejecutar el comportamiento original de la superclase y luego extenderlo en la subclase.

Cadena de Construcción (*Constructor Chaining*)

Al instanciar un objeto de una subclase, la ejecución de constructores se propaga en orden ascendente hasta `java.lang.Object` y se resuelve en orden descendente:

DIAGRAMA DE ARQUITECTURA / FLUJO — Mermaid Engine



---

3. Modificadores de Acceso y Visibilidad en Herencia

MODIFICADOR	MISMO CLASE	MISMO PAQUETE	SUBCLASE (`EXTENDS`)	GLOBAL (`PUBLIC`)
`private`	**SÍ**	NO	NO	NO
(Default / Package)	**SÍ**	**SÍ**	NO (fuera de paquete)	NO
`protected`	**SÍ**	**SÍ**	**SÍ** (incluso en otro paquete)	NO
`public`	**SÍ**	**SÍ**	**SÍ**	**SÍ**

> **CONSEJO PRÁCTICO**
> **TIP TIP TIP TIP Buena Práctica:** Mantén las variables de instancia `private`. Si deseas permitir que las subclases lean o modifiquen un valor, provee getters y setters `protected`.

---

4. Sobrescritura de Métodos (`@Override`) y Restricciones `final`

Sobrescritura (*Overriding*) vs Sobrecarga (*Overloading*)

- **Sobrescritura:** Ocurre entre subclase y superclase. Mantiene **idéntica firma de método** (nombre, tipos y cantidad de parámetros). Se evalúa en tiempo de ejecución.
- **Sobrecarga:** Ocurre dentro de la misma clase. Mantiene el mismo nombre pero **cambia la lista de parámetros**. Se evalúa en tiempo de compilación.

Restricciones con la palabra clave `final`

1. **Clase `final`:** Impide que otras clases la extiendan (ej. `final class DaviplataServicio`).
2. **Método `final`:** Impide que las subclases sobrescriban dicho método (ej. `public final boolean validarCedula()`).

---

5. La Clase Raíz `java.lang.Object` y sus Métodos Clave

Toda clase en Java hereda implícitamente de `java.lang.Object`.

Métodos Esenciales a Sobrescribir:

1. `public String toString()`: Representación en texto formateado del objeto.
2. `public boolean equals(Object obj)`: Comparación lógica basada en identificadores únicos (ej. Cédula `CC` o NIT).
3. `public int hashCode()`: Generación del código hash consecuente con `equals()`.

---

6. Operador `instanceof` y Pattern Matching

Permite validar el tipo real de un objeto antes de realizar un casteo:

JAVA

```java
// Pattern Matching (Java 16+)
if (persona instanceof EstudianteUni est && est.getPromedio() >= 4.5) {
    System.out.println("Candidato a Beca Excelencia: " + est.getNombre());
}
```

---

7. Ejemplos de Código Completos (Contexto Empresarial Colombiano)

Ejemplo 1: Jerarquía Empresarial y Nómina Colombiana (SMLMV y Auxilio de Transporte)

Este ejemplo modela empleados en Colombia. Se calcula el salario neto aplicando deducciones legales de seguridad social (Salud 4% y Pensión 4%) y auxilio de transporte para quienes ganan hasta 2 SMLMV (Salarios Mínimos Legales Mensuales Vigentes).

JAVA

```java
// Archivo: EjemploNominaColombia.java
import java.util.Objects;

class EmpleadoColombiano {
    protected String cedula;
    protected String nombre;
    protected double salarioBase; // En Pesos Colombianos (COP)

    // Constante SMLMV (Salario Mínimo Legal Mensual Vigente en Colombia)
    public static final double SMLMV = 1300000.0;
    public static final double AUXILIO_TRANSPORTE = 162000.0;

    public EmpleadoColombiano(String cedula, String nombre, double salarioBase) {
        if (salarioBase < SMLMV) {
            throw new IllegalArgumentException("El salario no puede ser inferior al SMLMV de Colombia ($"-
 + SMLMV + ")");
        }
        this.cedula = cedula;
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public double calcularSalarioNeto() {
        double salud = salarioBase * 0.04;
        double pension = salarioBase * 0.04;
        double auxilio = (salarioBase <= (2 * SMLMV)) ? AUXILIO_TRANSPORTE : 0.0;

        return salarioBase - salud - pension + auxilio;
    }

    public String getCedula() { return cedula; }
    public String getNombre() { return nombre; }

    @Override
    public String toString() {
        return String.format("CC: %-11s | Nom-
bre: %-15s | Salario Base: $%,.0f COP | Neto Mes: $%,.0f COP",
                cedula, nombre, salarioBase, calcularSalarioNeto());
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        EmpleadoColombiano queEmp = (EmpleadoColombiano) o;
        return Objects.equals(cedula, queEmp.cedula);
    }

    @Override
    public int hashCode() {
        return Objects.hash(cedula);
    }
}

```

```java
class EmpleadoTiempoCompleto extends EmpleadoColombiano {
    private double primaSemestral;

    public EmpleadoTiempoCompleto(String cedula, String nombre, double salarioBase, double primaSemestral) {
        super(cedula, nombre, salarioBase);
        this.primaSemestral = primaSemestral;
    }

    @Override
    public double calcularSalarioNeto() {
        // En los meses de junio y diciembre se suma la prima legal semestral
        return super.calcularSalarioNeto() + (primaSemestral / 6.0);
    }

    @Override
    public String toString() {
        return String.format("%s | Prima Prorrateada: $%,.0f COP", super.toString(), (primaSemestral / 6.0));
    }
}

class GerenteRegional extends EmpleadoTiempoCompleto {
    private String ciudad;
    private double bonoRepresentacion;

    public GerenteRegional(String cedula, String nombre, double salarioBase, double primaSemestral, String ciudad, double bonoRepresentacion) {
        super(cedula, nombre, salarioBase, primaSemestral);
        this.ciudad = ciudad;
        this.bonoRepresentacion = bonoRepresentacion;
    }

    @Override
    public double calcularSalarioNeto() {
        return super.calcularSalarioNeto() + bonoRepresentacion;
    }

    @Override
    public String toString() {
        return String.format("[GERENTE %s] %s | Gastos Rep: $%,.0f COP",
                ciudad.toUpperCase(), super.toString(), bonoRepresentacion);
    }
}

public class EjemploNominaColombia {
    public static void main(String[] args) {
        System.out.println("=== SISTEMA DE NÓMINA EMPRESARIAL COLOMBIA ===");

        EmpleadoColombiano emp1 = new EmpleadoColombiano("1020304050", "Carlos Mendoza", 1500000.0); // Recibe auxilio
        EmpleadoTiempoCompleto emp2 = new EmpleadoTiempoCompleto("9876543210", "Ana Lucía Gómez", 3500000.0, 1750000.0);
        GerenteRegional gerBogota = new GerenteRegional("1122334455", "Beatriz Restrepo", 8000000.0, 4000000.0, "Bogotá", 1500000.0);

        System.out.println(emp1);
        System.out.println(emp2);
        System.out.println(gerBogota);
    }
}
```

Ejemplo 2: Sistema Financiero Bancario Colombiano (4x1000 y Sobregiro en COP)

Muestra la jerarquía de cuentas en un banco colombiano aplicando la norma del Gravamen a los Movimientos Financieros (**4x1000**).

JAVA

```java
// Archivo: EjemploBancoColombia.java

class CuentaBancariaCOP {
    protected String numeroCuenta;
    protected String titularCedula;
    protected double saldo; // En COP

    public CuentaBancariaCOP(String numeroCuenta, String titularCedula, double saldoInicial) {
        this.numeroCuenta = numeroCuenta;
        this.titularCedula = titularCedula;
        this.saldo = saldoInicial;
    }

    public void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
            System.out.println("=Depósito exitoso: $ " + String.format("%,.0f", monto) + " COP. Saldo: $" + String.format("%,.0f", saldo) + " COP");
        }
    }

    public boolean retirar(double monto) {
        if (monto > 0 && saldo >= monto) {
            saldo -= monto;
            System.out.println(" Retiro exitoso: $ " + String.format("%,.0f", monto) + " COP. Saldo: $" + String.format("%,.0f", saldo) + " COP");
            return true;
        }
        System.out.println("LRetiro fallido en cuenta " + numeroCuenta + ". Saldo insuficiente.");
        return false;
    }

    @Override
    public String toString() {
        return "Cuenta: " + numeroCuenta + " | Cédula: " + titularCedula + " | Saldo: $" + String.format("%,.0f", saldo) + " COP";
    }
}

class CuentaAhorrosExenta4x1000 extends CuentaBancariaCOP {
    private double tasaRentabilidadAnual;

    public CuentaAhorrosExenta4x1000(String numeroCuenta, String titularCedula, double saldoInicial, double tasaRentabilidadAnual) {
        super(numeroCuenta, titularCedula, saldoInicial);
        this.tasaRentabilidadAnual = tasaRentabilidadAnual;
    }

    public void abonarRendimientosMensuales() {
        double rendimiento = (saldo * tasaRentabilidadAnual) / 12.0;
        saldo += rendimiento;
        System.out.println("=Rendimiento abonado exento: $" + String.format("%,.0f", rendimiento) + " COP. Nuevo Saldo: $" + String.format("%,.0f", saldo) + " COP");
    }
}

class CuentaCorrienteCon4x1000 extends CuentaBancariaCOP {
    private double limiteSobregiro;
```

```java

    public CuentaCorrienteCon4x1000(String numeroCuenta, String titularCedula, double saldoInicial, double limiteSobregiro) {
        super(numeroCuenta, titularCedula, saldoInicial);
        this.limiteSobregiro = limiteSobregiro;
    }

    @Override
    public boolean retirar(double monto) {
        // En Colombia el impuesto 4x1000 aplica un cobro del 0.4% sobre cada retiro
        double impuesto4x1000 = monto * 0.004;
        double totalDebitar = monto + impuesto4x1000;

        if (monto > 0 && (saldo + limiteSobregiro) >= totalDebitar) {
            saldo -= totalDebitar;
            System.out.println(" Retiro: $" + String.format("%,.0f", monto) + " COP (+ 4x1000: $" + String.format("%,.0f", impuesto4x1000) + " COP). Saldo resultante: $" + String.format("%,.0f", saldo) + " COP");
            return true;
        }
        System.out.println("LRetiro rechazado. Excede cupo total con impuesto 4x1000.");
        return false;
    }
}

public class EjemploBancoColombia {
    public static void main(String[] args) {
        System.out.println("=== SISTEMA BANCARIO COLOMBIANO ===");

        CuentaAhorrosExenta4x1000 ahorros = new CuentaAhorrosExenta4x1000("031-992211-01", "1015234890", 2000000.0, 0.10);
        CuentaCorrienteCon4x1000 corriente = new CuentaCorrienteCon4x1000("105-883322-05", "80123456", 500000.0, 1000000.0);

        System.out.println(ahorros);
        ahorros.abonarRendimientosMensuales();

        System.out.println("\n" + corriente);
        corriente.retirar(600000.0); // Retiro con sobregiro + impuesto 4x1000
    }
}
```

Ejemplo 3: Transporte y Movilidad (RUNT, Particulares y Eléctricos)

Modelado del registro de vehículos en Colombia bajo el RUNT (Registro Único Nacional de Tránsito).

JAVA

```java
// Archivo: EjemploMovilidadRunt.java

class VehiculoRunt {
    protected String placa; // Formato colombiano: ABC-123
    protected String marca;
    protected int modeloAño;

    public VehiculoRunt(String placa, String marca, int modeloAño) {
        this.placa = placa;
        this.marca = marca;
        this.modeloAño = modeloAño;
    }

    public double calcularImpuestoVehicular(double avaluoComercial) {
        // Tarifa base en Colombia: 1.5%
```

```
        return avaluoComercial * 0.015;
    }

    @Override
    public String toString() {
        return "Placa: [" + placa + "] | Marca: " + marca + " | Modelo: " + modeloAño;
    }
}

class AutoElectricoPicoYPlaca extends VehiculoRunt {
    private double capacidadBateriaKWh;

    public AutoElectricoPicoYPlaca(String placa, String marca, int modeloAño, double capacidadBateriaKWh) {
        super(placa, marca, modeloAño);
        this.capacidadBateriaKWh = capacidadBateriaKWh;
    }

    @Override
    public double calcularImpuestoVehicular(double avaluoComercial) {
        // En Colombia los vehículos eléctricos tienen beneficio tributario (máximo 1.0%)
        return avaluoComercial * 0.010;
    }

    public boolean tieneExencionPicoYPlaca() {
        return true; // En ciudades colombianas como Medellín o Bogotá los eléctricos están exentos de Pico y Placa
    }

    @Override
    public String toString() {
        return super.toString() + " |¡Eléctrico (" + capacidadBateriaKWh + " kWh) | Exento Pico y Placa: " + (tieneExencionPicoYPlaca() ? "SÍ" : "NO");
    }
}

public class EjemploMovilidadRunt {
    public static void main(String[] args) {
        VehiculoRunt convencional = new VehiculoRunt("FGH-456", "Chevrolet Onix", 2022);
        AutoElectricoPicoYPlaca electrico = new AutoElectricoPicoYPlaca("ZXC-789E", "BYD Yuan Plus", 2024, 60.5);

        double avaluo = 70000000.0; // 70 millones COP

        System.out.println(convencional + " | Impuesto Anual: $" + String.format("%,.0f", convencional.calcularImpuestoVehicular(avaluo)) + " COP");
        System.out.println(electrico + " | Impuesto Anual Beneficiado: $" + String.format("%,.0f", electrico.calcularImpuestoVehicular(avaluo)) + " COP");
    }
}
```

Ejemplo 4: Sistema de Videojuego RPG (Contexto Fantástico General)

JAVA

```
// Archivo: EjemploVideojuegoRPG.java

class PersonajeRPG {
    protected String nombre;
    protected int nivel;
    protected int salud;
```

```
    public PersonajeRPG(String nombre, int nivel, int salud) {
        this.nombre = nombre;
        this.nivel = nivel;
        this.salud = salud;
    }

    public void atacar(PersonajeRPG enemigo) {
        System.out.println("" " + nombre + " asesta un golpe estándar a " + enemigo.nombre + ".");
        enemigo.recibirDaño(12 * nivel);
    }

    public void recibirDaño(int cantidad) {
        this.salud = Math.max(0, salud - cantidad);
        System.out.println("=¥ + nombre + " recibe " + cantidad + " de daño. (Salud restante: " + salud -
+ ")");
    }
}

class Paladin extends PersonajeRPG {
    private int escudoSagrado;

    public Paladin(String nombre, int nivel, int salud, int escudoSagrado) {
        super(nombre, nivel, salud);
        this.escudoSagrado = escudoSagrado;
    }

    @Override
    public void recibirDaño(int cantidad) {
        int dañoReal = Math.max(0, cantidad - escudoSagrado);
        System.out.println("={PALADÍN] El escudo de " + nombre + " absorbe " + (cantidad - dañoReal) +-
 " de daño.");
        super.recibirDaño(dañoReal);
    }
}

public class EjemploVideojuegoRPG {
    public static void main(String[] args) {
        PersonajeRPG orco = new PersonajeRPG("Orco Berserker", 2, 80);
        Paladin paladin = new Paladin("Sir Arthur", 3, 120, 15);

        orco.atacar(paladin);
    }
}
```

Ejemplo 5: Sistema Universitario Colombiano (Escala de Calificación 0.0 - 5.0)

En las universidades de Colombia, el sistema de notas evalúa en un rango de **0.0 a 5.0**, donde el promedio mínimo aprobatorio es generalmente **3.0** y el promedio para beca o distinción de honor es superior a **4.5**.

JAVA

```
// Archivo: EjemploUniversidadColombia.java
import java.util.ArrayList;
import java.util.List;

class IntegranteUniversidad {
    protected String cedula;
    protected String nombreCompleto;

    public IntegranteUniversidad(String cedula, String nombreCompleto) {
        this.cedula = cedula;
        this.nombreCompleto = nombreCompleto;
```

```
    }

    public void imprimirCredencial() {
        System.out.print("CC: " + cedula + " | Nombre: " + nombreCompleto);
    }
}

class EstudianteColombiano extends IntegranteUniversidad {
    private String codigoBanner;
    private double promedioAcumulado; // Rango 0.0 a 5.0

    public EstudianteColombiano(String cedula, String nombreCompleto, String codigoBanner, double promedioAcumulado) {
        super(cedula, nombreCompleto);
        this.codigoBanner = codigoBanner;
        this.promedioAcumulado = promedioAcumulado;
    }

    @Override
    public void imprimirCredencial() {
        super.imprimirCredencial();
        System.out.println(" | Código: " + codigoBanner + " | Promedio: " + promedioAcumulado + " / 5.0");
    }

    public double getPromedioAcumulado() { return promedioAcumulado; }
}

class ProfesorCatedra extends IntegranteUniversidad {
    private String facultad;
    private int horasAsignadas;

    public ProfesorCatedra(String cedula, String nombreCompleto, String facultad, int horasAsignadas) {
        super(cedula, nombreCompleto);
        this.facultad = facultad;
        this.horasAsignadas = horasAsignadas;
    }

    @Override
    public void imprimirCredencial() {
        super.imprimirCredencial();
        System.out.println(" | Facultad: " + facultad + " | Horas Semanales: " + horasAsignadas);
    }
}

public class EjemploUniversidadColombia {
    public static void main(String[] args) {
        List<IntegranteUniversidad> comunidad = new ArrayList<>();
        comunidad.add(new EstudianteColombiano("1018223344", "Valentina Ospina", "EST-20241", 4.7));-
 // Candidata a Cum Laude
        comunidad.add(new EstudianteColombiano("1032445566", "Mateo Bermúdez", "EST-20242", 3.2));
        comunidad.add(new ProfesorCatedra("79887766", "Dr. Hernán Montenegro", "Ingeniería de Sistemas"-
, 16));

        System.out.println("=== COMUNIDAD UNIVERSITARIA COLOMBIA ===");
        for (IntegranteUniversidad i : comunidad) {
            i.imprimirCredencial();
        }

        System.out.println("\n=== RECONOCIMIENTO A BECA DE EXCELENCIA (Promedio >= 4.5) ===");
        for (IntegranteUniversidad i : comunidad) {
            if (i instanceof EstudianteColombiano est && est.getPromedioAcumulado() >= 4.5) {
                System.out.println("¡Felicitaciones a " + est.nombreCompleto + " por su promedio sobresaliente de " + est.getPromedioAcumulado());
            }
        }
    }
```

```
74 | }
```

8. Ejercicios Prácticos Resueltos Paso a Paso

Ejercicio 1: Facturación Comercial e IVA del 19% (DIAN)

Enunciado: Crea la superclase `ProductoComercial` (`codigoBarra`, `nombre`, `precioNeto`). Crea las subclases:

1. `ProductoExentoIVA`: Productos de la canasta familiar (ej. Arroz, Huevos, Leche). No aplican IVA (0%).
2. `ProductoGravadoIVA`: Aplica la tarifa general del **19% de IVA** estipulada por la DIAN en Colombia.

Solución Completa:

JAVA

```java
// Archivo: SolucionEjercicio1DIAN.java

class ProductoComercial {
    protected String codigoBarra;
    protected String nombre;
    protected double precioNeto; // Antes de impuestos (COP)

    public ProductoComercial(String codigoBarra, String nombre, double precioNeto) {
        this.codigoBarra = codigoBarra;
        this.nombre = nombre;
        this.precioNeto = precioNeto;
    }

    public double calcularPrecioVentaFinal() {
        return precioNeto;
    }

    @Override
    public String toString() {
        return String.format("[%s] %-22s | Neto: $%,.0f COP | Venta Final: $%,.0f COP",
                codigoBarra, nombre, precioNeto, calcularPrecioVentaFinal());
    }
}

class ProductoExentoIVA extends ProductoComercial {
    public ProductoExentoIVA(String codigoBarra, String nombre, double precioNeto) {
        super(codigoBarra, nombre, precioNeto);
    }

    @Override
    public String toString() {
        return super.toString() + " (Canasta Familiar - Exento IVA 0%)";
    }
}

class ProductoGravadoIVA extends ProductoComercial {
    public static final double TARIFA_IVA = 0.19; // 19% en Colombia

    public ProductoGravadoIVA(String codigoBarra, String nombre, double precioNeto) {
        super(codigoBarra, nombre, precioNeto);
    }

```

```
    @Override
    public double calcularPrecioVentaFinal() {
        return precioNeto * (1 + TARIFA_IVA);
    }

    @Override
    public String toString() {
        return super.toString() + " (+ IVA 19% DIAN: $" + String.format("%,.0f", precioNeto * TARIFA_IVA-
) + " COP)";
    }
}

public class SolucionEjercicio1DIAN {
    public static void main(String[] args) {
        ProductoComercial leche = new ProductoExentoIVA("770123456701", "Leche Alquería 1L", 4200.0);
        ProductoComercial tv = new ProductoGravadoIVA("770123456999", "Televisor Samsung 50\"", 1800000.0);

        System.out.println(leche);
        System.out.println(tv);
    }
}
```

Ejercicio 2: Propiedad Horizontal y Estratificación en Colombia

Enunciado: Crea una superclase `InmuebleColombia` con atributos `matriculaInmobiliaria`, `estrato` (1 a 6) y `areaM2`. Crea dos subclases:

- `ApartamentoEnConjunto`: Añade el costo de `administracion`. Si el estrato es 5 ó 6, suma una sobretasa del 10% por contribución de solidaridad.
- `LocalComercial`: Añade retención de fuente inmobiliaria fija del 2.5%.

Solución Completa:

JAVA

```
// Archivo: SolucionEjercicio2PropiedadHorizontal.java

class InmuebleColombia {
    protected String matriculaInmobiliaria;
    protected int estrato;
    protected double areaM2;

    public InmuebleColombia(String matriculaInmobiliaria, int estrato, double areaM2) {
        if (estrato < 1 || estrato > 6) {
            throw new IllegalArgumentException("El estrato en Colombia debe estar en el rango [1 - 6]");
        }
        this.matriculaInmobiliaria = matriculaInmobiliaria;
        this.estrato = estrato;
        this.areaM2 = areaM2;
    }

    public double calcularCostoMensual(double valorMetroBase) {
        return areaM2 * valorMetroBase;
    }
}

class ApartamentoEnConjunto extends InmuebleColombia {
    private double valorAdministracion;

```

```
    public ApartamentoEnConjunto(String matriculaInmobiliaria, int estrato, double areaM2, double valorAdministracion) {
        super(matriculaInmobiliaria, estrato, areaM2);
        this.valorAdministracion = valorAdministracion;
    }

    @Override
    public double calcularCostoMensual(double valorMetroBase) {
        double costoBase = super.calcularCostoMensual(valorMetroBase) + valorAdministracion;
        if (estrato >= 5) {
            costoBase += valorAdministracion * 0.10; // Sobretasa de solidaridad estrato alto
        }
        return costoBase;
    }

    @Override
    public String toString() {
        return "<Apto Matrícula: " + matriculaInmobiliaria + " | Estrato: " + estrato + " | Áre: " + areaM2 + "m²";
    }
}

public class SolucionEjercicio2PropiedadHorizontal {
    public static void main(String[] args) {
        ApartamentoEnConjunto aptoEstrato3 = new ApartamentoEnConjunto("50C-12345", 3, 75.0, 180000.0);
        ApartamentoEnConjunto aptoEstrato6 = new ApartamentoEnConjunto("50C-98765", 6, 120.0, 450000.0);

        double tarifaM2 = 10000.0;

        System.out.println(aptoEstrato3 + " | Pago Total Mes: $" + String.format("%,.0f", aptoEstrato3.calcularCostoMensual(tarifaM2)) + " COP");
        System.out.println(aptoEstrato6 + " | Pago Total Mes (con Sobretasa): $" + String.format("%,.0f", aptoEstrato6.calcularCostoMensual(tarifaM2)) + " COP");
    }
}
```

Ejercicio 3: Sistema de Afiliación a Seguridad Social (EPS y ARL)

Enunciado: Modelar las cotizaciones a EPS y ARL en el sistema de seguridad social colombiano para independientes y dependientes.

Solución Completa:

JAVA

```
// Archivo: SolucionEjercicio3SeguridadSocial.java

class CotizanteSalud {
    protected String cedula;
    protected double ingresoBaseCotizacion; // IBC

    public CotizanteSalud(String cedula, double ingresoBaseCotizacion) {
        this.cedula = cedula;
        this.ingresoBaseCotizacion = ingresoBaseCotizacion;
    }

    public double calcularAporteEPS() {
        return ingresoBaseCotizacion * 0.125; // 12.5% tarifa total EPS
    }
}

class TrabajadorIndependiente extends CotizanteSalud {
```

```
    private int nivelRiesgoARL; // Riesgo 1 a 5

    public TrabajadorIndependiente(String cedula, double honorariosMensuales, int nivelRiesgoARL) {
        // En Colombia el IBC de independientes es mínimo el 40% de los honorarios
        super(cedula, honorariosMensuales * 0.40);
        this.nivelRiesgoARL = nivelRiesgoARL;
    }

    public double calcularAporteARL() {
        double porcentajeARL = switch (nivelRiesgoARL) {
            case 1 -> 0.00522;
            case 2 -> 0.01044;
            case 3 -> 0.02436;
            case 4 -> 0.04350;
            case 5 -> 0.06960;
            default -> 0.00522;
        };
        return ingresoBaseCotizacion * porcentajeARL;
    }

    @Override
    public String toString() {
        return "CC: " + cedula + " | IBC (40%): $" + String.format("%,.0f", ingresoBaseCotizacion) +
               " COP | Aporte EPS: $" + String.format("%,.0f", calcularAporteEPS()) +
               " COP | Aporte ARL (Riesgo " + nivelRiesgoARL + "): $" + String.format("%,.0f", calcu-
larAporteARL()) + " COP";
    }
}

public class SolucionEjercicio3SeguridadSocial {
    public static void main(String[] args) {
        TrabajadorIndependiente ind = new TrabajadorIndependiente("1098765432", 5000000.0, 3);
        System.out.println("<ê<ô ind);
    }
}
```

9. Buenas Prácticas y Errores Comunes

Buenas Prácticas

1. **Aplica `@Override` consistentemente:** Evita errores sintácticos por variaciones en nombres de métodos.
2. **Utiliza constantes bien identificadas:** Define `SMLMV`, `TARIFA_IVA`, `TARIFA_4X1000` con `public static final`.
3. **Mantén atributos `private`:** Facilita la evolución de las superclases sin romper dependencias externas.

Resumen Consolidador

CONCEPTO	APLICACIÓN EN JAVA
`extends`	Enlaza la subclase a una única superclase.
`super(...)`	Constructor de la superclase (línea inicial indispensable).
`@Override`	Redefinición de comportamientos heredados.
`protected`	Visibilidad otorgada a subclases dentro y fuera del paquete.
`instanceof`	Comprobación y casteo seguro de tipos.

CAPÍTULO 3 • LECCIÓN 3.6

Clase 7 - Ejercicio Práctico

Actividad paso a paso: Sistema de Gestión de Animales en un Refugio

Esta actividad es completamente práctica y está diseñada para principiantes. Usaremos **Java** (el lenguaje más común para aprender estos conceptos de Programación Orientada a Objetos - POO).

Tema del ejemplo: Un **Refugio de Animales**. Tendremos una clase base `Animal`, dos clases hijas (`Perro` y `Gato`) y una clase `Refugio` que manejará una lista de animales utilizando `ArrayList`.

Conceptos que cubriremos

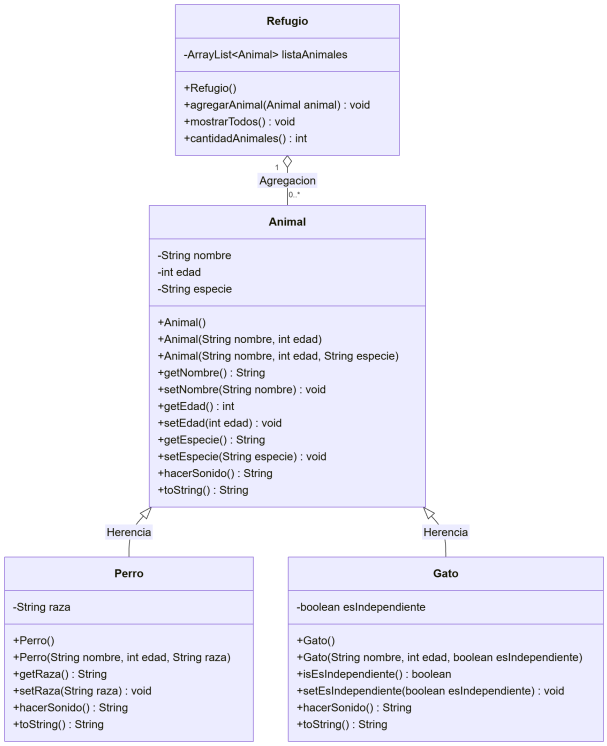
En lugar de ver clases completas de un solo golpe, iremos construyendo el código **fragmento por fragmento**, explicando detalladamente el "por qué" y el "cómo" de cada uno de los siguientes conceptos:

- **Clase** y **Atributos**
- **Encapsulamiento** (Modificadores de acceso `private` y `public`)
- **Constructores** (Por defecto y Sobrecarga)
- **Getters y Setters**
- **Lógica de negocio** y **Manejo de Excepciones** (`IllegalArgumentException`)
- Método **`toString`**
- **Herencia** (`extends` y uso de `super`)
- **Sobreescritura de métodos** (`@Override`)
- **`ArrayList`** y **Polimorfismo**
- **Menú Interactivo en Consola** con **`Scanner`** y captura de errores con **`try-catch`**
- **Diagrama de Clases UML** (Visualización con Mermaid)

Diagrama de Clases UML del Sistema

Antes de comenzar a escribir código, observemos la estructura y las relaciones entre nuestras clases en el siguiente diagrama UML:

DIAGRAMA DE ARQUITECTURA / FLUJO
Mermaid Engine



NOTA CLAVE

Explicación del Diagrama UML: - **`Animal <|-- Perro` / `Gato` (Herencia):** La flecha hueca apunta hacia la clase padre (`Animal`), indicando que `Perro` y `Gato` heredan de ella. - **`Refugio o-- Animal` (Agregación):** El rombo transparente indica que `Refugio` contiene una lista dinámica (`ArrayList`) de objetos de tipo `Animal`. - **Simbología (`-` y `+`):** El signo menos `-` indica que un atributo o método es privado (`private`), mientras que el signo más `+` representa visibilidad pública (`public`).

Paso 0: Preparar el proyecto

1. Abre **Visual Studio Code** y asegúrate de tener instalada la extensión **Extension Pack for Java** (de Microsoft).
2. Abre la paleta de comandos presionando `Ctrl + Shift + P` (o `Cmd + Shift + P` en macOS).
3. Escribe y selecciona la opción **`Java: Create Java Project...`**.
4. En las opciones de tipo de proyecto, selecciona **`Maven`**.
5. Selecciona la opción **`No Archetype`** (Sin arquetipo).
6. Cuando VS Code solicite el **Group Id / paquete por defecto**, deja la opción por defecto `com.example` (presiona `Enter`).
7. Ingresa el **Artifact Id** (nombre del proyecto): `RefugioAnimales` y presiona `Enter`.
8. Selecciona la carpeta de destino donde se guardará la carpeta del proyecto.
9. Abre el proyecto generado en VS Code. Dentro de la ruta `src/main/java/com/example`, crea la carpeta o paquete `refugio` (quedando la ruta completa en `src/main/java/com/example/refugio`) donde irán todos los archivos `.java`.

Estructura final esperada con Maven:

```text
RefugioAnimales/
  pom.xml
  src/
       main/
            java/
                 com/
                      example/
                           refugio/
                                Animal.java
                                Perro.java
                                Gato.java
                                Refugio.java
                                Main.java
```

Paso 1: Construcción de la clase base `Animal` (Paso a Paso)

Crea el archivo `Animal.java` e inicia la construcción agregando cada fragmento a medida que se explica.

1.1 Encabezado del Paquete y Declaración de la Clase

```java
package com.example.refugio;

public class Animal {
    // Aquí irán nuestros atributos y métodos
}
```

> **NOTA CLAVE**
> **Explicación:** - `package com.example.refugio;` le indica a Java que este archivo pertenece al paquete `refugio` dentro del paquete base `com.example`. - `public class Animal` define una **Clase**, que funciona como un molde o plantilla a partir de la cual crearemos objetos de tipo animal.

1.2 Atributos y Encapsulamiento

Agrega dentro de la clase `Animal`:

```java
    // ========== ATRIBUTOS ==========
    private String nombre;
    private int edad;
    private String especie;
```

NOTA CLAVE

Explicación: - Los **Atributos** representan las características o datos que tiene todo animal (`nombre`, `edad`, `especie`). - La palabra reservada `private` aplica el concepto de **Encapsulamiento**. Esto significa que los datos están protegidos y NO pueden ser modificados ni leídos directamente desde fuera de la clase `Animal`.

1.3 Constructor por Defecto

Agrega a continuación:

JAVA

```java
    // ========== CONSTRUCTOR POR DEFECTO ==========
    public Animal() {
        this.nombre = "Sin nombre";
        this.edad = 0;
        this.especie = "Desconocida";
    }
```

NOTA CLAVE

Explicación: - Un **Constructor** es un método especial que se ejecuta automáticamente al crear un objeto (`new Animal()`). No tiene tipo de retorno y se llama exactamente igual que la clase. - El **constructor por defecto** (sin parámetros) inicializa los atributos con valores seguros predeterminados. - `this` hace referencia a la instancia actual del objeto que se está creando.

1.4 Sobrecarga de Constructores

Agrega los siguientes constructores:

JAVA

```java
    // ========== SOBRECARGA DE CONSTRUCTORES ==========
    public Animal(String nombre, int edad) {
        this.setNombre(nombre);  // Usamos el setter para aplicar la lógica de negocio
        this.setEdad(edad);      // Usamos el setter para validar
        this.especie = "Desconocida";
    }

    public Animal(String nombre, int edad, String especie) {
        this.setNombre(nombre);
        this.setEdad(edad);
        this.setEspecie(especie);
    }
```

NOTA CLAVE

Explicación: - La **Sobrecarga de Constructores** ocurre cuando tenemos múltiples constructores con el mismo nombre (`Animal`), pero con diferente cantidad o tipo de parámetros. - Permite crear un animal solo con nombre y edad, o especificando también la especie. - En lugar de asignar directamente (`this.nombre = nombre;`), llamamos a los métodos *setter* (`setNombre`, `setEdad`) para reusar las reglas de validación que definiremos más adelante.

1.5 Métodos Getters (Lectura de datos)

Agrega los métodos para leer las propiedades:

JAVA

```java
    // ========== GETTERS ==========
    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public String getEspecie() {
        return especie;
    }
```

NOTA CLAVE

Explicación: - Los **Getters** permiten que otras clases lean el valor de los atributos `private` de forma controlada. - Retornan el tipo de dato del atributo (`String`, `int`).

1.6 Métodos Setters con Lógica de Negocio y Manejo de Excepciones

Agrega los métodos para asignar valores validando las reglas de negocio:

JAVA

```java
    // ========== SETTERS CON LÓGICA DE NEGOCIO Y EXCEPCIONES ==========
    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del animal no puede estar vacío.");
        }
        this.nombre = nombre.trim(); // Limpia espacios en blanco al inicio y final
    }

    public void setEdad(int edad) {
        if (edad < 0 || edad > 30) {
            throw new IllegalArgumentException("La edad debe ser un número entre 0 y 30 años.");
        }
        this.edad = edad;
    }

    public void setEspecie(String especie) {
        if (especie == null || especie.trim().isEmpty()) {
            throw new IllegalArgumentException("La especie no puede estar vacía.");
        }
        this.especie = especie.trim();
    }
```

NOTA CLAVE

Explicación sobre Lógica de Negocio y Excepciones: - **Lógica de Negocio**: Protegemos los datos impidiendo estados inválidos (nombres vacíos o edades fuera de rango). - **`throw new IllegalArgumentException(...)`**: En lugar de silenciar los errores o corregir datos automáticamente con `System.out.println`, lanzamos una **Excepción**. - Una excepción detiene la ejecución insegura e informa al código llamador (como un formulario o menú de consola) de que se introdujo un valor incorrecto, permitiendo capturar el error con `try-catch`.

1.7 Método `toString` y Método Base para Polimorfismo

Finaliza la clase `Animal` agregando:

JAVA

```java
    // ========== MÉTODO toString ==========
    @Override
    public String toString() {
        return "Animal [nombre=" + nombre + ", edad=" + edad + ", especie=" + especie + "]";
    }

    // ========== MÉTODO BASE PARA HERENCIA ==========
    public String hacerSonido() {
        return "El animal hace un sonido genérico";
    }
```

NOTA CLAVE

Explicación: - **`toString()`**: Método heredado de la clase base `Object` de Java. Lo sobreescribimos para representar en texto legible el estado del objeto al imprimirlo. - **`hacerSonido()`**: Define un comportamiento genérico que luego modificaremos en las clases hijas.

Paso 2: Crear la clase hija `Perro` (Herencia y Sobreescritura)

Crea el archivo `Perro.java` y constrúyelo por partes.

2.1 Herencia (`extends`) y Atributos Propios

JAVA

```java
package com.example.refugio;

public class Perro extends Animal {  // HERENCIA: Perro es un tipo de Animal

    // Atributo específico de Perro
    private String raza;
```

NOTA CLAVE
Explicación: - **Herencia**: La palabra clave `extends Animal` hace que `Perro` sea una **clase hija** (subclase) que hereda todos los atributos y métodos públicos de `Animal` (clase padre). - Además, agregamos un atributo único de los perros: `raza`.

2.2 Constructores y Uso de `super`

Agrega los constructores de `Perro`:

JAVA

```java
    // Constructor por defecto
    public Perro() {
        super();              // Llama al constructor por defecto de la clase padre (Animal)
        this.raza = "Mestizo";
    }

    // Constructor con parámetros
    public Perro(String nombre, int edad, String raza) {
        super(nombre, edad, "Perro");  // Envía los datos comunes al constructor de Animal
        this.setRaza(raza);            // Asigna la raza usando el setter propio
    }
```

NOTA CLAVE
Explicación: - **`super()`**: Es la forma en que la clase hija invoca al constructor de la clase padre `Animal`. - Al ejecutar `super(nombre, edad, "Perro")`, la clase padre se encarga de validar e inicializar `nombre`, `edad` y fijar la `especie` automáticamente como `"Perro"`.

2.3 Getter y Setter propio con Validación

Agrega el control para el atributo `raza`:

JAVA

```java
    public String getRaza() {
        return raza;
    }

    public void setRaza(String raza) {
        if (raza == null || raza.trim().isEmpty()) {
            this.raza = "Mestizo";
        } else {
            this.raza = raza.trim();
        }
    }
```

NOTA CLAVE
Explicación: - Administra el atributo `raza` garantizando que si se ingresa un valor nulo o en blanco, se asigne `"Mestizo"`.

2.4 Sobreescritura de Métodos (`@Override`)

Agrega la personalización de comportamientos:

JAVA

```java
    // SOBREESCRITURA del comportamiento sonoro
    @Override
    public String hacerSonido() {
        return "¡Guau guau!";
    }

    // SOBREESCRITURA de la representación en texto
    @Override
    public String toString() {
        return "Perro [nombre=" + getNombre() +
               ", edad=" + getEdad() +
               ", raza=" + raza + "]";
    }
}
```

NOTA CLAVE

Explicación: - **Sobreescritura de Métodos (`@Override`)**: Reemplaza la implementación heredada de la clase padre por una específica para `Perro`. Un perro no hace un "sonido genérico", hace `"¡Guau guau!"`. - Nota cómo usamos `getNombre()` y `getEdad()` porque dichos atributos son `private` en la clase padre `Animal`.

Paso 3: Crear la clase hija `Gato`

Crea el archivo `Gato.java` siguiendo la misma estructura por partes.

3.1 Atributos, Constructores y Métodos de `Gato`

JAVA

```java
package com.example.refugio;

public class Gato extends Animal {

    // Atributo específico de Gato
    private boolean esIndependiente;

    // Constructor por defecto
    public Gato() {
        super();
        this.esIndependiente = true;
    }

    // Constructor con parámetros
    public Gato(String nombre, int edad, boolean esIndependiente) {
        super(nombre, edad, "Gato");
        this.esIndependiente = esIndependiente;
    }

    // Getter y Setter (para booleanos se suele usar 'is' en lugar de 'get')
    public boolean isEsIndependiente() {
        return esIndependiente;
```

```
    }

    public void setEsIndependiente(boolean esIndependiente) {
        this.esIndependiente = esIndependiente;
    }

    // Sobreescritura de hacerSonido
    @Override
    public String hacerSonido() {
        return "¡Miau miau!";
    }

    // Sobreescritura de toString
    @Override
    public String toString() {
        return "Gato [nombre=" + getNombre() +
               ", edad=" + getEdad() +
               ", independiente=" + esIndependiente + "]";
    }
}
```

NOTA CLAVE

Explicación: - `Gato` también hereda de `Animal`. - Posee una propiedad booleana única (`esIndependiente`). - Personaliza su propio sonido (`"¡Miau miau!"`) y su propio formato `toString()`.

Paso 4: Crear la clase `Refugio` (Manejo de `ArrayList` y Polimorfismo)

Crea el archivo `Refugio.java` e integra el uso de colecciones dinámicas.

4.1 Importación e Inicialización del `ArrayList`

JAVA

```java
package com.example.refugio;

import java.util.ArrayList; // Importamos la clase ArrayList

public class Refugio {

    // Atributo: lista de tipo Animal
    private ArrayList<Animal> listaAnimales;

    // Constructor
    public Refugio() {
        listaAnimales = new ArrayList<>();  // Inicializamos la lista vacía en memoria
    }
```

NOTA CLAVE

Explicación: - **`ArrayList<Animal>`**: Es una estructura de datos dinámica (su tamaño crece o decrece según se agreguen o eliminen elementos). - Al definirla de tipo `<Animal>`, aprovechamos el **Polimorfismo**: la lista puede almacenar objetos `Animal`, `Perro` o `Gato`.

4.2 Métodos para la Lógica de Gestión en el Refugio

Agrega a la clase `Refugio`:

JAVA

```java
    // Método para agregar un animal
    public void agregarAnimal(Animal animal) {
        if (animal != null) {
            listaAnimales.add(animal);
            System.out.println("Animal agregado correctamente: " + animal.getNombre());
        } else {
            System.out.println("No se puede agregar un animal nulo.");
        }
    }

    // Método para mostrar todos los animales registrados
    public void mostrarTodos() {
        if (listaAnimales.isEmpty()) {
            System.out.println("El refugio está vacío.");
            return;
        }

        System.out.println("\n===== ANIMALES EN EL REFUGIO =====");
        for (Animal animal : listaAnimales) {
            System.out.println(animal);           // Invoca automáticamente el toString() específi-
co de cada objeto
            System.out.println("Sonido: " + animal.hacerSonido()); // Llama al hacerSonido() correspondiente
            System.out.println("---------------------------------");
        }
    }

    // Método para obtener el conteo de animales
    public int cantidadAnimales() {
        return listaAnimales.size();
    }
}
```

NOTA CLAVE

Explicación: - `listaAnimales.add(animal)`: Agrega un elemento a la lista. - `listaAnimales.isEmpty()`: Verifica si la lista no contiene elementos. - `for (Animal animal : listaAnimales)`: Bucle *for-each* para recorrer la colección. - **Polimorfismo en Acción**: Cuando se ejecuta `animal.hacerSonido()`, Java identifica dinámicamente si la instancia real es un `Perro` o un `Gato` y ejecuta el sonido correspondiente (`"¡Guau guau!"` o `"¡Miau miau!"`).

Paso 5: Crear la clase `Main` (Menú Interactivo en Consola con `Scanner` y `try-catch`)

Crea el archivo `Main.java`. Construiremos un menú interactivo en consola dividido en fragmentos explicativos.

5.1 Importación de `Scanner` e Inicialización

JAVA

```java
package com.example.refugio;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
        Refugio miRefugio = new Refugio();
        int opcion = 0;
```

NOTA CLAVE

Explicación: - `import java.util.Scanner;`: Permite leer la entrada por teclado que realiza el usuario en la consola. - `new Scanner(System.in)`: Inicializa la lectura del canal de entrada estándar de la consola.

5.2 Estructura del Bucle del Menú (`do-while` y `switch`)

JAVA

```java
        do {
            System.out.println("\n===== MENÚ DEL REFUGIO DE ANIMALES =====");
            System.out.println("1. Registrar Perro");
            System.out.println("2. Registrar Gato");
            System.out.println("3. Mostrar todos los animales");
            System.out.println("4. Salir");
            System.out.print("Seleccione una opción: ");

            try {
                opcion = Integer.parseInt(scanner.nextLine());

                switch (opcion) {
                    case 1:
                        // Lógica para registrar un Perro
                        break;
                    case 2:
                        // Lógica para registrar un Gato
                        break;
                    case 3:
                        miRefugio.mostrarTodos();
                        break;
                    case 4:
                        System.out.println("¡Gracias por usar el sistema del Refugio!");
                        break;
                    default:
                        System.out.println("Opción inválida. Intente de nuevo.");
                }
            } catch (NumberFormatException e) {
                System.out.println("Error: Debe ingresar un número entero válido para la opción.");
            }

        } while (opcion != 4);

        scanner.close();
    }
}
```

NOTA CLAVE

Explicación del Bucle y `NumberFormatException`: - `do-while`: Mantiene el menú en pantalla repitiéndose hasta que el usuario decida ingresar la opción 4. - `scanner.nextLine()` + `Integer.parseInt()`: Lee la línea ingresada como texto y la convierte a número entero. Es más seguro que `nextInt()` porque evita problemas con el salto de línea residual. - `catch (NumberFormatException e)`: Si el usuario escribe una letra en lugar de un número en el menú, el programa no colapsa; captura el error y le informa suavemente.

5.3 Lógica de Registro con Captura de Excepciones de Negocio (`try-catch`)

Agregamos la captura de datos en los casos 1 y 2 del `switch`:

JAVA

```
    // Caso 1: Registrar Perro
    System.out.print("Ingrese nombre del perro: ");
    String nombrePerro = scanner.nextLine();

    System.out.print("Ingrese edad del perro: ");
    int edadPerro = Integer.parseInt(scanner.nextLine());

    System.out.print("Ingrese raza del perro: ");
    String raza = scanner.nextLine();

    // Intentamos crear el perro. Si los setters detectan datos inválidos, lanzarán IllegalArgumentException
    Perro perro = new Perro(nombrePerro, edadPerro, raza);
    miRefugio.agregarAnimal(perro);

    // Captura de la excepción de negocio lanzada por los setters
    catch (IllegalArgumentException e) {
        System.out.println(" Error de validación: " + e.getMessage());
    }
```

NOTA CLAVE

Explicación del Manejo de Excepciones con `try-catch`: - Cuando ejecutamos `new Perro(nombrePerro, edadPerro, raza)`, los constructores de `Perro` y `Animal` invocan los *setters*. - Si el usuario escribió un nombre vacío o una edad negativa (`-5`), el *setter* lanzará un `throw new IllegalArgumentException(...)`. - El bloque `catch (IllegalArgumentException e)` atapa esa excepción y muestra `e.getMessage()`, impidiendo que el programa se cierre abruptamente.

5.4 Código Completo Ensamble de `Main.java`

Aquí tienes el archivo `Main.java` completo para que puedas verificar la integración final:

JAVA

```
package com.example.refugio;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
        Refugio miRefugio = new Refugio();
        int opcion = 0;

        do {
            System.out.println("\n===== MENÚ DEL REFUGIO DE ANIMALES =====");
            System.out.println("1. Registrar Perro");
            System.out.println("2. Registrar Gato");
            System.out.println("3. Mostrar todos los animales");
            System.out.println("4. Salir");
            System.out.print("Seleccione una opción: ");

            try {
                opcion = Integer.parseInt(scanner.nextLine());

                switch (opcion) {
                    case 1:
                        try {
                            System.out.print("Ingrese nombre del perro: ");
                            String nombre = scanner.nextLine();
                            System.out.print("Ingrese edad del perro: ");
                            int edad = Integer.parseInt(scanner.nextLine());
                            System.out.print("Ingrese raza del perro: ");
                            String raza = scanner.nextLine();

                            Perro perro = new Perro(nombre, edad, raza);
                            miRefugio.agregarAnimal(perro);
                        } catch (IllegalArgumentException e) {
                            System.out.println(" Error de validación: " + e.getMessage());
                        }
                        break;

                    case 2:
                        try {
                            System.out.print("Ingrese nombre del gato: ");
                            String nombre = scanner.nextLine();
                            System.out.print("Ingrese edad del gato: ");
                            int edad = Integer.parseInt(scanner.nextLine());
                            System.out.print("¿Es independiente? (true/false): ");
                            boolean independiente = Boolean.parseBoolean(scanner.nextLine());

                            Gato gato = new Gato(nombre, edad, independiente);
                            miRefugio.agregarAnimal(gato);
                        } catch (IllegalArgumentException e) {
                            System.out.println(" Error de validación: " + e.getMessage());
                        }
                        break;

                    case 3:
                        miRefugio.mostrarTodos();
                        break;

                    case 4:
                        System.out.println("¡Gracias por utilizar el sistema del Refugio!");
                        break;

                    default:
                        System.out.println("Opción no válida. Intente de nuevo.");
                }

            } catch (NumberFormatException e) {
                System.out.println(" Error: Debe ingresar un número válido.");
            }

        } while (opcion != 4);
```

```
72 |
73 |         scanner.close();
74 |     }
75 | }
```

Cómo ejecutar

1. Guarda todos los archivos `.java` dentro de la carpeta `src/main/java/com/example/refugio`.
2. Ejecuta la clase `Main` (Haciendo clic derecho sobre `Main.java` '*Run 'Main.main()'* o desde la terminal de VS Code).
3. Interactúa con el menú en la consola de VS Code registrando perros y gatos, y prueba ingresar intencionalmente edades negativas o nombres vacíos para observar cómo `try-catch` ataja los errores sin detener la aplicación.

Desafío Autónomo para el Estudiante

¡Es tu turno de poner a prueba lo aprendido! A continuación, se presenta el enunciado para extender el sistema agregando un nuevo tipo de animal al refugio. **Debes resolverlo por tu cuenta sin ver código de solución.**

Enunciado del Desafío: Incorporar la clase `Conejo`

El refugio ha comenzado a recibir conejos y requiere registrar sus datos en el sistema.

1. **Crear la clase `Conejo.java`:**
2. Ubícala dentro del paquete `com.example.refugio`.
3. Aplica **Herencia** para que sea una clase hija de `Animal`.
4. Agrega un atributo privado propio: `longitudOrejas` (de tipo `double`, en centímetros).
5. Implementa **Constructores** (por defecto y sobrecargado con parámetros).
6. Crea el **Getter y Setter** para `longitudOrejas`.
7. **Regla de negocio en el Setter:** Valida que la longitud de las orejas esté entre `1.0` cm y `30.0` cm. Si el valor está fuera de rango, lanza una `IllegalArgumentException` con un mensaje descriptivo.
8. Aplica **Sobreescritura (`@Override`)**:
9. `hacerSonido()`: Debe retornar el sonido característico de un conejo (por ejemplo: `"¡Sniff sniff!"`).
10. `toString()`: Debe retornar la representación en texto del conejo incluyendo el nombre, la edad y la longitud de sus orejas.

1. **Actualizar el Menú en `Main.java`:**
2. Agrega una nueva opción al menú en consola: `3. Registrar Conejo` (reorganizando las opciones para que *Mostrar todos* sea la opción 4 y *Salir* sea la 5).
3. Solicita al usuario los datos del conejo por teclado (`nombre`, `edad`, `longitudOrejas`).
4. Envuelve la creación e inserción del objeto en un bloque `try-catch` para manejar posibles errores de entrada o excepciones de validación.

1. **Verificación:**
2. Ejecuta el programa, registra un conejo e intenta ingresar una longitud de orejas negativa (ejemplo: `-5.0`) para verificar que tu manejo de excepciones ataje el error correctamente.

3. Selecciona la opción de mostrar todos los animales para confirmar que el conejo aparece junto a los perros y gatos en el mismo `ArrayList<Animal>` gracias al **Polimorfismo**.

CAPÍTULO 3 • LECCIÓN 3.7

Clase 8 - Polimorfismo en Java

Polimorfismo en Java — Tutorial para Principiantes

1. ¿Qué es el Polimorfismo?

La palabra proviene del griego: **poly** (muchos) + **morphos** (formas). En programación significa que **un mismo objeto puede comportarse de diferentes maneras** según el contexto.

> **NOTA CLAVE**
> **NOTE NOTE Idea central:** Una referencia de una clase padre puede apuntar a objetos de clases hijas, y al llamar a un método, Java ejecuta la versión específica del objeto real (no de la referencia).

2. Los Dos Tipos de Polimorfismo en Java

TIPO	NOMBRE TÉCNICO	¿CUÁNDO OCURRE?	MECANISMO
En tiempo de compilación	*Static/Compile-time*	Al compilar el código	Sobrecarga de métodos (`over-loading`)
En tiempo de ejecución	*Dynamic/Runtime*	Mientras el programa corre	Sobrescritura de métodos (-`overriding`)

3. Polimorfismo en Tiempo de Compilación (Sobrecarga)

Ocurre cuando una clase tiene **varios métodos con el mismo nombre pero diferentes parámetros** (diferente cantidad, tipo o orden).

Reglas de la sobrecarga:

- El nombre del método debe ser igual.
- La lista de parámetros debe ser diferente.
- El tipo de retorno **no importa** para sobrecargar (puede ser igual o diferente).
- Puede ocurrir dentro de la **misma clase** o entre clase padre e hija.

Ejemplo completo:

JAVA

```java
public class Calculadora {

    // Método 1: sumar dos enteros
    public int sumar(int a, int b) {
        return a + b;
    }
```

```
    // Método 2: SOBRECARGA — sumar tres enteros (diferente cantidad de parámetros)
    public int sumar(int a, int b, int c) {
        return a + b + c;
    }

    // Método 3: SOBRECARGA — sumar dos doubles (diferente tipo de parámetros)
    public double sumar(double a, double b) {
        return a + b;
    }

    // Método 4: SOBRECARGA — concatenar dos Strings
    public String sumar(String a, String b) {
        return a + b;
    }

    public static void main(String[] args) {
        Calculadora calc = new Calculadora();

        // Java decide en TIEMPO DE COMPILACIÓN cuál método usar
        // según los argumentos que pasamos.

        System.out.println(calc.sumar(5, 3));           // Llama al método 1 ' 8
        System.out.println(calc.sumar(5, 3, 2));        // Llama al método 2 ' 10
        System.out.println(calc.sumar(5.5, 3.2));       // Llama al método 3 ' 8.7
        System.out.println(calc.sumar("Hola ", "Mundo")); // Llama al método 4 ' "Hola Mundo"
    }
}
```

¿Qué pasó aquí?

- Java **no se confunde** aunque los métodos se llamen igual.
- En tiempo de compilación, el compilador ve los tipos de los argumentos y elige el método adecuado.
- Esto se llama **enlace estático** o *early binding*.

---

4. Polimorfismo en Tiempo de Ejecución (Sobrescritura)

Este es el polimorfismo **más importante** en Java y el que más se usa en programación orientada a objetos.

Conceptos previos imprescindibles:

a) Herencia

Para que exista sobrescritura, primero debe existir una relación de herencia (clase padre ' clase hija).

b) Upcasting (Conversión hacia arriba)

Es cuando asignas un objeto de una clase hija a una referencia de la clase padre.

JAVA

```
Animal miAnimal = new Perro();  // Upcasting implícito
```

c) Sobrescritura de métodos (`@Override`)

La clase hija **redefine** un método que heredó del padre, manteniendo la misma firma (nombre, parámetros y retorno).

Ejemplo completo paso a paso:

JAVA

```java
// ==================== CLASE PADRE ====================
class Animal {

    // Método que las clases hijas pueden sobrescribir
    public void hacerSonido() {
        System.out.println("El animal hace un sonido genérico");
    }

    public void dormir() {
        System.out.println("Zzz...");
    }
}

// ==================== CLASES HIJAS ====================
class Perro extends Animal {

    // SOBRESCRITURA: usamos @Override para indicar que estamos
    // redefiniendo un método del padre. Esto es opcional pero MUY recomendado.
    @Override
    public void hacerSonido() {
        System.out.println("El perro dice: ¡Guau guau!");
    }

    // Método exclusivo del Perro (NO está en Animal)
    public void olfatear() {
        System.out.println("El perro está olfateando");
    }
}

class Gato extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("El gato dice: ¡Miau miau!");
    }
}

class Vaca extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("La vaca dice: ¡Muuu!");
    }
}

// ==================== CLASE PRINCIPAL ====================
public class Granja {

    public static void main(String[] args) {

        // ============================================
        // POLIMORFISMO EN ACCIÓN: UPCASTING
        // ============================================

        // Creamos referencias de tipo Animal, pero apuntan a objetos de tipos hijos
        Animal animal1 = new Perro();   // Upcasting implícito
        Animal animal2 = new Gato();    // Upcasting implícito
        Animal animal3 = new Vaca();    // Upcasting implícito

        // Llamamos al mismo método, pero cada uno se comporta diferente
        System.out.println("=== Sonidos de los animales ===");
```

```
        animal1.hacerSonido();  // '"El perro dice: ¡Guau guau!"
        animal2.hacerSonido();  // '"El gato dice: ¡Miau miau!"
        animal3.hacerSonido();  // '"La vaca dice: ¡Muuu!"

        // El método dormir() NO fue sobrescrito, así que usa el de Animal
        System.out.println("\n=== Todos duermen igual ===");
        animal1.dormir();  // '"Zzz..."
        animal2.dormir();  // '"Zzz..."
        animal3.dormir();  // '"Zzz..."
    }
}
```

Salida del programa:

JAVASCRIPT

```
=== Sonidos de los animales ===
El perro dice: ¡Guau guau!
El gato dice: ¡Miau miau!
La vaca dice: ¡Muuu!

=== Todos duermen igual ===
Zzz...
Zzz...
Zzz...
```

5. ¿Por qué funciona esto? La magia detrás

Cuando haces:

JAVA

```
Animal animal = new Perro();
animal.hacerSonido();
```

1. **La referencia** (`animal`) es de tipo `Animal`.
2. **El objeto real** en memoria es de tipo `Perro`.
3. Java usa una tabla interna llamada **vtable** (virtual method table).
4. Al llamar `hacerSonido()`, Java **no mira el tipo de la referencia**, sino el **tipo real del objeto** (`Perro`).
5. Por eso ejecuta la versión de `Perro`, no la de `Animal`.

Esto se llama **enlace dinámico** o *late binding*.

6. Reglas estrictas para la sobrescritura (`@Override`)

Para que un método sea verdaderamente sobrescrito, debe cumplir **TODAS** estas reglas:

REGLA	DESCRIPCIÓN
Misma firma	Mismo nombre, mismos parámetros (tipo, cantidad, orden), mismo tipo de retorno (o covariante).

No puede ser `final`	Un método `final` en el padre **no puede** ser sobrescrito.
No puede ser `static`	Los métodos `static` se ocultan, no se sobrescriben.
No puede ser más restrictivo	El modificador de acceso en la hija debe ser igual o menos restrictivo (`public` ' `protected` ' default `private`).
No puede lanzar excepciones más amplias	Las excepciones declaradas con `throws` deben ser iguales o sub-clases de las del padre.

Ejemplo de reglas:

JAVA

```java
class Vehiculo {
    public void arrancar() {
        System.out.println("Vehículo arrancando");
    }

    protected void acelerar() {
        System.out.println("Acelerando");
    }

    private void secretMethod() {  // No se hereda, no se puede sobrescribir
        System.out.println("Secreto");
    }
}

class Moto extends Vehiculo {

    @Override
    public void arrancar() {  //  Misma firma, mismo acceso
        System.out.println("Moto arrancando con patada");
    }

    @Override
    public void acelerar() {  //  protected ' public (menos restrictivo, válido)
        System.out.println("Moto acelerando rápido");
    }

    // LNO podríamos hacer:
    // @Override
    // private void acelerar() { ... }  // ERROR: public ' private es más restrictivo
}
```

7. Ejemplo práctico completo: Sistema de pagos

Este es un ejemplo del mundo real donde el polimorfismo brilla. Imagina un sistema de e-commerce que acepta diferentes métodos de pago.

JAVA

```java
// ==================== CLASE PADRE ====================
abstract class MetodoPago {
    protected double monto;

    public MetodoPago(double monto) {
        this.monto = monto;
    }
```

```java

    // Método abstracto: cada método de pago lo implementa a su manera
    public abstract void procesarPago();

    // Método concreto: común para todos
    public void mostrarRecibo() {
        System.out.println("Pago procesado por: $" + monto);
    }
}

// ==================== CLASES HIJAS ====================
class PagoTarjeta extends MetodoPago {
    private String numeroTarjeta;
    private String cvv;

    public PagoTarjeta(double monto, String numeroTarjeta, String cvv) {
        super(monto);
        this.numeroTarjeta = numeroTarjeta;
        this.cvv = cvv;
    }

    @Override
    public void procesarPago() {
        System.out.println("=Procesando pago con tarjeta ****"
            + numeroTarjeta.substring(numeroTarjeta.length() - 4));
        System.out.println("   Validando CVV y fondos...");
    }
}

class PagoPayPal extends MetodoPago {
    private String email;

    public PagoPayPal(double monto, String email) {
        super(monto);
        this.email = email;
    }

    @Override
    public void procesarPago() {
        System.out.println("<Procesando pago con PayPal");
        System.out.println("   Cuenta: " + email);
        System.out.println("   Redirigiendo a PayPal API...");
    }
}

class PagoEfectivo extends MetodoPago {
    private String sucursal;

    public PagoEfectivo(double monto, String sucursal) {
        super(monto);
        this.sucursal = sucursal;
    }

    @Override
    public void procesarPago() {
        System.out.println("=Generando ficha de pago en efectivo");
        System.out.println("   Sucursal: " + sucursal);
        System.out.println("   El cliente tiene 48 horas para pagar");
    }
}

// ==================== SISTEMA DE PAGOS ====================
class SistemaPagos {

    // ESTE ES EL PODER DEL POLIMORFISMO:
```

```
    // Este método NO sabe (ni le importa) qué tipo de pago es.
    // Simplemente recibe un MetodoPago y lo procesa.
    public void ejecutarPago(MetodoPago pago) {
        System.out.println("\nTPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPW
        System.out.println("Q    INICIANDO TRANSACCIÓN        Q");
        System.out.println("ZPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP]

        pago.procesarPago();   // ¡Polimorfismo! Se ejecuta la versión correcta
        pago.mostrarRecibo();
        System.out.println(" Transacción completada\n");
    }
}

// ==================== MAIN ====================
public class TiendaOnline {
    public static void main(String[] args) {
        SistemaPagos sistema = new SistemaPagos();

        // Creamos diferentes pagos (todos son MetodoPago)
        MetodoPago pago1 = new PagoTarjeta(150.00, "4532123456789012", "123");
        MetodoPago pago2 = new PagoPayPal(89.99, "cliente@email.com");
        MetodoPago pago3 = new PagoEfectivo(45.50, "Oxxo Centro");

        // El mismo método 'ejecutarPago' maneja TODOS los tipos
        sistema.ejecutarPago(pago1);
        sistema.ejecutarPago(pago2);
        sistema.ejecutarPago(pago3);
    }
}
```

Salida:

JAVASCRIPT

```
TPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPW
Q    INICIANDO TRANSACCIÓN        Q
ZPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP]
=₱Procesando pago con tarjeta ****9012
   Validando CVV y fondos...
Pago procesado por: $150.0
 Transacción completada

TPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPW
Q    INICIANDO TRANSACCIÓN        Q
ZPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP]
<₱Procesando pago con PayPal
   Cuenta: cliente@email.com
   Redirigiendo a PayPal API...
Pago procesado por: $89.99
 Transacción completada

TPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPW
Q    INICIANDO TRANSACCIÓN        Q
ZPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP]
=₿enerando ficha de pago en efectivo
   Sucursal: Oxxo Centro
   El cliente tiene 48 horas para pagar
Pago procesado por: $45.5
 Transacción completada
```

¿Por qué esto es tan poderoso?

- Si mañana agregas `PagoCripto`, `PagoTransferencia`, etc., **no necesitas modificar** `SistemaPagos`.
- Solo creas la nueva clase que extienda `MetodoPago`, y el sistema automáticamente la acepta.
- Esto se conoce como el principio **Open/Closed**: abierto para extender, cerrado para modificar.

---

8. Polimorfismo con Arrays y Colecciones

Puedes almacenar objetos de diferentes tipos hijos en un array del tipo padre:

JAVA

```java
public class Zoologico {
    public static void main(String[] args) {

        // Array de tipo Animal que contiene objetos de tipos hijos
        Animal[] animales = {
            new Perro(),
            new Gato(),
            new Vaca(),
            new Perro()
        };

        // Recorremos y cada uno hace su sonido específico
        for (Animal animal : animales) {
            animal.hacerSonido();  // Polimorfismo en cada iteración
        }

        // También funciona con ArrayList
        java.util.ArrayList<Animal> lista = new java.util.ArrayList<>();
        lista.add(new Gato());
        lista.add(new Perro());

        for (Animal a : lista) {
            a.hacerSonido();
        }
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.8

Clase 9 - Clases Abstractas

Clases Abstractas en Java

1. ¿Qué es una clase abstracta?

Una **clase abstracta** es una clase que **no puede ser instanciada directamente** y que puede contener **métodos abstractos** (sin implementación) junto con métodos concretos (con implementación).

Se declara con la palabra clave `abstract`:

JAVA

```java
abstract class Figura {
    // cuerpo de la clase
}
```

Propiedad fundamental: solo puede existir a través de sus subclases.

JAVA

```java
abstract class Figura { }

class Circulo extends Figura { }

// LError de compilación
// Figura f = new Figura();

//  Correcto
Figura f = new Circulo();
```

> **NOTA CLAVE**
> **Analogía:** una clase abstracta es como un "contrato parcialmente definido". Dice *qué* debe hacer una figura, pero no siempre *cómo* debe hacerlo.

2. ¿Por qué usar clases abstractas?

RAZÓN	EXPLICACIÓN
Compartir código	Las subclases heredan implementaciones reutilizables
Polimorfismo	Puedes tratar objetos de distintas subclases de forma uniforme
Definir plantillas	Establecen el "esqueleto" del comportamiento
Evitar instanciación innecesaria	Conceptos genéricos no deberían existir solos

Caso de uso clásico

Imagina un sistema bancario. Toda cuenta tiene `depositar()` con la misma lógica, pero `calcularInteres()` varía según el tipo de cuenta:

JAVA

```java
abstract class CuentaBancaria {
    protected double saldo;

    public void depositar(double monto) {
        if (monto <= 0) throw new IllegalArgumentException("Monto inválido");
        saldo += monto;
    }

    public abstract double calcularInteres(); // cada subclase lo define
}

class CuentaAhorro extends CuentaBancaria {
    @Override
    public double calcularInteres() {
        return saldo * 0.05; // 5% anual
    }
}

class CuentaCorriente extends CuentaBancaria {
    @Override
    public double calcularInteres() {
        return 0; // no genera intereses
    }
}
```

JAVA

```java
CuentaBancaria[] cuentas = { new CuentaAhorro(), new CuentaCorriente() };
for (CuentaBancaria c : cuentas) {
    System.out.println(c.calcularInteres()); // polimorfismo en acción
}
```

3. Métodos abstractos vs. concretos

3.1 Método abstracto

- Se declara **sin cuerpo**, terminando en `;`
- Obliga a las subclases a implementarlo (a menos que también sean abstractas)

JAVA

```java
abstract double calcularArea();
```

3.2 Método concreto (normal)

- Tiene implementación completa
- Las subclases heredan el comportamiento tal cual o lo sobrescriben con `@Override`

JAVA

```
void mostrarInfo() {
    System.out.println("Figura genérica");
}
```

NOTA CLAVE
IMPORTANT 1. **Una clase con al menos un método abstracto DEBE ser abstracta.** 2. **Una clase abstracta NO tiene por qué tener métodos abstractos** (aunque es poco común).

JAVA

```
//  Válido: abstracta sin métodos abstractos
abstract class Utilidad {
    void ayuda() { System.out.println("Ayuda"); }
}
```

4. Reglas de herencia y sobrescritura

4.1 Subclases concretas deben implementar TODO

JAVA

```
abstract class Animal {
    abstract void sonido();
    abstract void moverse();
}

class Perro extends Animal {
    @Override void sonido() { System.out.println("Guau"); }
    @Override void moverse() { System.out.println("Corre"); }
}
```

4.2 Una subclase abstracta puede omitir implementaciones

JAVA

```
abstract class Ave extends Animal {
    @Override void moverse() { System.out.println("Vuela"); }
    // sonido() queda pendiente para otra subclase
}

class Gallo extends Ave {
    @Override void sonido() { System.out.println("Kikirikí"); }
}
```

4.3 Herencia única

Java no permite la herencia múltiple de clases; una clase solo puede heredar de una única clase padre (ya sea concreta o abstracta) mediante la palabra clave `extends`.

5. Constructores en clases abstractas

Aunque no puedas instanciar la clase abstracta, **sí puede tener constructores**. Se invocan desde las subclases con `super()`:

JAVA

```java
abstract class Empleado {
    protected String nombre;
    protected double salarioBase;

    // Constructor de la clase abstracta
    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    abstract double calcularPago();
}

class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double salarioBase, double bono) {
        super(nombre, salarioBase); // llama al constructor abstracto
        this.bono = bono;
    }

    @Override
    double calcularPago() {
        return salarioBase + bono;
    }
}
```

NOTA CLAVE

Nota: Los constructores sirven para inicializar el estado heredado por todas las subclases.

6. Variables, bloques y miembros estáticos

Una clase abstracta puede tener todo lo que una clase normal tiene:

JAVA

```java
abstract class Configuracion {
    static final int VERSION = 2;
    protected String entorno = "dev";

    { System.out.println("Configuración creada"); }

    static void info() {
        System.out.println("Versión: " + VERSION);
    }

    final void validar() {
        if (entorno == null) throw new IllegalStateException();
    }
}
```

> **ADVERTENCIA**
> **WARNING** *`static` + `abstract`: Un método estático no se hereda ni se sobrescribe, por lo que combinar estas palabras es ilegal.* **`private` + `abstract`**: Un método privado no es visible para las subclases, por lo que nunca podría implementarse.

---

7. `abstract` vs `final` — incompatibles

- `abstract` ' "debes heredarme e implementarme"
- `final` (en clase) ' "prohibido heredar de mí"

Son conceptos opuestos y, por lo tanto, no pueden usarse juntos en la declaración de una clase.

---

8. El patrón Template Method

Este es el uso más elegante de clases abstractas: el método plantilla define el **algoritmo** en la clase base y delega los pasos variables a las subclases.

JAVA

```java
abstract class BebidaCaliente {

    public final void preparar() {          // final ' no se puede alterar
        hervirAgua();
        agregarIngrediente();               // abstracto
        verterEnTaza();
        agregarExtras();                    // hook (opcional)
    }

    private void hervirAgua() { System.out.println("Hirviendo agua"); }
    private void verterEnTaza() { System.out.println("Vertiendo en la taza"); }

    protected abstract void agregarIngrediente();

    protected void agregarExtras() { } // Hook
}
```

---

9. Errores comunes

1. **Intentar instanciar:** `new ClaseAbstracta()` ' Error.
2. **Clase no abstracta con método abstracto:** Si la clase tiene un `abstract method`, la clase debe ser `abstract`.
3. **Método abstracto con cuerpo:** `abstract void metodo() { }` ' Error.
4. **No implementar métodos:** La subclase concreta está obligada a proveer código para todos los métodos abstractos heredados.

---

10. Ejemplo completo integrador

JAVA

```java
import java.util.ArrayList;
import java.util.List;

abstract class Vehiculo {
    protected String marca;
    protected String modelo;
    protected static int totalVehiculos = 0;

    public Vehiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        totalVehiculos++;
    }

    abstract double calcularConsumo(double km);

    public void mostrarDatos() {
        System.out.println(marca + " " + modelo);
    }

    public static int getTotalVehiculos() {
        return totalVehiculos;
    }
}

class Auto extends Vehiculo {
    private double rendimiento;

    public Auto(String marca, String modelo, double rendimiento) {
        super(marca, modelo);
        this.rendimiento = rendimiento;
    }

    @Override
    double calcularConsumo(double km) {
        return km / rendimiento;
    }
}

class Camion extends Vehiculo {
    private double capacidadTanque;

    public Camion(String marca, String modelo, double capacidadTanque) {
        super(marca, modelo);
        this.capacidadTanque = capacidadTanque;
    }

    @Override
    double calcularConsumo(double km) {
        return (km / 100.0) * 0.35 * capacidadTanque;
    }
}
```

CAPÍTULO 3 • LECCIÓN 3.9

Clase 10 - Interfaces

Interfaces en Java

Vamos a construir el concepto paso a paso, como si nunca hubieras visto una interfaz. Al final tendrás ejemplos completos y ejecutables.

1. ¿Qué problema resuelven las interfaces?

Antes de definir nada, entendamos el problema. Imagina que estás haciendo un juego y tienes dos clases:

JAVA

```java
class Perro {
    void hacerSonido() {
        System.out.println("Guau guau");
    }
}

class Gato {
    void hacerSonido() {
        System.out.println("Miau miau");
    }
}
```

Quieres una lista donde metas **perros y gatos** y recorrerla haciendo que cada animal emita su sonido. El problema: Java es de **tipado fuerte**, y una lista solo acepta un tipo:

JAVA

```java
Perro[] animales = new Perro[2]; // aquí solo caben Perros, no Gatos
```

Necesitas un **tipo común**, un "contrato" que diga: *"todo lo que sea de este tipo, garantiza que sabe hacer ciertas cosas"*. Ese contrato es una **interfaz**.

2. Definición formal

Una **interfaz** es un tipo de referencia en Java que define **un conjunto de métodos sin implementación** (solo la firma). Una clase puede **implementar** la interfaz y así se compromete a escribir el cuerpo de esos métodos.

Pensémoslo así:

- La **interfaz** es como el **manual de instrucciones** o la **especificación**: "todo dispositivo USB debe tener esta forma de conector y saber hacer X e Y".
- La **clase que la implementa** es el **dispositivo real**: un mouse, un teclado, un pendrive... todos cumplen el mismo contrato pero funcionan por dentro de forma diferente.

3. Sintaxis básica

Declarar una interfaz

```java
interface NombreDeLaInterfaz {
    tipoRetorno nombreMetodo(parametros);
}
```

Reglas importantes (Java agrega esto automáticamente por ti):

1. Todos los métodos son **públicos** y **abstractos** por defecto. Escribir `public abstract` es opcional.
2. Todos los atributos (variables) son **`public static final`** por defecto, es decir, **constantes**.

```java
interface Animal {
    // Esto es una constante implícita:
    // public static final int PATAS = 4;
    int PATAS = 4;

    // Esto es un método abstracto implícito:
    // public abstract void hacerSonido();
    void hacerSonido();
}
```

Implementar una interfaz

Se usa la palabra clave **`implements`** (no `extends`, que es para herencia de clases):

```java
class Perro implements Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau guau");
    }
}
```

Detalles que no debes omitir:

- **`@Override`** es una anotación opcional pero muy recomendada: le dice al compilador "estoy sobrescribiendo un método de la interfaz", y si te equivocas en el nombre o la firma, Java te avisa con error de compilación en vez de compilar algo silenciosamente mal.
- **No puedes reducir la visibilidad**: la interfaz declara el método como `public`, así que tu implementación también debe ser `public`.

4. Primer programa completo y ejecutable

```java
```

```
// Interfaz
interface Animal {
    void hacerSonido();   // sin cuerpo, solo la promesa
    void moverse();
}

// Clase 1 que cumple el contrato
class Perro implements Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau guau");
    }

    @Override
    public void moverse() {
        System.out.println("El perro corre en cuatro patas");
    }
}

// Clase 2 que cumple el contrato
class Gato implements Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Miau miau");
    }

    @Override
    public void moverse() {
        System.out.println("El gato salta silenciosamente");
    }
}

// Programa principal
public class Main {
    public static void main(String[] args) {
        // Polimorfismo: la referencia es del tipo interfaz
        Animal[] animales = new Animal[2];
        animales[0] = new Perro();
        animales[1] = new Gato();

        for (Animal a : animales) {
            a.hacerSonido();
            a.moverse();
        }
    }
}
```

Salida:

JAVASCRIPT

```
Guau guau
El perro corre en cuatro patas
Miau miau
El gato salta silenciosamente
```

Qué acaba de pasar aquí (lo más importante del tutorial)

1. **`Animal a`** declara una variable de tipo **interfaz**. No puedes hacer `new Animal()` (no se puede instanciar una interfaz directamente, porque no tiene cuerpos de métodos).

2. **`new Perro()`** crea un objeto real. Java verifica que `Perro` implementa todos los métodos de `Animal`; si faltara alguno, daría **error de compilación**.
3. En el bucle, cada llamada `a.hacerSonido()` ejecuta el código de la clase real del objeto (**Perro** o **Gato**), no un código genérico. Esto es **polimorfismo**: la misma línea de código se comporta diferente según el objeto real.

¿Por qué no usar herencia (extends) en su lugar?

Podrías hacer una clase abstracta `Animal` con `extends`. Pero Java **no permite heredar de dos clases** (no hay herencia múltiple de clases), y sí permite **implementar varias interfaces**. Además, con interfaces separas completamente *qué se debe hacer* (la interfaz) de *cómo se hace* (cada clase).

---

5. Implementar múltiples interfaces

Esto es una de las razones principales de existir de las interfaces:

JAVA

```java
interface Volador {
    void volar();
}

interface Nadador {
    void nadar();
}

// Un pato puede volar Y nadar
class Pato implements Volador, Nadador {
    @Override
    public void volar() {
        System.out.println("El pato aletea y vuela");
    }

    @Override
    public void nadar() {
        System.out.println("El pato nada en el estanque");
    }
}

public class Main {
    public static void main(String[] args) {
        Pato pato = new Pato();
        pato.volar();
        pato.nadar();
    }
}
```

Salida:

JAVASCRIPT

```
El pato aletea y vuela
El pato nada en el estanque
```

Nota: si dos interfaces declaran un método con la **m misma firma** (mismo nombre, mismos parámetros, mismo tipo de retorno), una sola implementación en la clase satisface ambas. Si las firmas son incompatibles (mismo nombre pero distinto retorno), Java fuerza a resolver el conflicto y es un tema avanzado que no necesitas ahora.

6. ¿Puede una interfaz tener atributos?

Sí, pero son **constantes** (`public static final` implícitos), es decir, valores fijos compartidos:

JAVA

```java
interface Configuracion {
    int MAX_USUARIOS = 100;          // constante
    String VERSION = "1.0";          // constante
}

class Servidor implements Configuracion {
    void mostrarConfig() {
        System.out.println("Máximo de usuarios: " + MAX_USUARIOS);
        System.out.println("Versión: " + VERSION);
    }
}
```

Accedes a ellos como `Configuracion.MAX_USUARIOS` (o desde la clase que las implementa). No puedes poner variables de instancia en una interfaz: no hay objetos de interfaz, no hay "estado" propio.

7. Interfaces que heredan de otras interfaces

Una interfaz puede **extender** otra interfaz con `extends`, y puede extender **varias** a la vez:

JAVA

```java
interface SerVivo {
    void respirar();
}

interface Animal extends SerVivo {
    void hacerSonido();
}

class Perro implements Animal {
    @Override
    public void respirar() {
        System.out.println("El perro respira");
    }

    @Override
    public void hacerSonido() {
        System.out.println("Guau guau");
    }
}
```

Fíjate en la cadena: `Animal extends SerVivo` hereda `respirar()`, y como `Perro implements Animal`, `Perro` está obligado a implementar **ambos** métodos.

8. Métodos `default` (desde Java 8)

¿Qué pasa si agregas un método nuevo a una interfaz con 50 clases que la implementan? Sin ayuda, las 50 dejan de compilar. Por eso desde Java 8 existe el método **`default`**: tiene cuerpo en la propia interfaz y las clases **pueden** sobrescribirlo, pero no están obligadas.

JAVA

```java
interface Animal {
    void hacerSonido();

    // Método default: ya viene con implementación
    default void dormir() {
        System.out.println("Zzz... (implementación por defecto)");
    }
}

class Perro implements Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Guau guau");
    }
}

class Gato implements Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Miau miau");
    }

    // Sobrescribimos el default porque queremos comportamiento propio
    @Override
    public void dormir() {
        System.out.println("El gato duerme 16 horas al día");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal p = new Perro();
        Animal g = new Gato();

        p.hacerSonido();
        p.dormir();   // usa el default de la interfaz

        g.hacerSonido();
        g.dormir();   // usa la sobrescritura del Gato
    }
}
```

Salida:

JAVASCRIPT

```
Guau guau
Zzz... (implementación por defecto)
Miau miau
El gato duerme 16 horas al día
```

Regla: si una clase implementa **dos interfaces** que tienen el **mismo método `default`**, la clase **debe** sobrescribirlo (Java no sabe cuál de las dos elegir).

9. Métodos `static` en interfaces (desde Java 8)

También pueden tener métodos estáticos con cuerpo. Se llaman usando el **nombre de la interfaz**, nunca desde un objeto:

JAVA

```java
interface Matematicas {
    static int cuadrado(int n) {
        return n * n;
    }
}

public class Main {
    public static void main(String[] args) {
        int r = Matematicas.cuadrado(5);   // se llama desde la interfaz
        System.out.println(r);              // 25
    }
}
```

No se heredan ni se sobrescriben en las clases que implementan la interfaz.

---

10. Métodos `private` en interfaces (desde Java 9)

Sirven para que los métodos `default` reutilicen código sin exponerlo. No son accesibles desde fuera de la interfaz:

JAVA

```java
interface Reporte {
    default void imprimirResumen() {
        String contenido = generarContenido(); // usa el método privado
        System.out.println(contenido);
    }

    private String generarContenido() {
        return "Resumen generado internamente";
    }
}
```

---

11. Ejemplo completo del mundo real: sistema de pagos

Este ejemplo integra casi todo lo visto. Piénsalo como el diseño de una tienda en línea que acepta varios métodos de pago.

JAVA

```java
// Contrato: todo método de pago debe poder procesar un monto
interface MetodoPago {
    void procesarPago(double monto);

    default void mostrarComprobante() {
        System.out.println("Comprobante generado.");
    }
}

// Implementación 1: tarjeta
class PagoTarjeta implements MetodoPago {
```

```
    private String numeroTarjeta;

    public PagoTarjeta(String numeroTarjeta) {
        this.numeroTarjeta = numeroTarjeta;
    }

    @Override
    public void procesarPago(double monto) {
        System.out.println("Cobrando $" + monto + " a la tarjeta ****" +
                numeroTarjeta.substring(numeroTarjeta.length() - 4));
    }
}

// Implementación 2: PayPal
class PagoPayPal implements MetodoPago {
    private String correo;

    public PagoPayPal(String correo) {
        this.correo = correo;
    }

    @Override
    public void procesarPago(double monto) {
        System.out.println("Cobrando $" + monto + " a la cuenta PayPal " + correo);
    }
}

// La tienda NO sabe ni le importa qué método de pago es.
// Solo habla con el contrato MetodoPago.
class Tienda {
    void cobrar(MetodoPago metodo, double monto) {
        metodo.procesarPago(monto);
        metodo.mostrarComprobante();
    }
}

public class Main {
    public static void main(String[] args) {
        Tienda tienda = new Tienda();

        MetodoPago tarjeta = new PagoTarjeta("1234567890123456");
        MetodoPago paypal  = new PagoPayPal("cliente@correo.com");

        tienda.cobrar(tarjeta, 250.50);
        tienda.cobrar(paypal, 99.99);
    }
}
```

Salida:

JAVASCRIPT

```
Cobrando $250.5 a la tarjeta ****3456
Comprobante generado.
Cobrando $99.99 a la cuenta PayPal cliente@correo.com
Comprobante generado.
```

El punto clave de diseño: si mañana agregas `PagoBitcoin`, **no tocas la clase `Tienda`**. Solo creas la nueva clase que implementa `MetodoPago`. A eso se le llama **desacoplamiento**, y es el beneficio práctico número uno de las interfaces.

12. Interfaz vs clase abstracta: tabla comparativa

CARACTERÍSTICA	INTERFAZ	CLASE ABSTRACTA
Herencia/implementación	`implements`, varias a la vez	`extends`, **una sola**
Métodos abstractos	Sí (todos por defecto)	Sí, mezclados con concretos
Métodos con cuerpo	Solo `default`, `static`, `private` (Java 8/9+)	Sí, normales
Variables	Solo constantes (`static final`)	Variables de instancia normales
Constructores	No	Sí
¿Cuándo usarla?	Definir **capacidades/contratos** independientes	Compartir **código y estado** entre subclases cercanas

Regla práctica: si solo necesitas decir *"cualquier cosa que sepa hacer X"*, usa interfaz. Si necesitas una clase base con datos y lógica compartida, usa clase abstracta.

---

14. Errores comunes de principiantes

1. **`new Interfaz()`** ’ Error: no se puede instanciar. Necesitas una clase que la implemente.
2. **Olvidar `public` en la implementación** ’ Error de compilación: "attempting to assign weaker access privileges".
3. **No implementar todos los métodos abstractos** ’ Error: la clase debe declararse `abstract` o implementar todo.
4. **Confundir `extends` con `implements`** ’ Las clases `extends` a otras clases; `implements` a interfaces. Una interfaz `extends` a otra interfaz.
5. **Poner lógica en un método de interfaz sin `default`** ’ Error: los métodos normales de interfaz no tienen cuerpo.
6. **Esperar que las interfaces guarden estado** ’ No tienen variables de instancia; son contratos puros.

CAPÍTULO 3 • LECCIÓN 3.10

Clase 13 - Gestores de Dependencias

Gestores de Dependencias en Java: Maven y Gradle

En el mundo real de la programación, rara vez escribimos todo el código desde cero. A menudo utilizamos **librerías** y **frameworks** creados por otros desarrolladores para resolver problemas comunes (como conectarse a una base de datos, crear una interfaz gráfica o consumir una API REST). A estos componentes externos los llamamos **dependencias**.

Imagina que estás construyendo una casa. No fabricas tus propios ladrillos, ni fundes tu propio acero; los compras a proveedores. Los gestores de dependencias son como los camiones de reparto que traen exactamente los materiales que necesitas, en las versiones correctas, directo a tu obra.

---

1. ¿Qué es un Gestor de Dependencias?

Un gestor de dependencias es una herramienta que automatiza el proceso de:

1. **Descargar librerías:** Busca en internet (en repositorios centrales) las librerías que tu proyecto necesita y las descarga.
2. **Gestionar versiones:** Asegura que uses la versión correcta de cada librería, evitando conflictos.
3. **Dependencias transitivas:** Si la librería "A" que necesitas depende a su vez de la librería "B", el gestor descargará ambas automáticamente.
4. **Compilar y empaquetar:** Transforma tu código fuente y las librerías en un programa ejecutable (como un archivo `.jar`).

En el ecosistema Java, los dos gestores más populares son **Maven** y **Gradle**.

---

2. Apache Maven

Maven es el gestor de dependencias más tradicional y utilizado en Java. Utiliza un archivo de configuración llamado `pom.xml` (Project Object Model) basado en XML.

El Archivo `pom.xml`

Es el corazón de cualquier proyecto Maven. Aquí defines la información de tu proyecto y qué librerías necesitas.

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <!-- 1. Identificación de tu proyecto -->
    <groupId>com.miempresa</groupId>       <!-- Organización -->
    <artifactId>mi-aplicacion</artifactId>  <!-- Nombre del proyecto -->
    <version>1.0-SNAPSHOT</version>         <!-- Versión actual -->

    <!-- 2. Configuración de Java -->
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
```

```
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <!-- 3. Lista de Dependencias -->
    <dependencies>
        <!-- Ejemplo: Librería para convertir objetos a JSON (Gson) -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
            <version>2.10.1</version>
        </dependency>
    </dependencies>
</project>
```

Ciclo de vida de Maven (Comandos principales)

Maven divide la construcción en "fases". Al ejecutar una fase, se ejecutan todas las anteriores.

- `mvn clean`: Borra los archivos compilados de ejecuciones anteriores (limpia el proyecto).
- `mvn compile`: Compila tu código fuente Java.
- `mvn test`: Ejecuta las pruebas automatizadas (unit tests).
- `mvn package`: Empaqueta tu código compilado en un archivo distribuible, como un JAR.
- `mvn install`: Guarda el paquete en tu repositorio local para usarlo en otros de tus proyectos.

---

3. Gradle

Gradle es un gestor de dependencias más moderno que surgió como una alternativa más flexible y rápida a Maven. En lugar de XML, utiliza un DSL (Domain Specific Language) basado en **Groovy** o **Kotlin**, lo que hace que los archivos de configuración sean más cortos y programables. Es el estándar en el desarrollo de aplicaciones para Android.

El Archivo `build.gradle`

Equivale al `pom.xml` de Maven, pero mucho más conciso.

GROOVY

```groovy
// 1. Plugins (herramientas adicionales)
plugins {
    id 'java'
}

// 2. Identificación del proyecto
group = 'com.miempresa'
version = '1.0-SNAPSHOT'

// 3. ¿De dónde se descargan las librerías?
repositories {
    mavenCentral() // Usa el mismo repositorio principal que Maven
}

// 4. Lista de Dependencias
dependencies {
    // Ejemplo: Librería Gson
    implementation 'com.google.code.gson:gson:2.10.1'

    // Librería solo para pruebas
    testImplementation 'org.junit.jupiter:junit-jupiter-api:5.9.2'
```

```
22 | }
```

Comandos principales de Gradle

- `gradle build`: Compila, prueba y empaqueta el proyecto.
- `gradle clean`: Limpia el directorio de construcción.
- `gradle test`: Ejecuta las pruebas.

---

4. ¿Maven o Gradle? ¿Cuál elegir?

CARACTERÍSTICA	MAVEN	GRADLE
Configuración	XML (Estricto, largo, declarativo).	Groovy/Kotlin (Conciso, programable).
Curva de aprendizaje	Baja (Las convenciones son estrictas).	Media/Alta (Mucha flexibilidad).
Rendimiento	Normal.	Muy rápido (Usa caché avanzada y ejecución incremental).
Uso principal	Proyectos Java empresariales (Spring Boot, etc.).	Proyectos Android, proyectos muy grandes o personalizados.

Recomendación: Para empezar y para la mayoría de proyectos backend o de escritorio sencillos, **Maven** es excelente porque es muy estándar y casi no requiere configuración (funciona "out of the box"). Cuando necesites configuraciones complejas de construcción o desarrolles para Android, **Gradle** es la elección.

---

5. Repositorios de Dependencias

¿De dónde saca Maven o Gradle las librerías? El lugar principal se llama **Maven Central Repository** (https://mvnrepository.com/). Es una inmensa base de datos pública y gratuita donde los creadores de librerías suben sus componentes.

Cuando pides una dependencia en tu `pom.xml`, tu gestor va a Maven Central, busca las coordenadas exactas (`groupId`, `artifactId`, `version`) y la descarga a tu computadora (a una carpeta oculta en tu usuario, usualmente `.m2`).

---

6. Ejemplos Prácticos de Integración

A continuación, veremos dos ejemplos prácticos completos de cómo un gestor de dependencias (Maven) nos permite crear aplicaciones poderosas consumiendo librerías externas de manera sencilla.

Proyecto Demo: Consumo de API REST con Retrofit

Este proyecto es un ejemplo básico de cómo consumir una API REST desde Java usando la librería **Retrofit**. El programa obtiene una lista de usuarios de una API mock, realiza todas las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y las imprime en consola.

---

1. Estructura del Proyecto

BASH

```
demo/
 pom.xml                                  # Configuración de Maven (dependencias)
 README.md                                # Esta documentación
 src/
     main/
         java
             com
                 example
                     Main.java            # Punto de entrada del programa
                     User.java            # Modelo de datos del usuario
                     Address.java         # Modelo de datos de la dirección
                     UserService.java     # Definición de la API (todas las operaciones)
```

2. ¿Qué es Maven y cómo funciona?

Maven es una herramienta que gestiona:

- **Dependencias**: Librerías externas que necesita tu proyecto (como Retrofit)
- **Compilación**: Convierte tu código Java en un programa ejecutable
- **Ejecución**: Permite correr tu programa fácilmente

El archivo `pom.xml`

`pom.xml` es el archivo de configuración de Maven. Aquí se definen todas las dependencias:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <!-- Identificación del proyecto -->
    <groupId>com.example</groupId>       <!-- Nombre de la organización -->
    <artifactId>demo</artifactId>        <!-- Nombre del proyecto -->
    <version>1.0-SNAPSHOT</version>     <!-- Versión -->

    <!-- Versión de Java que usamos -->
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <!-- Lista de dependencias (librerías externas) -->
    <dependencies>
        <!-- Retrofit: Sirve para hacer peticiones HTTP a APIs REST -->
        <dependency>
            <groupId>com.squareup.retrofit2</groupId>
            <artifactId>retrofit</artifactId>
            <version>3.0.0</version>
        </dependency>

        <!-- Gson: Convierte JSON (texto) a objetos Java y viceversa -->
        <dependency>
            <groupId>com.google.code.gson</groupId>
            <artifactId>gson</artifactId>
```

```
            <version>2.14.0</version>
        </dependency>

        <!-- Converter-Gson: Integration entre Retrofit y Gson -->
        <dependency>
            <groupId>com.squareup.retrofit2</groupId>
            <artifactId>converter-gson</artifactId>
            <version>3.0.0</version>
        </dependency>
    </dependencies>
</project>
```

3. Instalación de Dependencias

Opción A: Desde Visual Studio Code

1. Instala la extensión **"Extension Pack for Java"** de Microsoft
2. Instala la extensión **"Maven for Java"**
3. Al abrir el proyecto, VS Code detectará automáticamente el `pom.xml`
4. Las dependencias se descargarán automáticamente

Opción B: Desde Línea de Comandos

BASH

```bash
# Navega a la carpeta del proyecto
cd C:\Users\jhonf\Desktop\Ejercicios\demo

# Descarga todas las dependencias
mvn dependency:resolve

# Compila el proyecto
mvn compile

# Ejecuta el programa
mvn exec:java -Dexec.mainClass="com.example.Main"
```

Opción C: Usando Gradle (alternativa a Maven)

Si prefieres Gradle en lugar de Maven, crea un archivo `build.gradle`:

GROOVY

```groovy
plugins {
    id 'java'
}

group = 'com.example'
version = '1.0-SNAPSHOT'

repositories {
    mavenCentral()
}

dependencies {
    implementation 'com.squareup.retrofit2:retrofit:3.0.0'
    implementation 'com.google.code.gson:gson:2.14.0'
```

```
    implementation 'com.squareup.retrofit2:converter-gson:3.0.0'
}
```

4. Explicación Detallada de Cada Archivo

4.1. `Address.java` - Modelo de la Dirección

JAVA

```java
package com.example;

public class Address {
    private String street;   // Calle
    private String city;    // Ciudad
    private String country; // País

    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getCountry() { return country; }

    public void setStreet(String street) { this.street = street; }
    public void setCity(String city) { this.city = city; }
    public void setCountry(String country) { this.country = country; }
}
```

¿Qué hace este archivo?

- Define la estructura de datos para una dirección
- Cada usuario tiene una dirección anidada (un objeto dentro de otro)
- Los `getters` permiten leer los datos; los `setters` permiten modificarlos

4.2. `User.java` - Modelo del Usuario

JAVA

```java
package com.example;

public class User {
    private String id;         // Identificador único (UUID)
    private String name;       // Nombre
    private String email;      // Correo electrónico
    private String phone;      // Teléfono
    private Address address;    // Objeto Address (dirección anidada)
    private String birthdate;   // Fecha de nacimiento
    private boolean isActive;   // Si está activo o no

    // Getters para cada campo
    public String getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }
    public Address getAddress() { return address; }
    public String getBirthdate() { return birthdate; }
    public boolean isActive() { return isActive; }

```

```
    // Setters para cada campo
    public void setId(String id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setPhone(String phone) { this.phone = phone; }
    public void setAddress(Address address) { this.address = address; }
    public void setBirthdate(String birthdate) { this.birthdate = birthdate; }
    public void setActive(boolean isActive) { this.isActive = isActive; }
}
```

¿Qué hace este archivo?

- Define la estructura de un usuario
- Coincide con el JSON que devuelve la API
- `Address address` es un objeto dentro de User (relación "tiene una")

4.3. `UserService.java` - Definición de la API (CRUD Completo)

JAVA

```java
package com.example;

import java.util.List;
import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.DELETE;
import retrofit2.http.GET;
import retrofit2.http.PATCH;
import retrofit2.http.POST;
import retrofit2.http.PUT;
import retrofit2.http.Path;

public interface UserService {

    // GET /users - Obtener todos los usuarios
    @GET("users")
    Call<List<User>> getUsers();

    // GET /users/{id} - Obtener un usuario por su ID
    @GET("users/{id}")
    Call<User> getUserById(@Path("id") String id);

    // POST /users - Crear un nuevo usuario (genera UUID automáticamente)
    @POST("users")
    Call<User> createUser(@Body User user);

    // PUT /users/{id} - Actualización completa (reemplaza todo el objeto)
    @PUT("users/{id}")
    Call<User> updateUser(@Path("id") String id, @Body User user);

    // PATCH /users/{id} - Actualización parcial (mezcla solo los campos enviados)
    @PATCH("users/{id}")
    Call<User> patchUser(@Path("id") String id, @Body User user);

    // DELETE /users/{id} - Eliminar un usuario por su ID
    @DELETE("users/{id}")
    Call<Void> deleteUser(@Path("id") String id);
}
```

¿Qué hace este archivo?

- Define una **interfaz** (un contrato) para comunicarse con la API
- Cada método corresponde a una operación HTTP diferente
- `@Path("id")` inserta el valor del parámetro en la URL
- `@Body` indica que el objeto se envía en el cuerpo de la petición

Anotaciones de Retrofit:

ANOTACIÓN	MÉTODO HTTP	DESCRIPCIÓN
`@GET`	GET	Obtener datos
`@POST`	POST	Crear nuevos datos
`@PUT`	PUT	Reemplazar datos completamente
`@PATCH`	PATCH	Actualizar datos parcialmente
`@DELETE`	DELETE	Eliminar datos

4.4. `Main.java` - Programa Principal

JAVA

```java
package com.example;

import java.io.IOException;
import java.util.List;
import retrofit2.Response;
import retrofit2.Retrofit;
import retrofit2.converter.gson.GsonConverterFactory;

public class Main {
    public static void main(String[] args) {

        // PASO 1: Configurar Retrofit
        // ============================================
        Retrofit retrofit = new Retrofit.Builder()
                .baseUrl("https://playground.mockoon.com/")
                .addConverterFactory(GsonConverterFactory.create())
                .build();

        // PASO 2: Crear el servicio
        // ============================================
        UserService service = retrofit.create(UserService.class);

        // PASO 3: Ejecutar operaciones CRUD
        // ============================================

        // --- GET /users: Obtener todos los usuarios ---
        Response<List<User>> responseGetAll = service.getUsers().execute();
        if (responseGetAll.isSuccessful() && responseGetAll.body() != null) {
            for (User u : responseGetAll.body()) {
                System.out.println(u.getId() + " | " + u.getName() + " | " + u.getEmail());
            }
        }

        // --- GET /users/{id}: Obtener un usuario por ID ---
```

```
        String userId = "4d529d30-a87e-4b67-b858-fc54055bc7e1";
        Response<User> responseGetById = service.getUserById(userId).execute();

        // --- POST /users: Crear un nuevo usuario ---
        User newUser = new User();
        newUser.setName("Carlos García");
        newUser.setEmail("carlos@ejemplo.com");
        // ... configurar otros campos ...
        Response<User> responseCreate = service.createUser(newUser).execute();

        // --- PUT /users/{id}: Reemplazo completo ---
        User updatedUser = new User();
        updatedUser.setName("María López");
        // ... todos los campos ...
        Response<User> responsePut = service.updateUser(userId, updatedUser).execute();

        // --- PATCH /users/{id}: Actualización parcial ---
        User patchedUser = new User();
        patchedUser.setEmail("nuevo.email@ejemplo.com");
        // Solo se actualizan los campos proporcionados
        Response<User> responsePatch = service.patchUser(userId, patchedUser).execute();

        // --- DELETE /users/{id}: Eliminar usuario ---
        Response<Void> responseDelete = service.deleteUser(userId).execute();

    }
}
```

5. Operaciones CRUD Explicadas

¿Qué es CRUD?

CRUD son las cuatro operaciones básicas de cualquier sistema de almacenamiento:

OPERACIÓN	HTTP	DESCRIPCIÓN	EJEMPLO
Create	POST	Crear nuevo registro	Crear un usuario
Read	GET	Leer/Obtener registros	Ver lista de usuarios
Update	PUT/PATCH	Actualizar registro	Cambiar email de usuario
Delete	DELETE	Eliminar registro	Borrar usuario

Diferencia entre PUT y PATCH

JAVASCRIPT

```
Supongamos que el usuario original es:
{
  "id": "123",
  "name": "Juan",
  "email": "juan@mail.com",
  "phone": "123456",
  "isActive": true
}
```

```
=== PUT /users/123 ===
Envías:
{ "name": "Pedro", "email": "pedro@mail.com" }

Resultado: El usuario queda con SOLO name y email,
todos los demás campos se borran (nulos/false).

=== PATCH /users/123 ===
Envías:
{ "name": "Pedro", "email": "pedro@mail.com" }

Resultado: Solo se actualizan name y email,
los demás campos permanecen igual.
```

7. Ejemplo de JSON y Conversión

JSON que devuelve la API:

JSON

```json
[
  {
    "id": "4d529d30-a87e-4b67-b858-fc54055bc7e1",
    "name": "Juan Pérez",
    "email": "juan@ejemplo.com",
    "phone": "123456789",
    "address": {
      "street": "623 Jailyn Village",
      "city": "Pasadena",
      "country": "IA 54138"
    },
    "birthdate": "1990-05-15",
    "isActive": true
  }
]
```

Conversión a Objetos Java:

BASH

```
JSON Array []          '   Java List<>
   JSON Object {}    '   Java User
        id           '   user.setId("4d529d30-...")
        name         '   user.setName("Juan Pérez")
        email        '   user.setEmail("juan@ejemplo.com")
        phone        '   user.setPhone("123456789")
        address      '   user.setAddress(Address)
             street '   address.setStreet("623 Jailyn Village")
             city   '   address.setCity("Pasadena")
             country ' address.setCountry("IA 54138")
        birthdate    '   user.setBirthdate("1990-05-15")
        isActive     '   user.setActive(true)
```

8. Comandos Útiles de Maven

COMANDO	DESCRIPCIÓN
`mvn compile`	Compila el proyecto
`mvn exec:java -Dexec.mainClass="com.example.Main"`	Ejecuta el programa
`mvn clean`	Limpia archivos compilados
`mvn dependency:resolve`	Descarga las dependencias
`mvn dependency:tree`	Muestra el árbol de dependencias
`mvn package`	Crea un archivo JAR

10. Requisitos Previos

Para ejecutar este proyecto necesitas:

1. **Java Development Kit (JDK) 17 o superior**
2. Descargar: https://adoptium.net/
3. Verificar instalación: `java -version`

1. **Maven** (opcional, si no usas VS Code)
2. Descargar: https://maven.apache.org/download.cgi
3. Verificar instalación: `mvn -version`

1. **Conexión a Internet**
2. Necesaria para descargar las dependencias
3. Necesaria para hacer la petición a la API

11. Conceptos Clave Resumidos

CONCEPTO	EXPLICACIÓN SIMPLE
API REST	Un servicio web que sigue reglas específicas para comunicarse
HTTP	El protocolo que usa la web para transferir datos
GET	Solicitar datos (como leer una página web)
POST	Crear nuevos datos en el servidor
PUT	Reemplazar un registro completamente
PATCH	Actualizar solo algunos campos de un registro
DELETE	Eliminar un registro
JSON	Formato de texto para intercambiar datos

Retrofit	Librería que facilita hacer peticiones HTTP desde Java
Gson	Librería que convierte JSON a objetos Java
Maven	Herramienta que gestiona dependencias y compilación
Dependencia	Librería externa que tu proyecto necesita
CRUD	Create, Read, Update, Delete - operaciones básicas
UUID	Identificador único universal (ej: "4d529d30-a87e-4b67-b858-fc54055bc7e1")

Proyecto Demo2: Juego 2D con FXGL

Este proyecto es un ejemplo básico de cómo crear un juego 2D simple usando **FXGL** (un framework de juegos basado en JavaFX). El programa muestra una ventana con un personaje cuadrado de color azul que puede moverse con las teclas WASD.

1. Estructura del Proyecto

BASH

```
demo2/
 pom.xml                                # Configuración de Maven (dependencias)
 README.md                              # Esta documentación
 src/
     main/
         java
             com
                 example
                     Main.java          # Punto de entrada del programa
                     MiJuego.java       # Clase principal del juego (extiende GameApplication)
```

2. ¿Qué es Maven y cómo funciona?

Maven es una herramienta que gestiona:

- **Dependencias**: Librerías externas que necesita tu proyecto (como FXGL)
- **Compilación**: Convierte tu código Java en un programa ejecutable
- **Ejecución**: Permite correr tu programa fácilmente

El archivo `pom.xml`

`pom.xml` es el archivo de configuración de Maven. Aquí se definen todas las dependencias:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

```

```
    <!-- Identificación del proyecto -->
    <groupId>com.example</groupId>       <!-- Nombre de la organización -->
    <artifactId>demo2</artifactId>       <!-- Nombre del proyecto -->
    <version>1.0-SNAPSHOT</version>     <!-- Versión -->

    <!-- Versión de Java que usamos -->
    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
    </properties>

    <!-- Lista de dependencias (librerías externas) -->
    <dependencies>
        <!-- FXGL: Framework de juegos 2D basado en JavaFX -->
        <dependency>
            <groupId>com.github.almasb</groupId>
            <artifactId>fxgl</artifactId>
            <version>23.3</version>
        </dependency>
    </dependencies>
</project>
```

3. Instalación de Dependencias

Opción A: Desde Visual Studio Code

1. Instala la extensión **"Extension Pack for Java"** de Microsoft
2. Instala la extensión **"Maven for Java"**
3. Al abrir el proyecto, VS Code detectará automáticamente el `pom.xml`
4. Las dependencias se descargarán automáticamente

Opción B: Desde Línea de Comandos

BASH

```bash
# Navega a la carpeta del proyecto
cd C:\Users\jhonf\Desktop\Ejercicios\demo2

# Descarga todas las dependencias
mvn dependency:resolve

# Compila el proyecto
mvn compile

# Ejecuta el programa
mvn exec:java -Dexec.mainClass="com.example.Main"
```

4. Explicación Detallada de Cada Archivo

4.1. `Main.java` - Punto de Entrada

JAVA

```java
package com.example;
```

```
import com.almasb.fxgl.app.GameApplication;

public class Main {
    public static void main(String[] args) {
        // Este método estático de GameApplication busca la clase que
        // extiende de ella y arranca el ciclo de vida del juego.
        GameApplication.launch(MiJuego.class, args);
    }
}
```

¿Qué hace este archivo?

- Es el punto de entrada del programa
- Llama a `GameApplication.launch()` pasando la clase `MiJuego` como parámetro
- FXGL internamente busca una clase que extienda `GameApplication` y ejecuta su ciclo de vida

---

4.2. `MiJuego.java` - Clase Principal del Juego

JAVA

```java
package com.example;

import com.almasb.fxgl.app.GameApplication;
import com.almasb.fxgl.app.GameSettings;
import com.almasb.fxgl.entity.Entity;
import javafx.scene.input.KeyCode;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;

import static com.almasb.fxgl.dsl.FXGL.*;

public class MiJuego extends GameApplication {

    private Entity jugador;

    @Override
    protected void initSettings(GameSettings settings) {
        settings.setWidth(800);          // Ancho de la ventana
        settings.setHeight(600);          // Alto de la ventana
        settings.setTitle("FXGL Separado"); // Título de la ventana
        settings.setVersion("1.0");       // Versión del juego
    }

    @Override
    protected void initGame() {
        // Fondo simple
        getGameScene().setBackgroundColor(Color.LIGHTGRAY);

        // Jugador
        jugador = entityBuilder()
                .at(100, 100)
                .viewWithBBox(new Rectangle(40, 40, Color.BLUE))
                .buildAndAttach();

        // Texto de ayuda
        addUINode(new Text("Usa WASD para moverte"), 20, 30);
    }

    @Override
```

```
41      protected void initInput() {
42          onKey(KeyCode.W, () -> jugador.translateY(-5));
43          onKey(KeyCode.S, () -> jugador.translateY(5));
44          onKey(KeyCode.A, () -> jugador.translateX(-5));
45          onKey(KeyCode.D, () -> jugador.translateX(5));
46      }
47  }
```

¿Qué hace este archivo?

- Extiende `GameApplication` que proporciona el ciclo de vida del juego
- Define la configuración de la ventana (ancho, alto, título)
- Crea un jugador (rectángulo azul) en una posición inicial
- Configura los controles de teclado (WASD para mover)

---

5. Métodos del Ciclo de Vida de FXGL

FXGL proporciona varios métodos que puedes sobrescribir para controlar el juego:

MÉTODO	DESCRIPCIÓN
`initSettings(GameSettings settings)`	Configura las propiedades de la ventana (tamaño, título)
`initGame()`	Se ejecuta una vez al iniciar el juego. Aquí se crean las entidades
`initInput()`	Configura los controles de teclado y mouse
`initPhysics()`	Configura las propiedades de la física (gravedad, colisiones)
`onUpdate(long t)`	Se ejecuta constantemente (60 veces por segundo). Ideal para lógica del juego

---

6. Sistema de Entidades en FXGL

En FXGL, todo en el juego es una **Entity** (entidad). Las entidades tienen:

- **Position** (posición):Dónde está en la pantalla
- **View** (vista):Lo que se ve (imagen, forma, texto)
- **Bounding Box** (caja de colisión):Para detectar colisiones

Crear una Entidad

JAVA

```
1  jugador = entityBuilder()
2      .at(100, 100)                                  // Posición (x, y)
3      .viewWithBBox(new Rectangle(40, 40, Color.BLUE)) // Visual + caja de colisión
4      .buildAndAttach();                             // Crear y añadir al juego
```

---

7. Sistema de Entrada (Controles)

Eventos de Teclado

```java
onKey(KeyCode.W, () -> jugador.translateY(-5));  // Al presionar W: mover arriba
onKey(KeyCode.S, () -> jugador.translateY(5));    // Al presionar S: mover abajo
onKey(KeyCode.A, () -> jugador.translateX(-5));   // Al presionar A: mover izquierda
onKey(KeyCode.D, () -> jugador.translateX(5));    // Al presionar D: mover derecha
```

Métodos de Movimiento de Entity

MÉTODO	DESCRIPCIÓN
`translateX(n)`	Mueve la entidad n píxeles en el eje X
`translateY(n)`	Mueve la entidad n píxeles en el eje Y
`translate(nx, ny)`	Mueve la entidad en ambos ejes

---

9. Comandos Útiles de Maven

COMANDO	DESCRIPCIÓN
`mvn compile`	Compila el proyecto
`mvn exec:java -Dexec.mainClass="com.example.Main"`	Ejecuta el programa
`mvn clean`	Limpia archivos compilados
`mvn dependency:resolve`	Descarga las dependencias
`mvn dependency:tree`	Muestra el árbol de dependencias
`mvn package`	Crea un archivo JAR

---

11. Requisitos Previos

Para ejecutar este proyecto necesitas:

1. **Java Development Kit (JDK) 21 o superior**
2. Descargar: https://adoptium.net/
3. Verificar instalación: `java -version`

1. **Maven** (opcional, si no usas VS Code)
2. Descargar: https://maven.apache.org/download.cgi

3. Verificar instalación: `mvn -version`

---

12. Próximos Pasos para Extender el Juego

Una vez que entiendas este ejemplo básico, puedes:

1. **Añadir sprites**: Reemplazar el `Rectangle` por imágenes
2. **Añadir física**: Usar `initPhysics()` para gravedad y colisiones
3. **Añadir enemigos**: Crear más entidades con IA básica
4. **Añadir puntuación**: Usar `FXGL.incScore(n)` y mostrar en pantalla
5. **Añadir sonidos**: Usar `FXGL.play("sound.wav")`
6. **Crear niveles**: Gestionar múltiples `GameScene` o usar sistemas de escenas de FXGL

---

13. Conceptos Clave Resumidos

CONCEPTO	EXPLICACIÓN SIMPLE
FXGL	Framework de juegos 2D gratuito para Java
GameApplication	Clase base que proporciona el ciclo de vida del juego
Entity	Objeto en el juego (jugador, enemigo, obstáculo)
GameSettings	Configuración de la ventana del juego
Bounding Box	Caja de colisión para detectar cuándo dos entidades se tocan
translateX/Y	Mover una entidad en los ejes X o Y
onKey()	Evento que se ejecuta al presionar una tecla
Maven	Herramienta que gestiona dependencias y compilación
Dependencia	Librería externa que tu proyecto necesita
JavaFX	Biblioteca gráfica de Java que FXGL utiliza internamente

---

Proyecto Demo3: Generador de PDF con OpenPDF

Este proyecto es un ejemplo básico de cómo crear documentos PDF usando **OpenPDF** (una librería Java de código abierto). El programa genera un archivo PDF simple con un título y datos de ejemplo.

---

1. Estructura del Proyecto

BASH

```
demo3/
 pom.xml                  # Configuración de Maven (dependencias)
 README.md                # Esta documentación
```

```
src/
    main/
        java
            com
                example
                    Main.java      # Genera un PDF simple
```

2. ¿Qué es Maven y cómo funciona?

Maven es una herramienta que gestiona:

- **Dependencias**: Librerías externas que necesita tu proyecto (como OpenPDF)
- **Compilación**: Convierte tu código Java en un programa ejecutable
- **Ejecución**: Permite correr tu programa fácilmente

El archivo `pom.xml`

`pom.xml` es el archivo de configuración de Maven:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <!-- Identificación del proyecto -->
    <groupId>com.example</groupId>
    <artifactId>demo3</artifactId>
    <version>1.0-SNAPSHOT</version>

    <!-- Versión de Java -->
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <!-- Dependencias -->
    <dependencies>
        <!-- OpenPDF: Librería para crear PDFs -->
        <dependency>
            <groupId>com.github.librepdf</groupId>
            <artifactId>openpdf</artifactId>
            <version>1.3.35</version>
        </dependency>
    </dependencies>
</project>
```

3. Instalación y Ejecución

Opción A: Desde Visual Studio Code

1. Instala la extensión **"Extension Pack for Java"** de Microsoft
2. Instala la extensión **"Maven for Java"**

3. Al abrir el proyecto, VS Code detectará automáticamente el `pom.xml`
4. Las dependencias se descargarán automáticamente

Opción B: Desde Línea de Comandos

BASH

```bash
# Navega a la carpeta del proyecto
cd C:\Users\jhonf\Desktop\Ejercicios\demo3

# Descarga todas las dependencias
mvn dependency:resolve

# Compila el proyecto
mvn compile

# Ejecuta el programa
mvn exec:java -Dexec.mainClass="com.example.Main"
```

4. Explicación del Archivo Main.java

JAVA

```java
package com.example;

import com.lowagie.text.Document;          // Documento PDF
import com.lowagie.text.FontFactory;      // Factory para crear fuentes
import com.lowagie.text.pdf.PdfWriter;   // Escritor de PDF
import java.io.FileOutputStream;          // Salida a archivo

public class Main {
    public static void main(String[] args) throws Exception {
        // 1. Crear un documento PDF
        var doc = new Document();

        // 2. Crear el escritor que genera el archivo
        PdfWriter.getInstance(doc, new FileOutputStream("reporte.pdf"));

        // 3. Abrir el documento para agregar contenido
        doc.open();

        // 4. Agregar título
        var titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 18);
        doc.add(new com.lowagie.text.Paragraph("Reporte de Sistema", titleFont));
        doc.add(new com.lowagie.text.Paragraph(" "));  // Espacio vacío

        // 5. Agregar datos
        var normalFont = FontFactory.getFont(FontFactory.HELVETICA, 12);
        doc.add(new com.lowagie.text.Paragraph("Estado: Operativo", normalFont));
        doc.add(new com.lowagie.text.Paragraph("Sector: Java Backend", normalFont));

        // 6. Cerrar el documento (genera el archivo)
        doc.close();

        System.out.println("PDF generado: reporte.pdf");
    }
}
```

5. Conceptos Clave de OpenPDF

5.1. Document

El `Document` es el contenedor del PDF. Debes:

1. **Crearlo**: `new Document()`
2. **Abrirlo**: `doc.open()` antes de agregar contenido
3. **Cerrarlo**: `doc.close()` al terminar

5.2. Paragraph

Un `Paragraph` representa un bloque de texto. Puede contener:

- Texto plano
- Fuente personalizada
- Espaciado

```java
doc.add(new Paragraph("Texto"));             // Texto simple
doc.add(new Paragraph(" ", font));          // Espacio (saltos de línea)
```

5.3. FontFactory

Permite crear fuentes con diferentes estilos:

CONSTANTE	DESCRIPCIÓN
`FontFactory.HELVETICA`	Fuente sans-serif
`FontFactory.HELVETICA_BOLD`	Sans-serif bold
`FontFactory.TIMES_ROMAN`	Fuente serif
`FontFactory.COURIER`	Fuente monoespaciada

Parámetros: `FontFactory.getFont(nombre, tamaño)`

5.4. PdfWriter

`PdfWriter` conecta el `Document` con un archivo de salida:

```java
PdfWriter.getInstance(doc, new FileOutputStream("reporte.pdf"));
```

7. Comandos Útiles de Maven

COMANDO	DESCRIPCIÓN
`mvn compile`	Compila el proyecto
`mvn exec:java -Dexec.mainClass="com.example.Main"`	Ejecuta el programa
`mvn clean`	Limpia archivos compilados
`mvn dependency:resolve`	Descarga las dependencias
`mvn dependency:tree`	Muestra el árbol de dependencias
`mvn package`	Crea un archivo JAR

9. Requisitos Previos

Para ejecutar este proyecto necesitas:

1. **Java Development Kit (JDK) 17 o superior**
2. Descargar: https://adoptium.net/
3. Verificar instalación: `java -version`

1. **Maven** (opcional, si no usas VS Code)
2. Descargar: https://maven.apache.org/download.cgi
3. Verificar instalación: `mvn -version`

10. Elementos Disponibles en OpenPDF

OpenPDF ofrece muchos más elementos para crear PDFs complejos:

ELEMENTO	DESCRIPCIÓN
`Paragraph`	Bloque de texto
`Table`	Tabla con filas y celdas
`Image`	Imagen insertada
`List`	Lista con viñetas
`Chapter` / `Section`	Estructura jerárquica
`Anchor`	Hipervínculo
`Barcode`	Código de barras

11. Próximos Pasos para Extender

Una vez que entiendas este ejemplo básico, puedes:

1. **Añadir tablas**: Usar `Table` para datos estructurados
2. **Añadir imágenes**: Usar `Image.getInstance(path)`
3. **Añadir formato**: Combinas fuentes, colores y tamaños
4. **Añadir páginas**: `doc.newPage()` para múltiples páginas
5. **Añadir encabezados/pie de página**: Usar `PdfPageEvent`
6. **Crear facturas o reportes**: Combinar tablas, imágenes y párrafos

---

12. Conceptos Clave Resumidos

CONCEPTO	EXPLICACIÓN SIMPLE
OpenPDF	Librería Java para crear archivos PDF
Document	Contenedor del PDF que se genera
Paragraph	Bloque de texto en el PDF
FontFactory	Factory para crear fuentes (tipo, tamaño, estilo)
PdfWriter	Conecta el Document con un archivo de salida
Maven	Herramienta que gestiona dependencias y compilación
Dependencia	Librería externa que el proyecto necesita

UNIDAD TEMÁTICA • CAPÍTULO 04

Proyecto Integrador

LECCIONES COMPRENDIDAS EN ESTE MÓDULO:

- **4.1:** 00_INDICE_Y_ARQUITECTURA
- **4.2:** 01_CONFIGURACION_INICIAL_Y_POM
- **4.3:** 02_CONEXION_POSTGRESQL_Y_PROPERTIES
- **4.4:** 03_ENTIDADES_MODELO_JPA
- **4.5:** 04_SERVICIO_CRUD_CON_HIBERNATE
- **4.6:** 05_INTERFAZ_GRAFICA_MAINVIEW
- **4.7:** 06_EJECUCION_Y_PRUEBAS
- **4.8:** 07_GUIA_ADAPTACION_NUEVAS_IDEAS_DE_NEGOCIO

Proyecto Integrador

Escribe aquí tu contenido...

CAPÍTULO 4 • LECCIÓN 4.1

00_INDICE_Y_ARQUITECTURA

Tutorial: Sistema de Inventario desde Cero

REPOSITORIO PROYECTO INTEGRADOR
Descripción del sitio *Enlace:* https://github.com/jfinforecursos/backend1-proyecto-integrador.git

Guía Completa de Desarrollo con Spring Boot, Vaadin Flow, Hibernate ORM y PostgreSQL

Bienvenido a esta guía paso a paso. En este tutorial aprenderás a construir desde cero una aplicación web empresarial completa para un **Sistema Básico de Inventario**.

El proyecto utiliza un enfoque moderno, limpio y sin complicaciones:

- **Frontend**: **Vaadin Flow** (permite construir interfaces web interactivas y reactivas 100% en Java, sin necesidad de escribir HTML, JavaScript o CSS).
- **Backend**: **Spring Boot**, el framework estándar de la industria para desarrollo en Java.
- **ORM (Capa de Persistencia)**: **Hibernate ORM** (mediante Jakarta Persistence / `EntityManager`), interactuando directamente con la base de datos sin necesidad de crear interfaces de repositorios repetitivas.
- **Base de Datos**: **PostgreSQL**, un motor de base de datos relacional potente y robusto.

---

1. Arquitectura General del Sistema

A diferencia de las arquitecturas tradicionales que requieren decenas de clases y carpetas intermedias, este proyecto utiliza una **arquitectura simplificada y directa**:

JAVASCRIPT

```
+-------------------------------------------------------------+
|                    NAVEGADOR WEB (CLIENTE)                  |
|                  http://localhost:8080                      |
+-------------------------------------------------------------+
                              ²
                               Comunicación Web (Vaadin Flow)
                              ¼
+-------------------------------------------------------------+
|                     CAPA DE VISTA (UI)                      |
|                       MainView.java                         |
|   - Pestañas (TabSheet)                                     |
|   - Formularios (FormLayout) y Grillas de Datos (Grid)      |
|   - Eventos de botones (Crear, Consultar, Actualizar, etc.) |
+-------------------------------------------------------------+
                              ²
                               Inyección de dependencias (@Autowired)
                              ¼
+-------------------------------------------------------------+
|                  CAPA DE SERVICIO GENÉRICO                  |
|                       CrudService.java                      |
|   - EntityManager de Hibernate directo                      |
|   - Métodos CRUD universales: persist, find, merge, remove  |
```

```
+-------------------------------------------------------------+
                              ²
                               Mapeo Objeto-Relacional (JPA)
                              ¼
+-------------------------------------------------------------+
|                    MODELO DE ENTIDADES (JPA)                |
|    - Producto.java     -> Tabla "productos"                 |
|    - Categoria.java    -> Tabla "categorias"                |
|    - Proveedor.java    -> Tabla "proveedores"               |
|    - Usuario.java      -> Tabla "usuarios"                  |
+-------------------------------------------------------------+
                              ²
                               Conector JDBC (Driver de PostgreSQL)
                              ¼
+-------------------------------------------------------------+
|                 BASE DE DATOS POSTGRESQL                    |
|                      inventario_db                          |
+-------------------------------------------------------------+
```

2. Requisitos Previos e Instalación de Herramientas

Para seguir este tutorial y ejecutar el proyecto en tu computadora, asegúrate de tener instaladas las siguientes herramientas:

1. **Java Development Kit (JDK)**:
2. Versión 21 o superior (este proyecto está configurado para Java 25 o 21 LTS).
3. Puedes descargarlo de **Adoptium Eclipse Temurin** o de Oracle.
4. Para verificar en tu terminal:

BASH

```bash
    java -version
```

1. **Motor de Base de Datos PostgreSQL**:
2. Descarga e instala PostgreSQL desde su sitio oficial **postgresql.org**.
3. Durante la instalación, toma nota del puerto (por defecto `5432`) y de la contraseña del usuario `postgres`.

1. **Entorno de Desarrollo (IDE)**:
2. Puedes usar Visual Studio Code, IntelliJ IDEA, Eclipse o NetBeans.

1. **Apache Maven**:
2. El proyecto incluye los archivos ejecutables `mvnw` (Linux/Mac) y `mvnw.cmd` (Windows), por lo que **no es obligatorio** tener Maven instalado globalmente; el wrapper se encarga de todo.

3. Estructura de Directorios del Proyecto

Al finalizar el tutorial, la estructura de archivos de tu proyecto será exactamente la siguiente:

TEXT

```
proyecto-integrador-b1-2026-2/
```

```
2  |
3  |  pom.xml                                # Dependencias y configuración de Maven
4  |  mvnw / mvnw.cmd                        # Maven Wrapper para compilar sin instalar Maven
5  |
6  |  src/
7  |     main/
8  |        java/
9  |           com/
10 |               example/
11 |                   Application.java   # Clase principal para iniciar Spring Boot y Vaadin
12 |                   MainView.java      # Vista web con formularios y grillas CRUD
13 |
14 |                   model/             # Entidades del dominio mapeadas con JPA
15 |                      Categoria.java
16 |                      Producto.java
17 |                      Proveedor.java
18 |                      Usuario.java
19 |
20 |                   service/           # Capa de operaciones CRUD con Hibernate
21 |                       CrudService.java
22 |
23 |        resources/
24 |            application.properties     # Configuración de base de datos y Hibernate
25 |            META-INF/resources/styles.css # Estilos opcionales
26 |
27 |  docs/                                  # Manuales y tutoriales paso a paso
28 |      00_INDICE_Y_ARQUITECTURA.md
29 |      01_CONFIGURACION_INICIAL_Y_POM.md
30 |      02_CONEXION_POSTGRESQL_Y_PROPERTIES.md
31 |      03_ENTIDADES_MODELO_JPA.md
32 |      04_SERVICIO_CRUD_CON_HIBERNATE.md
33 |      05_INTERFAZ_GRAFICA_MAINVIEW.md
34 |      06_EJECUCION_Y_PRUEBAS.md
35 |      07_GUIA_ADAPTACION_NUEVAS_IDEAS_DE_NEGOCIO.md
```

4. Índice del Tutorial

Sigue los capítulos en este orden para comprender cada parte del sistema:

1. **Capítulo 1: Configuración Inicial y Archivo pom.xml**
2. Creación del esqueleto del proyecto y explicación de dependencias.
3. **Capítulo 2: Conexión con PostgreSQL y application.properties**
4. Creación de la base de datos y configuración del ORM Hibernate.
5. **Capítulo 3: Entidades del Modelo JPA**
6. Creación y anotación de Producto, Categoría, Proveedor y Usuario.
7. **Capítulo 4: Servicio CRUD Directo con Hibernate**
8. Creación de `CrudService` universal con `EntityManager` sin repositorios.
9. **Capítulo 5: Interfaz de Usuario con Vaadin Flow**
10. Construcción de la vista interactiva `MainView.java`.
11. **Capítulo 6: Ejecución, Pruebas y Validación**
12. Puesta en marcha, pruebas funcionales y resolución de incidencias.
13. **Capítulo 7: Guía de Adaptación a Nuevas Ideas de Negocio**
14. Tutorial para estudiantes sobre cómo convertir este código en cualquier proyecto integrador (veterinaria, gimnasio, hotel, etc.).

CAPÍTULO 4 • LECCIÓN 4.2

01_CONFIGURACION_INICIAL_Y_POM

Capítulo 1: Configuración Inicial y Archivo pom.xml

En este capítulo aprenderás cómo inicializar el proyecto base y cómo configurar todas las librerías necesarias utilizando Apache Maven.

---

1. ¿Cómo crear el proyecto desde cero?

Existen dos formas sencillas de generar la estructura base:

Método A: Usando Vaadin Start (Recomendado para proyectos Vaadin)

1. Ingresa en tu navegador a **https://start.vaadin.com**.
2. Selecciona **Flow (Java only)** como framework web.
3. Elige el tema visual por defecto (**Aura** o **Lumo**).
4. Descarga el archivo `.zip`, descomprímelo en tu carpeta de proyectos y ábrelo en tu IDE.

Método B: Usando Spring Initializr

1. Ingresa a **https://start.spring.io**.
2. Configura:
3. **Project**: Maven
4. **Language**: Java
5. **Spring Boot**: Versión 3.x o 4.x
6. **Group**: `com.example`
7. **Artifact**: `app`
8. **Dependencies**: Vaadin, Spring Data JPA, PostgreSQL Driver.
9. Genera y descarga el archivo comprimido.

---

2. El archivo de configuración `pom.xml`

El archivo `pom.xml` (Project Object Model) es el corazón de cualquier proyecto Maven. En él se especifican las versiones de Java, las dependencias externas (librerías) y los plugins de compilación.

A continuación, se muestra el archivo `pom.xml` completo del proyecto:

XML

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="-
http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
```

```
6  |
7  |     <groupId>com.example</groupId>
8  |     <artifactId>app</artifactId>
9  |     <version>1.0-SNAPSHOT</version>
10 |     <packaging>jar</packaging>
11 |
12 |     <properties>
13 |         <java.version>25</java.version>
14 |         <vaadin.version>25.3.0</vaadin.version>
15 |     </properties>
16 |
17 |     <!-- Heredamos de Spring Boot Starter Parent para control de versiones -->
18 |     <parent>
19 |         <groupId>org.springframework.boot</groupId>
20 |         <artifactId>spring-boot-starter-parent</artifactId>
21 |         <version>4.1.1</version>
22 |         <relativePath/>
23 |     </parent>
24 |
25 |     <!-- Gestión centralizada de versiones de Vaadin mediante su BOM -->
26 |     <dependencyManagement>
27 |         <dependencies>
28 |             <dependency>
29 |                 <groupId>com.vaadin</groupId>
30 |                 <artifactId>vaadin-bom</artifactId>
31 |                 <version>${vaadin.version}</version>
32 |                 <type>pom</type>
33 |                 <scope>import</scope>
34 |             </dependency>
35 |         </dependencies>
36 |     </dependencyManagement>
37 |
38 |     <dependencies>
39 |         <!-- Componentes visuales de Vaadin -->
40 |         <dependency>
41 |             <groupId>com.vaadin</groupId>
42 |             <artifactId>vaadin</artifactId>
43 |         </dependency>
44 |
45 |         <!-- Herramientas de desarrollo en caliente (hot reload) de Vaadin -->
46 |         <dependency>
47 |             <groupId>com.vaadin</groupId>
48 |             <artifactId>vaadin-dev</artifactId>
49 |             <optional>true</optional>
50 |         </dependency>
51 |
52 |         <!-- Integración oficial entre Vaadin y Spring Boot -->
53 |         <dependency>
54 |             <groupId>com.vaadin</groupId>
55 |             <artifactId>vaadin-spring-boot-starter</artifactId>
56 |         </dependency>
57 |
58 |         <!-- ORM Hibernate y persistencia JPA mediante Spring Boot -->
59 |         <dependency>
60 |             <groupId>org.springframework.boot</groupId>
61 |             <artifactId>spring-boot-starter-data-jpa</artifactId>
62 |         </dependency>
63 |
64 |         <!-- Conector JDBC oficial para PostgreSQL -->
65 |         <dependency>
66 |             <groupId>org.postgresql</groupId>
67 |             <artifactId>postgresql</artifactId>
68 |             <scope>runtime</scope>
69 |         </dependency>
70 |
```

```
        <!-- Dependencias para pruebas unitarias -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.vaadin</groupId>
            <artifactId>browserless-test-spring</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <defaultGoal>spring-boot:run</defaultGoal>
        <plugins>
            <!-- Plugin para empaquetar y ejecutar aplicaciones Spring Boot -->
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>

            <!-- Plugin de Vaadin para compilar recursos frontend en producción -->
            <plugin>
                <groupId>com.vaadin</groupId>
                <artifactId>vaadin-maven-plugin</artifactId>
                <version>${vaadin.version}</version>
                <executions>
                    <execution>
                        <goals>
                            <goal>build-frontend</goal>
                        </goals>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

3. Explicación Detallada de cada Dependencia

1. **`spring-boot-starter-parent`**:

Define la configuración base de Spring Boot y gestiona automáticamente las versiones compatibles de la mayoría de librerías de Java para evitar conflictos de versiones.

1. **`vaadin-bom` y `vaadin`**:
2. `vaadin-bom` (Bill of Materials) fija la versión exacta de todos los módulos de Vaadin.
3. `vaadin` incluye toda la suite de componentes web (botones, formularios, grillas, diálogos, notificaciones, pestañas, etc.).

1. **`vaadin-spring-boot-starter`**:

Permite que Spring Boot reconozca automáticamente las vistas de Vaadin (`@Route`), habilita la inyección de dependencias (`@Autowired`) dentro de los componentes visuales y arranca el servidor embebido Tomcat.

1. **`spring-boot-starter-data-jpa`**:

Descarga e integra automáticamente:

- **Hibernate ORM**: el motor que traduce los objetos Java a consultas SQL.
- **Jakarta Persistence API (JPA)**: la interfaz estándar para interactuar con bases de datos relacionales en Java (`EntityManager`).
- **HikariCP**: el gestor de grupo de conexiones (connection pool) de alto rendimiento.

1. **`org.postgresql:postgresql`**:

Es el controlador (JDBC Driver) que permite a Java hablar el protocolo de comunicación nativo de PostgreSQL. Tiene `<scope>runtime</scope>` porque solo se necesita cuando el programa se ejecuta y conecta a la base de datos.

---

4. La Clase Principal: `Application.java`

Ubicada en: `src/main/java/com/example/Application.java`.

Es el punto de entrada de la aplicación (`public static void main`). Spring Boot la utiliza para inicializar el contenedor de servlets, escanear componentes y levantar el servidor web:

JAVA

```java
package com.example;

import com.vaadin.flow.component.dependency.StyleSheet;
import com.vaadin.flow.component.page.AppShellConfigurator;
import com.vaadin.flow.component.page.Push;
import com.vaadin.flow.theme.aura.Aura;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@StyleSheet(Aura.STYLESHEET)     // Aplica el tema visual Aura de Vaadin
@StyleSheet("styles.css")         // Carga una hoja de estilos CSS personalizada
@Push                             // Habilita comunicación bidireccional en tiempo real (WebSockets)
public class Application implements AppShellConfigurator {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Elementos clave de esta clase:

- **`@SpringBootApplication`**: Indica a Spring que esta clase es el arranque, activando la autoconfiguración, el escaneo de componentes y la configuración de propiedades.
- **`implements AppShellConfigurator`**: Permite a Vaadin configurar el caparazón HTML de la página web (meta tags, viewport para móviles, título, etc.).
- **`@Push`**: Permite actualizaciones instantáneas en el navegador impulsadas desde el servidor mediante WebSockets.

CAPÍTULO 4 • LECCIÓN 4.3

02_CONEXION_POSTGRESQL_Y_PROPERTIES

Capítulo 2: Conexión con PostgreSQL y application.properties

En este capítulo aprenderás cómo crear la base de datos en PostgreSQL y cómo configurar el archivo `application.properties` para que Spring Boot y Hibernate se conecten automáticamente.

1. Crear la Base de Datos en PostgreSQL

Antes de iniciar la aplicación por primera vez, es necesario crear una base de datos vacía en PostgreSQL. Puedes hacerlo usando la herramienta que prefieras:

Opción A: Usando la terminal (psql)

Abre tu consola o terminal y ejecuta:

BASH

```bash
# Iniciar sesión en PostgreSQL con el usuario administrador
psql -U postgres

# Dentro de la consola psql, crear la base de datos:
CREATE DATABASE inventario_db;

# Salir de psql
\q
```

Opción B: Usando pgAdmin o DBeaver

1. Abre pgAdmin o DBeaver y conéctate a tu servidor PostgreSQL local (`localhost:5432`).
2. Haz clic derecho sobre **Databases** (Bases de datos).
3. Selecciona **Create** > **Database...**.
4. Escribe el nombre: `inventario_db`.
5. Haz clic en **Save** (Guardar).

> **NOTA CLAVE**
> **NOTE** No es necesario crear tablas manualmente. Hibernate las creará por nosotros de forma automática al iniciar el proyecto.

2. El archivo de configuración `application.properties`

Ubicado en: `src/main/resources/application.properties`.

Este archivo contiene los parámetros del servidor web, las credenciales de la base de datos y el comportamiento del motor ORM Hibernate.

A continuación, se muestra el contenido completo:

PROPERTIES

```properties
# ========================================
# 1. Configuración del Servidor y Vaadin
# ========================================
# Puerto HTTP en el que corre la aplicación (8080 por defecto)
server.port=${PORT:8080}

# Reduce mensajes repetitivos en los logs de WebSocket
logging.level.org.atmosphere=warn

# Abre automáticamente el navegador predeterminado al iniciar en desarrollo
vaadin.launch-browser=true

# Paquetes permitidos para escaneo rápido de Vaadin
vaadin.allowed-packages=com.vaadin,org.vaadin,com.flowingcode,com.example

# ========================================
# 2. Conexión a la Base de Datos PostgreSQL
# ========================================
# URL de conexión JDBC hacia PostgreSQL (base de datos inventario_db)
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/inventario_db}

# Usuario de la base de datos (por defecto 'postgres')
spring.datasource.username=${DB_USERNAME:postgres}

# Contraseña del usuario en PostgreSQL
spring.datasource.password=${DB_PASSWORD:postgres}

# Clase del driver JDBC oficial de PostgreSQL
spring.datasource.driver-class-name=org.postgresql.Driver

# ========================================
# 3. Configuración del ORM Hibernate / JPA
# ========================================
# Generación y actualización automática del esquema en la base de datos
spring.jpa.hibernate.ddl-auto=update

# Imprime las sentencias SQL ejecutadas en la consola para depuración
spring.jpa.show-sql=true

# Formatea el SQL impreso con saltos de línea para facilitar su lectura
spring.jpa.properties.hibernate.format_sql=true

# Dialecto específico para aprovechar las funciones nativas de PostgreSQL
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

3. Explicación Detallada de cada Parámetro

Variables con Valores por Defecto (`${VARIABLE:valor_defecto}`)

En la línea:

PROPERTIES

```properties
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/inventario_db}
```

Se utiliza la sintaxis `${DB_URL:...}`. Esto significa:

- Si existe una variable de entorno llamada `DB_URL` (por ejemplo, en un contenedor Docker o en un servidor en la nube), se usará ese valor.
- Si no existe, se usará el valor por defecto: `jdbc:postgresql://localhost:5432/inventario_db`.
- Lo mismo aplica para el usuario (`${DB_USERNAME:postgres}`) y la contraseña (`${DB_PASSWORD:postgres}`).

El parámetro `spring.jpa.hibernate.ddl-auto=update` (Magia de Hibernate)

Este es uno de los parámetros más importantes del proyecto:

- **`update`**: Al arrancar la aplicación, Hibernate lee todas las clases Java que tengan la anotación `@Entity` (Producto, Categoria, Proveedor, Usuario) y compara su estructura con las tablas existentes en PostgreSQL.
- Si la tabla no existe en la base de datos, Hibernate ejecuta un `CREATE TABLE` automáticamente.
- Si agregas un nuevo campo a una clase Java (por ejemplo, `codigo`), Hibernate añade la columna a la tabla sin borrar los datos existentes.

> **ADVERTENCIA**
> **WARNING** En entornos de producción reales se suelen usar herramientas de migración como Flyway o Liquibase. Sin embargo, para desarrollo y proyectos académicos, `ddl-auto=update` es la forma más rápida y cómoda de trabajar.

`spring.jpa.show-sql=true` y `format_sql=true`

Cada vez que tu aplicación guarde un producto, consulte una categoría o elimine un usuario, Hibernate imprimirá en la consola de tu terminal la consulta SQL exacta que envió a PostgreSQL:

```sql
Hibernate:
    insert
    into
        productos
        (categoria, codigo, nombre, precio, stock)
    values
        (?, ?, ?, ?, ?)
```

Esto te permite comprobar con total transparencia que las operaciones CRUD se están ejecutando correctamente en la base de datos.

CAPÍTULO 4 • LECCIÓN 4.4

03_ENTIDADES_MODELO_JPA

Capítulo 3: Entidades del Modelo JPA

En este capítulo aprenderás qué es una Entidad en JPA, cómo mapear clases Java a tablas de PostgreSQL y conocerás el código completo y detallado de las 4 entidades del sistema.

---

1. ¿Qué es una Entidad JPA y qué hacen sus anotaciones?

Una **Entidad** es una clase simple de Java (POJO: Plain Old Java Object) que representa una fila y una tabla en la base de datos relacional.

Para que Hibernate sepa cómo transformar esa clase en una tabla SQL, usamos las anotaciones estándar de **Jakarta Persistence (JPA)**:

- **`@Entity`**: Le indica a Hibernate que esta clase debe gestionarse como una tabla de base de datos.
- **`@Table(name = "nombre_tabla")`**: Especifica el nombre exacto de la tabla en PostgreSQL (por ejemplo, `productos`).
- **`@Id`**: Indica cuál de los atributos es la **clave primaria** (Primary Key).
- **`@GeneratedValue(strategy = GenerationType.IDENTITY)`**: Le indica a PostgreSQL que genere el ID de forma incremental automáticamente (`GENERATED BY DEFAULT AS IDENTITY` o tipo `SERIAL`), de modo que no tengamos que ingresar el ID al crear nuevos registros.

---

2. Código Fuente de las Entidades

Todas las entidades están ubicadas en el paquete `com.example.model`:

2.1. Entidad `Producto.java`

Archivo: `src/main/java/com/example/model/Producto.java`.

Maneja los artículos del inventario con su código, nombre, categoría asignada, precio y existencias en stock.

JAVA

```java
package com.example.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String codigo;
    private String nombre;
    private String categoria;
    private Double precio;
    private Integer stock;

    // ====================
    // Constructores
    // ====================
    // Constructor vacío obligatorio para que Hibernate pueda instanciar la clase
    public Producto() {
    }

    // Constructor sin ID (se usa para crear productos nuevos antes de insertarlos en la BD)
    public Producto(String codigo, String nombre, String categoria, Double precio, Integer stock) {
        this(null, codigo, nombre, categoria, precio, stock);
    }

    // Constructor completo con ID
    public Producto(Long id, String codigo, String nombre, String categoria, Double precio, Integer stock-
) {
        this.id = id;
        this.codigo = codigo;
        this.nombre = nombre;
        this.categoria = categoria;
        this.precio = precio;
        this.stock = stock;
    }

    // ====================
    // Getters y Setters
    // ====================
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getCodigo() {
        return codigo;
    }

    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getCategoria() {
        return categoria;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public Double getPrecio() {
        return precio;
```

```
    }

    public void setPrecio(Double precio) {
        this.precio = precio;
    }

    public Integer getStock() {
        return stock;
    }

    public void setStock(Integer stock) {
        this.stock = stock;
    }

    @Override
    public String toString() {
        return "Producto [id=" + id + ", codigo=" + codigo + ", nombre=" + nombre + ", stock=" + stock + "]";
    }
}
```

2.2. Entidad `Categoria.java`

Archivo: `src/main/java/com/example/model/Categoria.java`.

Permite agrupar los productos por familias o tipos (por ejemplo: "Electrónica", "Papelería").

JAVA

```java
package com.example.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "categorias")
public class Categoria {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String descripcion;

    // Constructores
    public Categoria() {
    }

    public Categoria(String nombre, String descripcion) {
        this(null, nombre, descripcion);
    }

    public Categoria(Long id, String nombre, String descripcion) {
        this.id = id;
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    // Getters y Setters
```

```
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    @Override
    public String toString() {
        return "Categoria [id=" + id + ", nombre=" + nombre + "]";
    }
}
```

2.3. Entidad `Proveedor.java`

Archivo: `src/main/java/com/example/model/Proveedor.java`.

Almacena la información de los distribuidores que suministran los productos.

JAVA

```java
package com.example.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "proveedores")
public class Proveedor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String contacto;
    private String telefono;
    private String email;
    private String direccion;

    // Constructores
    public Proveedor() {
    }
```

```
25 |
26 |     public Proveedor(String nombre, String contacto, String telefono, String email, String direccion) {
27 |         this(null, nombre, contacto, telefono, email, direccion);
28 |     }
29 |
30 |     public Proveedor(Long id, String nombre, String contacto, String telefono, String email, String di-
   reccion) {
31 |         this.id = id;
32 |         this.nombre = nombre;
33 |         this.contacto = contacto;
34 |         this.telefono = telefono;
35 |         this.email = email;
36 |         this.direccion = direccion;
37 |     }
38 |
39 |     // Getters y Setters
40 |     public Long getId() {
41 |         return id;
42 |     }
43 |
44 |     public void setId(Long id) {
45 |         this.id = id;
46 |     }
47 |
48 |     public String getNombre() {
49 |         return nombre;
50 |     }
51 |
52 |     public void setNombre(String nombre) {
53 |         this.nombre = nombre;
54 |     }
55 |
56 |     public String getContacto() {
57 |         return contacto;
58 |     }
59 |
60 |     public void setContacto(String contacto) {
61 |         this.contacto = contacto;
62 |     }
63 |
64 |     public String getTelefono() {
65 |         return telefono;
66 |     }
67 |
68 |     public void setTelefono(String telefono) {
69 |         this.telefono = telefono;
70 |     }
71 |
72 |     public String getEmail() {
73 |         return email;
74 |     }
75 |
76 |     public void setEmail(String email) {
77 |         this.email = email;
78 |     }
79 |
80 |     public String getDireccion() {
81 |         return direccion;
82 |     }
83 |
84 |     public void setDireccion(String direccion) {
85 |         this.direccion = direccion;
86 |     }
87 |
88 |     @Override
89 |     public String toString() {
```

```
        return "Proveedor [id=" + id + ", nombre=" + nombre + ", contacto=" + contacto + "]";
    }
}
```

2.4. Entidad `Usuario.java`

Archivo: `src/main/java/com/example/model/Usuario.java`.

Gestiona los usuarios que operan el sistema de inventario (administradores, supervisores u operadores).

JAVA

```java
package com.example.model;

import java.time.LocalDateTime;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;
    private String direccion;
    private String rol;    // 'Administrador', 'Operador', etc.
    private String estado; // 'Activo', 'Inactivo'
    private LocalDateTime fechaRegistro;

    // Constructores
    public Usuario() {
        this.fechaRegistro = LocalDateTime.now();
    }

    public Usuario(String nombre, String apellido, String email, String telefono,
            String direccion, String rol, String estado, LocalDateTime fechaRegistro) {
        this(null, nombre, apellido, email, telefono, direccion, rol, estado, fechaRegistro);
    }

    public Usuario(Long id, String nombre, String apellido, String email, String telefono,
            String direccion, String rol, String estado, LocalDateTime fechaRegistro) {
        this.id = id;
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
        this.direccion = direccion;
        this.rol = rol;
        this.estado = estado;
        this.fechaRegistro = fechaRegistro != null ? fechaRegistro : LocalDateTime.now();
    }

    // Getters y Setters
    public Long getId() {
```

```
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public String getDireccion() {
        return direccion;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }

    public String getRol() {
        return rol;
    }

    public void setRol(String rol) {
        this.rol = rol;
    }

    public String getEstado() {
        return estado;
    }

    public void setEstado(String estado) {
        this.estado = estado;
    }

    public LocalDateTime getFechaRegistro() {
        return fechaRegistro;
```

```
116     }
117 
118     public void setFechaRegistro(LocalDateTime fechaRegistro) {
119         this.fechaRegistro = fechaRegistro;
120     }
121 
122     @Override
123     public String toString() {
124         return "Usuario [id=" + id + ", nombre=" + nombre + ", email=" + email + ", rol=" + rol + "]";
125     }
126 }
```

3. Mapeo entre Tipos de Java y Tipos de PostgreSQL

Cuando Hibernate crea las tablas, realiza la siguiente conversión automática:

TIPO EN JAVA	TIPO EN POSTGRESQL	EJEMPLO DE USO
`Long` (con `@Id`)	`bigint GENERATED BY DEFAULT AS IDENTITY`	Clave primaria incremental (`id`)
`String`	`varchar(255)`	`nombre`, `codigo`, `email`
`Double`	`float8` / `double precision`	`precio` (3500.50)
`Integer`	`integer`	`stock` (45 unidades)
`LocalDateTime`	`timestamp(6)`	`fechaRegistro` (2026-10-19 22:30:00)

CAPÍTULO 4 • LECCIÓN 4.5

04_SERVICIO_CRUD_CON_HIBERNATE

Capítulo 4: Servicio CRUD Directo con Hibernate

En este capítulo aprenderás cómo interactuar directamente con el ORM Hibernate utilizando el **`EntityManager`** de JPA sin necesidad de crear interfaces de repositorios repetitivas.

---

1. ¿Por qué usar directamente `EntityManager` sin repositorios?

Tradicionalmente, en Spring Boot se acostumbra crear una interfaz `Repository` por cada entidad:

- `ProductoRepository`
- `CategoriaRepository`
- `ProveedorRepository`
- `UsuarioRepository`

Si bien esto es útil en proyectos con consultas muy especializadas, en la gran mayoría de sistemas básicos de inventario las operaciones que se necesitan son exactamente las mismas: **Guardar, Buscar por ID, Listar todos, Actualizar y Eliminar**.

Al usar directamente el **`EntityManager`** con métodos genéricos (`<T>`), podemos crear **una sola clase** que resuelva el CRUD para todas las entidades actuales y cualquier entidad futura, reduciendo el código en más de un 70%.

---

2. Código Fuente Completo de `CrudService.java`

Ubicado en: `src/main/java/com/example/service/CrudService.java`.

JAVA

```java
package com.example.service;

import java.util.List;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * Servicio genérico que interactúa directamente con el ORM Hibernate (vía JPA EntityManager)
 * sin necesidad de crear interfaces de Repositorios para cada entidad.
 */
@Service
@Transactional
public class CrudService {

    @PersistenceContext
    private EntityManager entityManager;

    /**
     * CREATE / Guardar una nueva entidad en la base de datos
```

```
     */
    public <T> T guardar(T entidad) {
        entityManager.persist(entidad);
        return entidad;
    }

    /**
     * READ / Consultar una entidad por su ID
     */
    @Transactional(readOnly = true)
    public <T> T buscarPorId(Class<T> clase, Long id) {
        if (id == null) {
            return null;
        }
        return entityManager.find(clase, id);
    }

    /**
     * READ / Obtener todos los registros de una entidad
     */
    @Transactional(readOnly = true)
    public <T> List<T> listarTodos(Class<T> clase) {
        String hql = "SELECT e FROM " + clase.getSimpleName() + " e";
        return entityManager.createQuery(hql, clase).getResultList();
    }

    /**
     * UPDATE / Actualizar una entidad existente
     */
    public <T> T actualizar(T entidad) {
        return entityManager.merge(entidad);
    }

    /**
     * DELETE / Eliminar una entidad por su ID
     */
    public <T> boolean eliminar(Class<T> clase, Long id) {
        if (id == null) {
            return false;
        }
        T entidad = entityManager.find(clase, id);
        if (entidad != null) {
            entityManager.remove(entidad);
            return true;
        }
        return false;
    }
}
```

3. Explicación Detallada del Código

Las Anotaciones de Clase

1. **`@Service`**:

Le indica a Spring que esta clase es un componente de servicio empresarial. Spring la registrará en su contenedor de Beans y la inyectará automáticamente donde sea solicitada (por ejemplo, en `MainView`).

1. **`@Transactional`**:

Indica que cada método de la clase debe ejecutarse dentro de una **transacción de base de datos**. Si la operación tiene éxito, se ejecuta un `COMMIT`; si ocurre un error inesperado, se realiza un `ROLLBACK` automático para no corromper la información.

El objeto `EntityManager` y `@PersistenceContext`

JAVA

```java
@PersistenceContext
private EntityManager entityManager;
```

- **`EntityManager`**: Es la interfaz central de JPA / Hibernate. Proporciona los métodos nativos para interactuar con el contexto de persistencia.
- **`@PersistenceContext`**: Permite a Spring inyectar el EntityManager asociado a la conexión activa de PostgreSQL.

---

4. Desglose de cada Operación CRUD

A) Operación CREATE (`guardar`)

JAVA

```java
public <T> T guardar(T entidad) {
    entityManager.persist(entidad);
    return entidad;
}
```

- **`entityManager.persist(entidad)`**: Pasa el objeto Java de un estado "nuevo" (transient) a un estado "gestionado" (managed). Hibernate genera la instrucción SQL `INSERT INTO tabla (...) VALUES (...)`.
- Al guardarse, PostgreSQL genera el ID autoincremental y Hibernate actualiza automáticamente el campo `id` del objeto Java.

B) Operación READ por ID (`buscarPorId`)

JAVA

```java
@Transactional(readOnly = true)
public <T> T buscarPorId(Class<T> clase, Long id) {
    return entityManager.find(clase, id);
}
```

- **`entityManager.find(clase, id)`**: Ejecuta un `SELECT * FROM tabla WHERE id = ?`. Si el registro existe, Hibernate instancia la clase y la retorna. Si no existe, devuelve `null`.
- **`readOnly = true`**: Es una optimización que avisa a Hibernate que solo vamos a leer datos, evitando trabajo innecesario de verificación de cambios.

C) Operación READ de Todos los Registros (`listarTodos`)

JAVA

```java
@Transactional(readOnly = true)
public <T> List<T> listarTodos(Class<T> clase) {
```

```
    String hql = "SELECT e FROM " + clase.getSimpleName() + " e";
    return entityManager.createQuery(hql, clase).getResultList();
}
```

- Utiliza **HQL (Hibernate Query Language)** o JPQL.
- `clase.getSimpleName()` obtiene el nombre de la clase (por ejemplo, `Producto`, `Categoria`, etc.).
- Devuelve una lista tipada `List<T>` con todos los registros de la tabla.

D) Operación UPDATE (`actualizar`)

JAVA

```java
public <T> T actualizar(T entidad) {
    return entityManager.merge(entidad);
}
```

- **`entityManager.merge(entidad)`**: Toma los valores modificados del objeto Java y genera un `UPDATE tabla SET columna1 = ?, ... WHERE id = ?`.

E) Operación DELETE (`eliminar`)

JAVA

```java
public <T> boolean eliminar(Class<T> clase, Long id) {
    T entidad = entityManager.find(clase, id);
    if (entidad != null) {
        entityManager.remove(entidad);
        return true;
    }
    return false;
}
```

- Primero busca la entidad con `find()`. Si la encuentra, ejecuta **`entityManager.remove(entidad)`**, lo cual genera un `DELETE FROM tabla WHERE id = ?` en PostgreSQL y retorna `true`. Si no existe, retorna `false`.

CAPÍTULO 4 • LECCIÓN 4.6

05_INTERFAZ_GRAFICA_MAINVIEW

Capítulo 5: Interfaz de Usuario con Vaadin Flow

En este capítulo aprenderás cómo construir la interfaz gráfica de usuario completa utilizando **Vaadin Flow** en Java puro y cómo conectar los botones y las grillas directamente con nuestro `CrudService`.

---

1. ¿Qué es Vaadin Flow?

Vaadin Flow es un framework web en Java que te permite programar la interfaz de usuario en el servidor. Cada botón, campo de texto, tabla o pestaña es un objeto Java real:

- No necesitas crear archivos HTML, ni manipular el DOM con JavaScript, ni configurar llamadas manuales con `fetch` o `axios`.
- Vaadin se encarga de renderizar la interfaz en el navegador y mantenerla sincronizada automáticamente con el backend mediante WebSockets y peticiones seguras.

---

2. Componentes Visuales Utilizados

En este proyecto utilizamos los siguientes componentes estándar de Vaadin:

- **`VerticalLayout`**: Dispone los componentes uno debajo de otro.
- **`HorizontalLayout`**: Dispone los componentes uno al lado de otro (ideal para la barra de botones de acción).
- **`TabSheet`**: Contenedor visual organizado por pestañas (`Productos`, `Categorías`, `Proveedores`, `Usuarios`).
- **`FormLayout`**: Organiza los campos de entrada en un diseño responsivo que se adapta a pantallas de escritorio o móviles.
- **`TextField`**: Campo de entrada de texto (usado para ID, nombres, precios, etc.).
- **`Button`**: Botones interactivos (`Crear`, `Consultar`, `Actualizar`, `Eliminar`, `Limpiar`).
- **`Grid<T>`**: Grilla o tabla de datos de alto rendimiento, fuertemente tipada con la clase de la entidad.
- **`Notification`**: Mensajes emergentes (toasts) que informan al usuario del resultado de cada acción.

---

3. Código Fuente Completo de `MainView.java`

Ubicado en: `src/main/java/com/example/MainView.java`.

JAVA

```java
package com.example;

import java.time.LocalDateTime;

import com.example.model.Categoria;
import com.example.model.Producto;
```

```
import com.example.model.Proveedor;
import com.example.model.Usuario;
import com.example.service.CrudService;
import com.vaadin.flow.component.Component;
import com.vaadin.flow.component.button.Button;
import com.vaadin.flow.component.button.ButtonVariant;
import com.vaadin.flow.component.formlayout.FormLayout;
import com.vaadin.flow.component.grid.Grid;
import com.vaadin.flow.component.html.H2;
import com.vaadin.flow.component.notification.Notification;
import com.vaadin.flow.component.orderedlayout.HorizontalLayout;
import com.vaadin.flow.component.orderedlayout.VerticalLayout;
import com.vaadin.flow.component.tabs.TabSheet;
import com.vaadin.flow.component.textfield.TextField;
import com.vaadin.flow.router.PageTitle;
import com.vaadin.flow.router.Route;

@PageTitle("Gestión CRUD - Sistema de Inventario")
@Route("")
public class MainView extends VerticalLayout {

    private final CrudService crudService;

    // Inyección de dependencias por constructor: Spring suministra CrudService automáticamente
    public MainView(CrudService crudService) {
        this.crudService = crudService;
        setSizeFull();
        setPadding(true);
        setSpacing(true);

        TabSheet tabs = new TabSheet();
        tabs.setWidthFull();
        tabs.add("Productos", crearSeccionProductos());
        tabs.add("Categorías", crearSeccionCategorias());
        tabs.add("Proveedores", crearSeccionProveedores());
        tabs.add("Usuarios", crearSeccionUsuarios());

        add(new H2("Sistema de Inventario - CRUD"), tabs);
    }

    // ========================================
    // 1. PRODUCTOS
    // ========================================
    private Component crearSeccionProductos() {
        VerticalLayout layout = new VerticalLayout();
        TextField idField = new TextField("ID");
        TextField codigoField = new TextField("Código");
        TextField nombreField = new TextField("Nombre");
        TextField categoriaField = new TextField("Categoría");
        TextField precioField = new TextField("Precio");
        TextField stockField = new TextField("Stock");

        FormLayout form = new FormLayout(idField, codigoField, nombreField, categoriaField, precioField-
, stockField);

        Grid<Producto> grid = new Grid<>(Producto.class, false);
        grid.addColumn(Producto::getId).setHeader("ID").setAutoWidth(true);
        grid.addColumn(Producto::getCodigo).setHeader("Código").setAutoWidth(true);
        grid.addColumn(Producto::getNombre).setHeader("Nombre").setAutoWidth(true);
        grid.addColumn(Producto::getCategoria).setHeader("Categoría").setAutoWidth(true);
        grid.addColumn(Producto::getPrecio).setHeader("Precio").setAutoWidth(true);
        grid.addColumn(Producto::getStock).setHeader("Stock").setAutoWidth(true);

        // Cargar registros de la base de datos en la grilla
        Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Producto.class));
        refrescar.run();
```

```
72 |
73 |         // Al hacer clic en una fila, cargar datos en el formulario
74 |         grid.asSingleSelect().addValueChangeListener(e -> {
75 |             Producto p = e.getValue();
76 |             if (p != null) {
77 |                 idField.setValue(p.getId() != null ? p.getId().toString() : "");
78 |                 codigoField.setValue(p.getCodigo() != null ? p.getCodigo() : "");
79 |                 nombreField.setValue(p.getNombre() != null ? p.getNombre() : "");
80 |                 categoriaField.setValue(p.getCategoria() != null ? p.getCategoria() : "");
81 |                 precioField.setValue(p.getPrecio() != null ? p.getPrecio().toString() : "");
82 |                 stockField.setValue(p.getStock() != null ? p.getStock().toString() : "");
83 |             }
84 |         });
85 |
86 |         // Botón CREAR
87 |         Button btnCrear = new Button("Crear", e -> {
88 |             Producto p = new Producto(
89 |                 codigoField.getValue(),
90 |                 nombreField.getValue(),
91 |                 categoriaField.getValue(),
92 |                 parseDouble(precioField.getValue()),
93 |                 parseInt(stockField.getValue())
94 |             );
95 |             crudService.guardar(p);
96 |             refrescar.run();
97 |             idField.setValue(p.getId() != null ? p.getId().toString() : "");
98 |             Notification.show("Producto guardado con ID: " + p.getId());
99 |         });
100|         btnCrear.addThemeVariants(ButtonVariant.LUMO_PRIMARY);
101|
102|         // Botón CONSULTAR
103|         Button btnConsultar = new Button("Consultar", e -> {
104|             Producto p = crudService.buscarPorId(Producto.class, parseId(idField));
105|             if (p != null) {
106|                 grid.asSingleSelect().setValue(p);
107|                 Notification.show("Encontrado: " + p.getNombre());
108|             } else {
109|                 Notification.show("No encontrado");
110|             }
111|         });
112|
113|         // Botón ACTUALIZAR
114|         Button btnActualizar = new Button("Actualizar", e -> {
115|             Producto p = crudService.buscarPorId(Producto.class, parseId(idField));
116|             if (p != null) {
117|                 p.setCodigo(codigoField.getValue());
118|                 p.setNombre(nombreField.getValue());
119|                 p.setCategoria(categoriaField.getValue());
120|                 p.setPrecio(parseDouble(precioField.getValue()));
121|                 p.setStock(parseInt(stockField.getValue()));
122|                 crudService.actualizar(p);
123|                 refrescar.run();
124|                 Notification.show("Producto actualizado");
125|             } else {
126|                 Notification.show("ID no encontrado");
127|             }
128|         });
129|
130|         // Botón ELIMINAR
131|         Button btnEliminar = new Button("Eliminar", e -> {
132|             crudService.eliminar(Producto.class, parseId(idField));
133|             refrescar.run();
134|             limpiar(idField, codigoField, nombreField, categoriaField, precioField, stockField);
135|             Notification.show("Producto eliminado");
136|         });
```

```
137         btnEliminar.addThemeVariants(ButtonVariant.LUMO_ERROR);
138
139         // Botón LIMPIAR
140         Button btnLimpiar = new Button("Limpiar", e -> {
141             limpiar(idField, codigoField, nombreField, categoriaField, precioField, stockField);
142             grid.asSingleSelect().clear();
143         });
144
145         layout.add(form, crearAcciones(btnCrear, btnConsultar, btnActualizar, btnEliminar, btnLimpiar)-
    , grid);
146         return layout;
147     }
148
149     // ========================================
150     // 2. CATEGORÍAS
151     // ========================================
152     private Component crearSeccionCategorias() {
153         VerticalLayout layout = new VerticalLayout();
154         TextField idField = new TextField("ID");
155         TextField nombreField = new TextField("Nombre");
156         TextField descripcionField = new TextField("Descripción");
157
158         FormLayout form = new FormLayout(idField, nombreField, descripcionField);
159
160         Grid<Categoria> grid = new Grid<>(Categoria.class, false);
161         grid.addColumn(Categoria::getId).setHeader("ID").setAutoWidth(true);
162         grid.addColumn(Categoria::getNombre).setHeader("Nombre").setAutoWidth(true);
163         grid.addColumn(Categoria::getDescripcion).setHeader("Descripción").setAutoWidth(true);
164
165         Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Categoria.class));
166         refrescar.run();
167
168         grid.asSingleSelect().addValueChangeListener(e -> {
169             Categoria c = e.getValue();
170             if (c != null) {
171                 idField.setValue(c.getId() != null ? c.getId().toString() : "");
172                 nombreField.setValue(c.getNombre() != null ? c.getNombre() : "");
173                 descripcionField.setValue(c.getDescripcion() != null ? c.getDescripcion() : "");
174             }
175         });
176
177         Button btnCrear = new Button("Crear", e -> {
178             Categoria c = new Categoria(nombreField.getValue(), descripcionField.getValue());
179             crudService.guardar(c);
180             refrescar.run();
181             idField.setValue(c.getId() != null ? c.getId().toString() : "");
182             Notification.show("Categoría guardada con ID: " + c.getId());
183         });
184         btnCrear.addThemeVariants(ButtonVariant.LUMO_PRIMARY);
185
186         Button btnConsultar = new Button("Consultar", e -> {
187             Categoria c = crudService.buscarPorId(Categoria.class, parseId(idField));
188             if (c != null) {
189                 grid.asSingleSelect().setValue(c);
190                 Notification.show("Encontrada: " + c.getNombre());
191             } else {
192                 Notification.show("No encontrada");
193             }
194         });
195
196         Button btnActualizar = new Button("Actualizar", e -> {
197             Categoria c = crudService.buscarPorId(Categoria.class, parseId(idField));
198             if (c != null) {
199                 c.setNombre(nombreField.getValue());
200                 c.setDescripcion(descripcionField.getValue());
201                 crudService.actualizar(c);
```

```
                refrescar.run();
                Notification.show("Categoría actualizada");
            } else {
                Notification.show("ID no encontrado");
            }
        });

        Button btnEliminar = new Button("Eliminar", e -> {
            crudService.eliminar(Categoria.class, parseId(idField));
            refrescar.run();
            limpiar(idField, nombreField, descripcionField);
            Notification.show("Categoría eliminada");
        });
        btnEliminar.addThemeVariants(ButtonVariant.LUMO_ERROR);

        Button btnLimpiar = new Button("Limpiar", e -> {
            limpiar(idField, nombreField, descripcionField);
            grid.asSingleSelect().clear();
        });

        layout.add(form, crearAcciones(btnCrear, btnConsultar, btnActualizar, btnEliminar, btnLimpiar)-
, grid);
        return layout;
    }

    // ========================================
    // 3. PROVEEDORES
    // ========================================
    private Component crearSeccionProveedores() {
        VerticalLayout layout = new VerticalLayout();
        TextField idField = new TextField("ID");
        TextField nombreField = new TextField("Nombre");
        TextField contactoField = new TextField("Contacto");
        TextField telefonoField = new TextField("Teléfono");
        TextField emailField = new TextField("Email");
        TextField direccionField = new TextField("Dirección");

        FormLayout form = new FormLayout(idField, nombreField, contactoField, telefonoField, emailField-
, direccionField);

        Grid<Proveedor> grid = new Grid<>(Proveedor.class, false);
        grid.addColumn(Proveedor::getId).setHeader("ID").setAutoWidth(true);
        grid.addColumn(Proveedor::getNombre).setHeader("Nombre").setAutoWidth(true);
        grid.addColumn(Proveedor::getContacto).setHeader("Contacto").setAutoWidth(true);
        grid.addColumn(Proveedor::getTelefono).setHeader("Teléfono").setAutoWidth(true);
        grid.addColumn(Proveedor::getEmail).setHeader("Email").setAutoWidth(true);
        grid.addColumn(Proveedor::getDireccion).setHeader("Dirección").setAutoWidth(true);

        Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Proveedor.class));
        refrescar.run();

        grid.asSingleSelect().addValueChangeListener(e -> {
            Proveedor p = e.getValue();
            if (p != null) {
                idField.setValue(p.getId() != null ? p.getId().toString() : "");
                nombreField.setValue(p.getNombre() != null ? p.getNombre() : "");
                contactoField.setValue(p.getContacto() != null ? p.getContacto() : "");
                telefonoField.setValue(p.getTelefono() != null ? p.getTelefono() : "");
                emailField.setValue(p.getEmail() != null ? p.getEmail() : "");
                direccionField.setValue(p.getDireccion() != null ? p.getDireccion() : "");
            }
        });

        Button btnCrear = new Button("Crear", e -> {
            Proveedor p = new Proveedor(
                nombreField.getValue(),
```

```
                contactoField.getValue(),
                telefonoField.getValue(),
                emailField.getValue(),
                direccionField.getValue()
            );
            crudService.guardar(p);
            refrescar.run();
            idField.setValue(p.getId() != null ? p.getId().toString() : "");
            Notification.show("Proveedor guardado con ID: " + p.getId());
        });
        btnCrear.addThemeVariants(ButtonVariant.LUMO_PRIMARY);

        Button btnConsultar = new Button("Consultar", e -> {
            Proveedor p = crudService.buscarPorId(Proveedor.class, parseId(idField));
            if (p != null) {
                grid.asSingleSelect().setValue(p);
                Notification.show("Encontrado: " + p.getNombre());
            } else {
                Notification.show("No encontrado");
            }
        });

        Button btnActualizar = new Button("Actualizar", e -> {
            Proveedor p = crudService.buscarPorId(Proveedor.class, parseId(idField));
            if (p != null) {
                p.setNombre(nombreField.getValue());
                p.setContacto(contactoField.getValue());
                p.setTelefono(telefonoField.getValue());
                p.setEmail(emailField.getValue());
                p.setDireccion(direccionField.getValue());
                crudService.actualizar(p);
                refrescar.run();
                Notification.show("Proveedor actualizado");
            } else {
                Notification.show("ID no encontrado");
            }
        });

        Button btnEliminar = new Button("Eliminar", e -> {
            crudService.eliminar(Proveedor.class, parseId(idField));
            refrescar.run();
            limpiar(idField, nombreField, contactoField, telefonoField, emailField, direccionField);
            Notification.show("Proveedor eliminado");
        });
        btnEliminar.addThemeVariants(ButtonVariant.LUMO_ERROR);

        Button btnLimpiar = new Button("Limpiar", e -> {
            limpiar(idField, nombreField, contactoField, telefonoField, emailField, direccionField);
            grid.asSingleSelect().clear();
        });

        layout.add(form, crearAcciones(btnCrear, btnConsultar, btnActualizar, btnEliminar, btnLimpiar)-
, grid);
        return layout;
    }

    // ========================================
    // 4. USUARIOS
    // ========================================
    private Component crearSeccionUsuarios() {
        VerticalLayout layout = new VerticalLayout();
        TextField idField = new TextField("ID");
        TextField nombreField = new TextField("Nombre");
        TextField apellidoField = new TextField("Apellido");
        TextField emailField = new TextField("Email");
        TextField telefonoField = new TextField("Teléfono");
```

```
        TextField direccionField = new TextField("Dirección");
        TextField rolField = new TextField("Rol");
        TextField estadoField = new TextField("Estado");

        FormLayout form = new FormLayout(idField, nombreField, apellidoField, emailField, telefonoField-
, direccionField, rolField, estadoField);

        Grid<Usuario> grid = new Grid<>(Usuario.class, false);
        grid.addColumn(Usuario::getId).setHeader("ID").setAutoWidth(true);
        grid.addColumn(Usuario::getNombre).setHeader("Nombre").setAutoWidth(true);
        grid.addColumn(Usuario::getApellido).setHeader("Apellido").setAutoWidth(true);
        grid.addColumn(Usuario::getEmail).setHeader("Email").setAutoWidth(true);
        grid.addColumn(Usuario::getTelefono).setHeader("Teléfono").setAutoWidth(true);
        grid.addColumn(Usuario::getRol).setHeader("Rol").setAutoWidth(true);
        grid.addColumn(Usuario::getEstado).setHeader("Estado").setAutoWidth(true);

        Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Usuario.class));
        refrescar.run();

        grid.asSingleSelect().addValueChangeListener(e -> {
            Usuario u = e.getValue();
            if (u != null) {
                idField.setValue(u.getId() != null ? u.getId().toString() : "");
                nombreField.setValue(u.getNombre() != null ? u.getNombre() : "");
                apellidoField.setValue(u.getApellido() != null ? u.getApellido() : "");
                emailField.setValue(u.getEmail() != null ? u.getEmail() : "");
                telefonoField.setValue(u.getTelefono() != null ? u.getTelefono() : "");
                direccionField.setValue(u.getDireccion() != null ? u.getDireccion() : "");
                rolField.setValue(u.getRol() != null ? u.getRol() : "");
                estadoField.setValue(u.getEstado() != null ? u.getEstado() : "");
            }
        });

        Button btnCrear = new Button("Crear", e -> {
            Usuario u = new Usuario(
                nombreField.getValue(),
                apellidoField.getValue(),
                emailField.getValue(),
                telefonoField.getValue(),
                direccionField.getValue(),
                rolField.getValue(),
                estadoField.getValue(),
                LocalDateTime.now()
            );
            crudService.guardar(u);
            refrescar.run();
            idField.setValue(u.getId() != null ? u.getId().toString() : "");
            Notification.show("Usuario guardado con ID: " + u.getId());
        });
        btnCrear.addThemeVariants(ButtonVariant.LUMO_PRIMARY);

        Button btnConsultar = new Button("Consultar", e -> {
            Usuario u = crudService.buscarPorId(Usuario.class, parseId(idField));
            if (u != null) {
                grid.asSingleSelect().setValue(u);
                Notification.show("Encontrado: " + u.getNombre() + " " + u.getApellido());
            } else {
                Notification.show("No encontrado");
            }
        });

        Button btnActualizar = new Button("Actualizar", e -> {
            Usuario u = crudService.buscarPorId(Usuario.class, parseId(idField));
            if (u != null) {
                u.setNombre(nombreField.getValue());
                u.setApellido(apellidoField.getValue());
```

```
                u.setEmail(emailField.getValue());
                u.setTelefono(telefonoField.getValue());
                u.setDireccion(direccionField.getValue());
                u.setRol(rolField.getValue());
                u.setEstado(estadoField.getValue());
                crudService.actualizar(u);
                refrescar.run();
                Notification.show("Usuario actualizado");
            } else {
                Notification.show("ID no encontrado");
            }
        });

        Button btnEliminar = new Button("Eliminar", e -> {
            crudService.eliminar(Usuario.class, parseId(idField));
            refrescar.run();
            limpiar(idField, nombreField, apellidoField, emailField, telefonoField, direccionField,-
 rolField, estadoField);
            Notification.show("Usuario eliminado");
        });
        btnEliminar.addThemeVariants(ButtonVariant.LUMO_ERROR);

        Button btnLimpiar = new Button("Limpiar", e -> {
            limpiar(idField, nombreField, apellidoField, emailField, telefonoField, direccionField,-
 rolField, estadoField);
            grid.asSingleSelect().clear();
        });

        layout.add(form, crearAcciones(btnCrear, btnConsultar, btnActualizar, btnEliminar, btnLimpiar)-
, grid);
        return layout;
    }

    // ========================================
    // MÉTODOS AUXILIARES
    // ========================================
    private Long parseId(TextField field) {
        try {
            return Long.parseLong(field.getValue().trim());
        } catch (Exception e) {
            return null;
        }
    }

    private Double parseDouble(String val) {
        try {
            return Double.parseDouble(val.trim());
        } catch (Exception e) {
            return 0.0;
        }
    }

    private Integer parseInt(String val) {
        try {
            return Integer.parseInt(val.trim());
        } catch (Exception e) {
            return 0;
        }
    }

    private void limpiar(TextField... campos) {
        for (TextField c : campos) {
            c.clear();
        }
    }

```

```
459     private HorizontalLayout crearAcciones(Button... botones) {
460         HorizontalLayout layout = new HorizontalLayout(botones);
461         layout.getStyle().set("flex-wrap", "wrap");
462         return layout;
463     }
464 }
```

4. Claves del Funcionamiento de la Vista

1. **Anotación `@Route("")`**:

Indica que esta vista responderá en la ruta raíz del navegador (`http://localhost:8080/`).

1. **Inyección de `CrudService`**:

Al declarar `public MainView(CrudService crudService)`, Spring busca automáticamente el componente de servicio registrado y se lo entrega a la vista listo para ser usado.

1. **La función `refrescar` (`Runnable`)**:

JAVA

```java
Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Producto.class));
refrescar.run();
```

Esta pequeña expresión lambda se ejecuta al cargar la pestaña y vuelve a llamarse después de cada inserción, actualización o eliminación para mantener la tabla siempre sincronizada con la base de datos PostgreSQL.

1. **Selección interactiva en la Grilla**:

JAVA

```java
grid.asSingleSelect().addValueChangeListener(e -> { ... });
```

Permite que cuando el usuario haga clic sobre cualquier fila de la tabla, los datos de ese registro se trasladen inmediatamente a las cajas de texto del formulario. Esto hace que editar o eliminar sea una experiencia ágil y natural.

CAPÍTULO 4 • LECCIÓN 4.7

06_EJECUCION_Y_PRUEBAS

Capítulo 6: Ejecución, Pruebas y Validación

En este último capítulo aprenderás cómo compilar y arrancar la aplicación, qué esperar en la consola del servidor, cómo realizar las pruebas de las operaciones CRUD en el navegador y cómo verificar los datos directamente en PostgreSQL.

---

1. ¿Cómo Iniciar la Aplicación?

Asegúrate de que el servicio de **PostgreSQL esté activo** y de haber creado la base de datos `inventario_db`.

Opción A: Desde la terminal con el Maven Wrapper (Recomendado)

Abre tu consola en la carpeta raíz del proyecto y ejecuta:

- **En Windows**:

```cmd
.\mvnw.cmd spring-boot:run
```

- **En Linux o macOS**:

```bash
./mvnw spring-boot:run
```

Opción B: Desde tu IDE (VS Code / IntelliJ / Eclipse)

1. Abre el archivo **Application.java**.
2. Haz clic derecho y selecciona **Run Java** (o pulsa el botón de reproducción ¶ "Run" / "Debug").

---

2. ¿Qué esperar en la Consola del Servidor?

Al iniciar la aplicación por primera vez, verás una secuencia de eventos como la siguiente en la consola:

1. **Banner de Spring Boot**:

```text
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v4.1.1)
```

1. **Creación Automática de Tablas por Hibernate (`ddl-auto=update`)**:

Hibernate detecta las entidades y envía las instrucciones DDL a PostgreSQL:

```sql
create table productos (
    id bigint generated by default as identity,
    categoria varchar(255),
    codigo varchar(255),
    nombre varchar(255),
    precio float8,
    stock integer,
    primary key (id)
);
create table categorias (
    id bigint generated by default as identity,
    descripcion varchar(255),
    nombre varchar(255),
    primary key (id)
);
create table proveedores ( ... );
create table usuarios ( ... );
```

1. **Confirmación de Inicio de Vaadin**:

```text
INFO: Started Application in 15.038 seconds
INFO: Application running at http://localhost:8080/
```

Tu navegador web se abrirá automáticamente (o puedes ingresar manualmente a **http://localhost:8080**).

---

3. Pruebas Funcionales en el Navegador

Una vez en la aplicación web, realiza las siguientes pruebas en cada pestaña:

3.1. Probar "Crear" (Insertar)

1. En la pestaña **Productos**, llena los campos:
2. Código: `PROD-001`
3. Nombre: `Laptop Lenovo ThinkPad`
4. Categoría: `Electrónica`
5. Precio: `3500000`
6. Stock: `12`
7. Haz clic en el botón azul **Crear**.
8. **Resultado**:
9. Aparecerá una notificación emergente: `Producto guardado con ID: 1`.
10. La tabla inferior se actualizará inmediatamente mostrando el nuevo producto con su ID generado por PostgreSQL.

3.2. Probar "Selección y Consulta" (Leer)

1. Haz clic directamente sobre la fila que acabas de crear en la tabla.
2. Observa cómo todos los datos de esa fila se cargan de inmediato en las cajas de texto del formulario.
3. Alternativamente, escribe el número `1` en la caja **ID** y pulsa el botón **Consultar**: la fila quedará seleccionada y se notificará que fue encontrado.

3.3. Probar "Actualizar" (Modificar)

1. Con los datos cargados en el formulario, modifica el campo **Stock** de `12` a `20` y el **Precio** a `3400000`.
2. Haz clic en el botón **Actualizar**.
3. **Resultado**:
4. Se mostrará el mensaje `Producto actualizado`.
5. La grilla reflejará el nuevo precio y stock al instante.

3.4. Probar "Eliminar" (Borrar)

1. Ingresa el ID `1` en el campo correspondiente (o selecciona la fila).
2. Haz clic en el botón rojo **Eliminar**.
3. **Resultado**:
4. Aparecerá la notificación `Producto eliminado`.
5. La fila desaparecerá de la grilla y el formulario se limpiará.

---

4. Verificación Directa en PostgreSQL

Para tener certeza absoluta de que los datos están almacenados en el disco duro por PostgreSQL, abre tu terminal y consulta la base de datos:

```sql
psql -U postgres -d inventario_db
```

Ejecuta las siguientes consultas SQL:

```sql
-- Listar las tablas creadas por Hibernate:
\dt

-- Consultar los productos insertados:
SELECT * FROM productos;

-- Consultar las categorías:
SELECT * FROM categorias;

-- Consultar los proveedores:
SELECT * FROM proveedores;

-- Consultar los usuarios:
SELECT * FROM usuarios;
```

5. Solución de Problemas Frecuentes (Troubleshooting)

A. Error: `Connection to localhost:5432 refused`

- **Causa**: El servicio de PostgreSQL no está en ejecución.
- **Solución**: En Windows, abre la aplicación **Servicios** (`services.msc`), busca **postgresql-x64-XX** y haz clic en **Iniciar**. En Linux/Mac: `sudo systemctl start postgresql` o `brew services start postgresql`.

B. Error: `FATAL: password authentication failed for user "postgres"`

- **Causa**: La contraseña configurada en `src/main/resources/application.properties` no coincide con la contraseña que definiste al instalar PostgreSQL.
- **Solución**: Abre **application.properties** y actualiza la línea `spring.datasource.password=tu_contraseña_real`.

C. Error: `database "inventario_db" does not exist`

- **Causa**: No creaste la base de datos antes de iniciar la app.
- **Solución**: Abre psql o pgAdmin y ejecuta `CREATE DATABASE inventario_db;`.

D. Error: `Web server failed to start. Port 8080 was already in use.`

- **Causa**: Ya hay otra aplicación o instancia anterior corriendo en el puerto 8080.
- **Solución**:
- Cierra la terminal o proceso que esté usando el puerto, o bien:
- Cambia el puerto en **application.properties**: `server.port=8081`.

¡Felicitaciones! Has completado y puesto en marcha un sistema de inventario empresarial profesional, limpio y escalable con Spring Boot, Vaadin Flow, Hibernate y PostgreSQL.

CAPÍTULO 4 • LECCIÓN 4.8

07_GUIA_ADAPTACION_NUEVAS_IDEAS_DE_NEGOCIO

Capítulo 7: Cómo Convertir este Proyecto en Cualquier Otra Idea de Negocio

Este capítulo es una **guía práctica para estudiantes**. Aquí aprenderás cómo tomar este proyecto base de inventario y transformarlo en el **Proyecto Integrador** que tú o tu equipo hayan elegido (por ejemplo: una clínica veterinaria, un gimnasio, un hotel, un taller mecánico, una biblioteca, un restaurante o cualquier otra idea).

1. La Gran Ventaja de esta Arquitectura

Gracias a cómo construimos este proyecto, **más del 70% del código no necesita modificarse**:

- **Maven (`pom.xml`)**: Ya tiene todas las librerías necesarias (Spring Boot, Vaadin, Hibernate, PostgreSQL). **No tienes que cambiar nada**.
- **La conexión a base de datos (`application.properties`)**: Solo cambias el nombre de la base de datos si lo deseas.
- **El servicio CRUD (`CrudService.java`)**: Al ser un servicio genérico (`<T>`), **¡NO TIENES QUE TOCARLO!** Funciona automáticamente para cualquier entidad nueva que crees.
- **El arranque (`Application.java`)**: Queda exactamente igual.

> **CONSEJO PRÁCTICO**
> **TIP Tu único trabajo como estudiante consiste en:** 1. Pensar las entidades de tu negocio (3 o 4 clases en `com.example.model`). 2. Diseñar los formularios y grillas en `MainView.java` para mostrar tus datos.

2. Los 5 Pasos para Transformar el Proyecto

JAVASCRIPT

```
+-----------------------------------------------------------------+
| PASO 1: Define las 3 o 4 entidades de tu idea de negocio       |
+-----------------------------------------------------------------+
                                ¼
+-----------------------------------------------------------------+
| PASO 2: Crea las clases en el paquete com.example.model         |
|         (Anotadas con @Entity, @Table, @Id, @GeneratedValue)    |
+-----------------------------------------------------------------+
                                ¼
+-----------------------------------------------------------------+
| PASO 3: (Opcional) Cambia el nombre de la BD en properties      |
+-----------------------------------------------------------------+
                                ¼
+-----------------------------------------------------------------+
| PASO 4: Adapta las pestañas, formularios y grillas en MainView  |
+-----------------------------------------------------------------+
```

```
                                ¼
+-------------------------------------------------------------------+
| PASO 5: Inicia el proyecto y comprueba la creación automática     |
+-------------------------------------------------------------------+
```

3. Ejemplo Completo Paso a Paso: Transformación a "Clínica Veterinaria"

Para que veas exactamente qué debes hacer, vamos a transformar el sistema de inventario en un **Sistema de Gestión de Clínica Veterinaria**.

Paso 1: Definir las Entidades del Negocio

En una veterinaria necesitamos gestionar:

1. **Mascota**: El paciente animal (nombre, especie, raza, edad, nombre del dueño).
2. **Veterinario**: El médico profesional (nombre, especialidad, teléfono, correo).
3. **CitaMedica**: La consulta médica (fecha, motivo, diagnóstico, costo).
4. **Usuario**: El personal que inicia sesión (administrador, recepcionista).

Paso 2: Crear las Clases en `com.example.model`

Por ejemplo, eliminamos `Producto.java` y creamos en su lugar `Mascota.java`:

Archivo: `src/main/java/com/example/model/Mascota.java`

JAVA

```java
package com.example.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "mascotas") // Hibernate creará la tabla 'mascotas' en PostgreSQL
public class Mascota {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String especie; // 'Perro', 'Gato', etc.
    private String raza;
    private Integer edad;
    private String nombreDueno;
    private String telefonoContacto;

    // 1. Constructor vacío (Obligatorio para Hibernate)
    public Mascota() {
    }

    // 2. Constructor sin ID (Para crear nuevas mascotas)
    public Mascota(String nombre, String especie, String raza, Integer edad, String nombreDueno, String-
telefonoContacto) {
```

```
        this(null, nombre, especie, raza, edad, nombreDueno, telefonoContacto);
    }

    // 3. Constructor completo
    public Mascota(Long id, String nombre, String especie, String raza, Integer edad, String nombreDueno-
, String telefonoContacto) {
        this.id = id;
        this.nombre = nombre;
        this.especie = especie;
        this.raza = raza;
        this.edad = edad;
        this.nombreDueno = nombreDueno;
        this.telefonoContacto = telefonoContacto;
    }

    // 4. Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getEspecie() { return especie; }
    public void setEspecie(String especie) { this.especie = especie; }

    public String getRaza() { return raza; }
    public void setRaza(String raza) { this.raza = raza; }

    public Integer getEdad() { return edad; }
    public void setEdad(Integer edad) { this.edad = edad; }

    public String getNombreDueno() { return nombreDueno; }
    public void setNombreDueno(String nombreDueno) { this.nombreDueno = nombreDueno; }

    public String getTelefonoContacto() { return telefonoContacto; }
    public void setTelefonoContacto(String telefonoContacto) { this.telefonoContacto = telefonoContacto; }
}
```

Paso 3: Ajustar el Nombre de la Base de Datos en `application.properties`

Abre `src/main/resources/application.properties` y cambia el nombre de la base de datos:

PROPERTIES

```
# Cambiamos 'inventario_db' por 'veterinaria_db'
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/veterinaria_db}
spring.datasource.username=${DB_USERNAME:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
```

Luego en tu PostgreSQL creas esa base de datos:

SQL

```
CREATE DATABASE veterinaria_db;
```

Paso 4: Adaptar `MainView.java` a la Nueva Entidad

En `MainView.java`:

1. Cambias el título de la pestaña a `"Mascotas"`.
2. Creas el método `crearSeccionMascotas()`. Observa lo sencillo que resulta:

JAVA

```java
// ========================================
// SECCIÓN MASCOTAS (CLÍNICA VETERINARIA)
// ========================================
private Component crearSeccionMascotas() {
    VerticalLayout layout = new VerticalLayout();

    // 1. Campos del formulario
    TextField idField = new TextField("ID");
    TextField nombreField = new TextField("Nombre de la Mascota");
    TextField especieField = new TextField("Especie (Perro, Gato...)");
    TextField razaField = new TextField("Raza");
    TextField edadField = new TextField("Edad (Años)");
    TextField duenoField = new TextField("Nombre del Dueño");
    TextField telefonoField = new TextField("Teléfono de Contacto");

    FormLayout form = new FormLayout(idField, nombreField, especieField, razaField, edadField,-
 duenoField, telefonoField);

    // 2. Grilla con las columnas de Mascota
    Grid<Mascota> grid = new Grid<>(Mascota.class, false);
    grid.addColumn(Mascota::getId).setHeader("ID").setAutoWidth(true);
    grid.addColumn(Mascota::getNombre).setHeader("Nombre").setAutoWidth(true);
    grid.addColumn(Mascota::getEspecie).setHeader("Especie").setAutoWidth(true);
    grid.addColumn(Mascota::getRaza).setHeader("Raza").setAutoWidth(true);
    grid.addColumn(Mascota::getEdad).setHeader("Edad").setAutoWidth(true);
    grid.addColumn(Mascota::getNombreDueno).setHeader("Dueño").setAutoWidth(true);
    grid.addColumn(Mascota::getTelefonoContacto).setHeader("Teléfono").setAutoWidth(true);

    // 3. Función para refrescar datos desde PostgreSQL usando CrudService
    Runnable refrescar = () -> grid.setItems(crudService.listarTodos(Mascota.class));
    refrescar.run();

    // 4. Al seleccionar una mascota en la grilla, cargarla en el formulario
    grid.asSingleSelect().addValueChangeListener(e -> {
        Mascota m = e.getValue();
        if (m != null) {
            idField.setValue(m.getId() != null ? m.getId().toString() : "");
            nombreField.setValue(m.getNombre() != null ? m.getNombre() : "");
            especieField.setValue(m.getEspecie() != null ? m.getEspecie() : "");
            razaField.setValue(m.getRaza() != null ? m.getRaza() : "");
            edadField.setValue(m.getEdad() != null ? m.getEdad().toString() : "");
            duenoField.setValue(m.getNombreDueno() != null ? m.getNombreDueno() : "");
            telefonoField.setValue(m.getTelefonoContacto() != null ? m.getTelefonoContacto() : "");
        }
    });

    // 5. Botón CREAR
    Button btnCrear = new Button("Crear", e -> {
        Mascota m = new Mascota(
            nombreField.getValue(),
            especieField.getValue(),
            razaField.getValue(),
            parseInt(edadField.getValue()),
            duenoField.getValue(),
            telefonoField.getValue()
        );
        crudService.guardar(m); // ¡Se guarda en PostgreSQL con Hibernate!
```

```
        refrescar.run();
        idField.setValue(m.getId() != null ? m.getId().toString() : "");
        Notification.show("Mascota registrada con ID: " + m.getId());
    });
    btnCrear.addThemeVariants(ButtonVariant.LUMO_PRIMARY);

    // 6. Botón CONSULTAR por ID
    Button btnConsultar = new Button("Consultar", e -> {
        Mascota m = crudService.buscarPorId(Mascota.class, parseId(idField));
        if (m != null) {
            grid.asSingleSelect().setValue(m);
            Notification.show("Mascota encontrada: " + m.getNombre());
        } else {
            Notification.show("No encontrada");
        }
    });

    // 7. Botón ACTUALIZAR
    Button btnActualizar = new Button("Actualizar", e -> {
        Mascota m = crudService.buscarPorId(Mascota.class, parseId(idField));
        if (m != null) {
            m.setNombre(nombreField.getValue());
            m.setEspecie(especieField.getValue());
            m.setRaza(razaField.getValue());
            m.setEdad(parseInt(edadField.getValue()));
            m.setNombreDueno(duenoField.getValue());
            m.setTelefonoContacto(telefonoField.getValue());
            crudService.actualizar(m);
            refrescar.run();
            Notification.show("Mascota actualizada correctamente");
        } else {
            Notification.show("ID no encontrado");
        }
    });

    // 8. Botón ELIMINAR
    Button btnEliminar = new Button("Eliminar", e -> {
        crudService.eliminar(Mascota.class, parseId(idField));
        refrescar.run();
        limpiar(idField, nombreField, especieField, razaField, edadField, duenoField, telefonoField);
        Notification.show("Mascota eliminada");
    });
    btnEliminar.addThemeVariants(ButtonVariant.LUMO_ERROR);

    // 9. Botón LIMPIAR
    Button btnLimpiar = new Button("Limpiar", e -> {
        limpiar(idField, nombreField, especieField, razaField, edadField, duenoField, telefonoField);
        grid.asSingleSelect().clear();
    });

    layout.add(form, crearAcciones(btnCrear, btnConsultar, btnActualizar, btnEliminar, btnLimpiar),-
  grid);
    return layout;
}
```

¡Y listo! Al iniciar la aplicación, Hibernate creará la tabla `mascotas` en PostgreSQL y la interfaz web te permitirá gestionar mascotas en tiempo real.

---

4. Catálogo de Ideas de Proyecto Integrador Listas para Usar

A continuación tienes 5 ideas de negocio habituales en proyectos integradores, con las entidades y atributos recomendados para que tú y tu equipo solo tengan que aplicarlos:

Idea 1: Sistema para Gimnasio y Fitness Center

- **`Miembro`**: `id`, `documento`, `nombreCompleto`, `telefono`, `email`, `estado` (Activo/Inactivo).
- **`PlanMembresia`**: `id`, `nombrePlan` (Mensual, Trimestral, Anual), `precio`, `duracionDias`, `beneficios`.
- **`Entrenador`**: `id`, `nombre`, `especialidad` (Pesas, Cardio, Yoga), `telefono`, `horarioTurno`.
- **`Usuario`**: `id`, `nombre`, `email`, `rol` (Recepcionista, Administrador), `estado`.

Idea 2: Sistema para Taller Mecánico / Automotriz

- **`Vehiculo`**: `id`, `placa`, `marca`, `modelo`, `anio`, `color`, `nombrePropietario`.
- **`Cliente`**: `id`, `cedula`, `nombre`, `telefono`, `email`, `direccion`.
- **`Mecanico`**: `id`, `nombre`, `especialidad` (Frenos, Motor, Eléctrico), `telefono`.
- **`OrdenServicio`**: `id`, `fechaIngreso`, `fallaReportada`, `diagnostico`, `costoTotal`, `estado` (Pendiente, En Reparación, Entregado).

Idea 3: Sistema de Gestión Hotelera / Hospedaje

- **`Habitacion`**: `id`, `numeroHabitacion`, `tipo` (Sencilla, Doble, Suite), `precioNoche`, `estado` (Disponible, Ocupada, Mantenimiento).
- **`Huesped`**: `id`, `documentoIdentidad`, `nombreCompleto`, `nacionalidad`, `telefono`, `email`.
- **`Reserva`**: `id`, `fechaCheckIn`, `fechaCheckOut`, `numeroHuespedes`, `totalPagar`, `estadoPago`.
- **`ServicioAdicional`**: `id`, `nombreServicio` (Desayuno buffet, Lavandería, Spa), `precio`.

Idea 4: Sistema de Restaurante y Menú Digital

- **`Plato`**: `id`, `codigoMenu`, `nombre`, `categoria` (Entrada, Fuerte, Postre, Bebida), `precio`, `disponible` (Sí/No).
- **`Mesa`**: `id`, `numeroMesa`, `capacidadPersonas`, `ubicacion` (Terraza, Salón principal), `estado` (Libre, Ocupada, Reservada).
- **`Mesero`**: `id`, `nombre`, `telefono`, `turno` (Mañana, Tarde, Noche).
- **`Pedido`**: `id`, `numeroMesa`, `descripcionPlatos`, `totalVenta`, `horaPedido`, `estado` (En cocina, Servido, Pagado).

Idea 5: Sistema de Biblioteca / Préstamos de Libros

- **`Libro`**: `id`, `isbn`, `titulo`, `autor`, `categoria` (Ciencia, Novela, Historia), `ejemplaresDisponibles`.
- **`Lector`**: `id`, `codigoEstudiante`, `nombreCompleto`, `carrera`, `telefono`, `email`.
- **`Prestamo`**: `id`, `fechaPrestamo`, `fechaDevolucionLimite`, `estadoPrestamo` (Prestado, Devuelto, Vencido).
- **`Bibliotecario`**: `id`, `nombre`, `correo`, `rol`.

---

5. Lista de Verificación (Checklist) para el Estudiante

Antes de entregar o presentar tu proyecto integrador, asegúrate de haber completado:

- [ ] Elegir la idea de negocio y definir los nombres de las 3 o 4 entidades.

- [ ] Crear las clases Java dentro de `com.example.model` con las anotaciones JPA (`@Entity`, `@Table`, `@Id`, `@GeneratedValue`).
- [ ] Asegurarte de que cada clase tenga un constructor vacío y sus getters/setters.
- [ ] Verificar que `CrudService.java` esté en `com.example.service` (no requiere cambios).
- [ ] Adaptar las pestañas y formularios de `MainView.java` para que muestren los campos de tus nuevas entidades.
- [ ] Crear la base de datos en PostgreSQL con el nombre que colocaste en `application.properties`.
- [ ] Ejecutar el proyecto con `./mvnw.cmd spring-boot:run` y comprobar que Hibernate crea las tablas automáticamente.
- [ ] Probar desde el navegador (`http://localhost:8080`) que funcionen las 4 operaciones (Crear, Consultar, Actualizar y Eliminar) para cada pestaña.
- [ ] Verificar con `psql` o pgAdmin que los datos se guardan en las tablas de PostgreSQL.